Model- and Data-Based Approaches to Wind Compensation for Airship Drones

Alexander Niclas Soukup

Master's Thesis

2026

Supervisor: Sebastian Gieray
M. Sc.

Examiner: Walter Fichter, Univ.-Prof. Dr.-Ing.



University of Stuttgart
Institute of Flight Mechanics and Controls

Erklärungen für Abschlussarbeiten am iFR

Erklärungen bei Ausgabe des Themas der Arbeit

Mir ist bekannt, dass die Arbeit im Online-Katalog der Bibliothek und auf der Institutswebseite erfasst wird, was eine dauerhafte, weltweite Sichtbarkeit der bibliographischen Daten der Arbeit zur Folge hat.

☐ Ich stimme der Nennung meines Namens und einer Kurzfassung der Arbeit zu diesem Zweck zu.

☐ Ich räume der Universität Stuttgart, Institut für Flugmechanik und Flugregelung, an dieser Arbeit und den im Rahmen dieser Arbeit entstandenen Ergebnissen ein kostenloses, zeitlich und räumlich unbeschränktes, einfaches Nutzungsrecht ein. Dieses erstreckt sich auf sämtliche bekannte Nutzungsarten und umfasst neben dem Recht auf Nutzung in Forschung, Lehre und Studium insbesondere das Recht der Vervielfältigung, Verbreitung und Änderung sowie das Recht der öffentlichen Zugänglichmachung.

Ort, Datum	Alexander Niclas Soukup

Erklärungen bei Abgabe der Arbeit

Hiermit versichere ich, dass ich diese Arbeit selbständig mit Unterstützung der Betreuer angefertigt und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Fremde Beiträge habe ich als solche gekennzeichnet. Die Arbeit oder wesentliche Bestandteile davon sind weder an dieser noch an einer anderen Bildungseinrichtung bereits Gegenstand einer Prüfungsleistung gewesen und wurden nicht ohne Zustimmung der Prüferin bzw. des Prüfers bereits veröffentlicht. Ich erkläre weiterhin, bei der Erstellung der Arbeit die einschlägigen Bestimmungen entsprechend den Regeln guter wissenschaftlicher Praxis[1] eingehalten zu haben. Ich versichere, dass alle gedruckten Exemplare mit der digitalen Abgabe der Arbeit übereinstimmen.

Sollte ich der Universität Stuttgart, Institut für Flugmechanik und Flugregelung, Nutzungsrechte an dieser Arbeit und den im Rahmen dieser Arbeit entstandenen Ergebnissen eingeräumt haben, versichere ich, eventuell erforderlich gewordene Zustimmungen der Urheber zur Nutzung fremder Beiträge in meiner Arbeit eingeholt zu haben. Ich erkläre mich für den Fall der Verletzung der Rechte Dritter durch meine Arbeit bereit, der Universität Stuttgart einen daraus entstehenden Schaden zu ersetzen bzw. die Universität Stuttgart auf deren Aufforderung von eventuellen Ansprüchen Dritter freizustellen. Zusätzlich zu den vom Prüfer geforderten Exemplaren, übertrage ich der Universität Stuttgart das Eigentum an einem von mir der Bibliothek des Instituts für Flugmechanik und Flugregelung kostenlos zur Verfügung gestellten Exemplar dieser Arbeit.

[1] Siehe "Leitlinien zur Sicherung guter wissenschaftlicher Praxis" der DFG bzw. "Satzung der Universität Stuttgart zur Sicherung der Integrität wissenschaftlicher Praxis und zum Umgang mit Fehlverhalten in der Wissenschaft".

Ort, Datum

Alexander Niclas Soukup

Abstract

In here, you should briefly describe the thesis content in the primary language.

Kurzfassung

In here, you should briefly describe the thesis content in the secondary language.

Contents

Erklärungen iii

Kurzfassung / Abstract v

List of Figures ix

List of Tables xi

Nomenclature xiii

1. Introduction 1
- 1.1. Aim of this work . . . 2
- 1.2. State of the art / Related work . . . 2
- 1.3. Airship squid . . . 3
  - 1.3.1. Airship Configuration . . . 3
  - 1.3.2. Control Architecture . . . 4

2. Fundamentals 5
- 2.1. Coordinate Systems . . . 5
  - 2.1.1. Body-Fixed Coordinate System . . . 5
  - 2.1.2. Earth-Fixed Coordinate System . . . 5
- 2.2. Airship Model . . . 6
  - 2.2.1. Gravitation and Buoyancy . . . 9
  - 2.2.2. Aerodynamics . . . 9
  - 2.2.3. Propulsion . . . 13
- 2.3. Wind Model . . . 13
  - 2.3.1. Dryden Turbulence Model . . . 14
- 2.4. Kalman Filter . . . 16
  - 2.4.1. Extended Kalman Filter . . . 21
  - 2.4.2. Unscented Kalman Filter . . . 23
- 2.5. Disturbance Compensation . . . 26
  - 2.5.1. Disturbance Feedforward . . . 26
  - 2.5.2. Nonlinear Dynamic Inversion . . . 27
- 2.6. Artificial Neural Networks . . . 28
  - 2.6.1. Multi-Layer Perceptron . . . 29
  - 2.6.2. Long Short-Term Memory Networks . . . 30

3. model-based wind estimation and compensation 33
- 3.1. Kalman Filter . . . 33
  - 3.1.1. State-Space Description . . . 33
  - 3.1.2. Including the Wind Model . . . 36
  - 3.1.3. Airspeed Sensor . . . 38
  - 3.1.4. Discretization . . . 38

3.1.5. Extended Kalman Filter Implementation . . . 40
3.1.6. Unscented Kalman Filter Implementation . . . 41
3.1.7. Filter Initialization . . . 42
3.2. Compensation . . . 42
3.2.1. Disturbance Feedforward . . . 44
3.2.2. Nonlinear Dynamic Inversion . . . 45

4. data-driven wind estimation and compensation **47**
4.1. Problem Formulation . . . 47
4.2. Data Set . . . 48
4.2.1. Data Splitting . . . 48
4.2.2. Preprocessing . . . 48
4.2.3. Input Generation . . . 49
4.3. Model Architectures . . . 49
4.4. Training Procedure . . . 51

5. Evaluation and results **53**
5.1. Model based Wind Estimation . . . 54
5.1.1. Selection of Reference Wind Speed . . . 54
5.1.2. UKF vs EKF . . . 56
5.1.3. Computational Cost . . . 63
5.1.4. Effect of Airspeed Sensor . . . 64
5.1.5. Mean Wind Computation . . . 65
5.2. Data Driven Wind Estimation . . . 66
5.2.1. Robustness to Noisy Inputs . . . 69
5.2.2. Computational Cost . . . 70
5.3. Wind compensation . . . 72

6. Evaluation with Real Flight Data **77**
6.1. Test Flight Setup . . . 77
6.1.1. Test Flight and Logged Data . . . 77
6.1.2. Wind Reference Measurement . . . 77
6.2. Offline Evaluation Procedure . . . 77

7. Conclusion and Outlook **79**

A. First Appendix **81**

B. Second Appendix **83**
B.1. W20 . . . 83
B.2. Wind estimation . . . 83

List of Figures

1.1. CAD model of the Airship Squid and its components. 3

2.1. Airship with body-fixed coordinate system B and earth-fixed coordinate system E. The orange dot marks the Center of Gravity and the blue dot marks the Center of Volume (COV), connected by the position vector $\boldsymbol{r}_{COV}$. 6
2.2. Comparison of wind turbulence created by the random walk, von Kármán and Dryden model for identical white noise input. 14
2.3. Structure of the Dryden Turbulence model. 15
2.4. Structure of a compensation-based control approach. Both the controller and the compensation module may additionally use available system outputs of the airship. For clarity, the corresponding signal paths are omitted. 26
2.5. Schematic illustration of a multi-layer perceptron with n inputs, three fully connected hidden layers, and m outputs. Each circle represents a single neuron, computing a weighted sum of its inputs followed by a nonlinear activation, as defined in (2.115). 30

3.1. Block diagram of the control structure, combining PID and compensation. 44

4.1. Training and validation loss of the MLP on the Experiment A training set. The moving average training loss is computed over the last 50 samples for improved readability. . . . 52

5.1. Comparison of RMSE, averaged over three axes and 27 wind scenarios for varying assumed reference wind speeeds $W_{20,\text{ass}}$. The orange bar highlights the true configuration scenario where $W_{20,\text{ass}} = W_{20,\text{true}}$. 55
5.2. Comparison of wind estimation performance for different assumed reference wind speeds $W_{20,\text{ass}}$ and $W_{20,\text{true}} = 5$. 56
5.3. Actual and estimated wind for Curve flight scenario, constant wind component $\boldsymbol{v}_{W,\text{const}} = [2.5 - 1 - 0.5]\text{m/s}$ and reference wind speed $W_{20,\text{true}} = 10\,\text{m/s}$ 57
5.4. Distribution of the per-run wind estimation RMSE for the EKF and UKF across the investigated test suite. 58
5.5. Average wind estimation RMSE for different values of $W_{20,\text{true}}$, averaged over the six flight scenarios and 27 wind conditions. Boxes show the interquartile range (Q1–Q3) with the median line; whiskers extend to the most extreme point within 1.5×IQR. Outlier points beyond the whiskers are omitted. 59
5.6. Average wind estimation RMSE by scenario for Unscented KF and Extended KF. Boxes show the interquartile range (Q1–Q3) with the median line; whiskers extend to the most extreme point within 1.5×IQR. Outlier points beyond the whiskers are omitted. 60
5.7. Minimum (top) and maximum (bottom) singular values of $\boldsymbol{F}_{v_W}$ over time for Hover, Forward fast, and Rotation, with constant wind component $\boldsymbol{v}_{W,\text{const}} = [-2\ 0\ -0.5]\text{m/s}$ and reference wind speed $W_{20,\text{true}} = 5\,\text{m/s}$. 61
5.8. Actual and estimated wind for hover flight scenario, constant wind component $\boldsymbol{v}_{W,\text{const}} = [2.5\ 0\ -0.5]\ \text{m/s}$ and $W_{20,\text{true}} = 0\,\text{m/s}$. Process noise is disabled for visualization purposes to highlight the convergence behavior of the wind estimate. 62

5.9. Comparison of EKF wind estimation with and without the 1D airspeed measurement for the fast forward flight scenario with $\boldsymbol{v}_{W,\text{const}} = [-2\ 1\ -0.5]\text{m/s}$ and $W_{20,\text{true}} = 5\,\text{m/s}$. . 64
5.10. Comparison of the true constant wind component with estimates obtained by applying low-pass filtering and a moving average to the wind estimate from the Kalman filter, over a $500\,\text{s}$ flight. 66
5.11. Comparison of MLP and LSTM wind estimation for the rotation flight scenario with $\boldsymbol{v}_{W,\text{const}} = [-2.5\ 0\ 0]\text{m/s}$ and $W_{20,\text{true}} = 10\,\text{m/s}$. 67
5.12. Comparison of the RMSE of the estimated wind for all seven experiments. The x-axis indicates the scenario on which the networks are tested. All denotes Experiment A, in which all scenarios are used for training. Each of the remaining labels denotes one run of Experiment B, in which the named scenario is excluded from the training data and used for testing. 68
5.13. Comparison of wind estimation RMSE for MLP, LSTM and EKF for different noise levels. 70
5.14. Comparison of the trajectories for the PID baseline and the DFF and NDI compensation approaches with Robust and Aggressive gain settings for a representative Curve scenario test run. 74
5.15. Comparison of the Scenario Performance Metric across selected controller/gain combinations for each flight scenario, sorted from highest to lowest. The orange bar highlights the NDI controller with Robust gains. 75
5.16. Grouped errors (position, velocity, attitude, and angular rate) for the NDI controller with Robust gains across all evaluated flight scenarios. 76

List of Tables

4.1. Number of simulation runs in each data split. In Experiment B, the held-out flight scenario forms the test set. 48
4.2. Layer structure and number of trainable parameters of the two networks. 51
4.3. Training settings of the two networks. 52

5.1. Mean computational runtime per simulation step for the EKF and UKF. The results are obtained from 100 simulations with a simulated duration of $1000\,\mathrm{s}$ and a simulation step size of $0.01\,\mathrm{s}$. The normalized runtime is given relative to the EKF. 63
5.2. Comparison of mean wind-estimation RMSE, averaged over all scenarios and wind conditions, with and without the 1D airspeed measurement, considering only converged test cases from the complete EKF and UKF test suite. 65
5.3. Mean computational runtime per step of the EKF, UKF, MLP, and LSTM. The results are obtained from 100 simulation runs with a simulated duration of $1000\,\mathrm{s}$. One step is performed every $0.01\,\mathrm{s}$. The normalized runtime is given relative to the EKF. 71
5.4. Overall Performance Metric for each controller architecture and gain setting. Values below one indicate better-than-average, and values above one worse-than-average, tracking performance. 73

6.1. Logged signals of the test flights. 77

A.1. Modeling Coefficients and constants 81
A.2. Measurement noise standard deviations used for the Kalman filter. 81

B.1. Mean RMSE of wind estimation error signals per $(W_{20,\mathrm{true}},\ W_{20,\mathrm{ass}})$ pair. For each $W_{20,\mathrm{true}}$ the matching case ($W_{20,\mathrm{ass}} = W_{20,\mathrm{true}}$) and the case yielding the lowest overall RMSE are shown. 83
B.2. Overall wind-estimation RMSE across all test runs, for the full flight trajectory and for the converged time window ($t = 20$–$100\,\mathrm{s}$). 83
B.3. Wind-estimation RMSE by turbulence intensity W_{20}, computed on the converged window ($t = 20$–$100\,\mathrm{s}$). 84
B.4. Wind-estimation RMSE by flight scenario, computed on the converged window ($t = 20$–$100\,\mathrm{s}$). 84

Nomenclature

Abbreviations

ANN	Artificial Neural Network
COG	Center of Gravity
COV	Center of Volume
CS	Coordinate System
DFF	Disturbance Feedforward
EKF	Extended Kalman Filter
KF	Kalman Filter
LSTM	Long Short-Term Memory
LTI	linear time-invariant
MLP	Multi-Layer Perceptron
MMSE	Minimum Mean Square Error
NDI	Nonlinear Dynamic Inversion
NED	North-East-Down convention
NED	North-East-Down
RNN	Recurrent Neural Network
UKF	Unscented Kalman Filter

Latin Letters

$\boldsymbol{I}$	Inertia Matrix with respect to the COG

Greek Letters

ω	Angular velocity ($\mathrm{rad\,s^{-1}}$)

Indices

$(\cdot)_B$	Quantity describing the body
${}_B(\cdot)$	Quantity in body-fixed coordinate system
${}_E(\cdot)$	Quantity in Earth coordinate system

1. Introduction

In the last decade, unmanned aerial vehicles (UAVs) have become increasingly common in civil and commercial applications. Improvements in hardware and software have made it possible to build lightweight, energy-efficient and cost-effective vehicles with active stabilization. UAVs are particularly suited for operations beyond human reach or where human deployment poses a serious risk. Besides popular applications in media and aerial photography, tasks such as infrastructure maintenance and surveying in agriculture, wildlife monitoring and security are becoming more and more common [**MazurEtAl16**, **dro18**]. Among the different UAV types, fixed-wing and multirotor vehicles are by far the most widespread and well researched. Quadcopters in particular have low entry barriers owing to their simple, transferable and inexpensive design.

Lighter-than-air (LTA) vehicles such as airships offer a distinct advantage over these platforms. Because buoyancy carries the weight of the vehicle, an airship does not have to spend energy to stay in the air. This allows considerably longer flight times than multirotors, which must produce thrust continuously, and makes airships well suited for long-duration tasks such as inspection, monitoring and surveying. Also its low speed and soft structure reduce the risk of damage to people and the environment during operation close to infrastructure. These advantages come at a price. Above all, their large volume in relation to their mass makes them highly susceptible to wind, so that controllers capable of handling strong disturbances require substantially more development effort.

Atmospheric wind is one of the dominant sources of disturbance for lighter-than-air vehicles. Airships are characterized by their large volume, low flight speed and high aerodynamic drag, which makes them considerably more susceptible to atmospheric disturbances than conventional fixed-wing aircraft or multirotors. Even moderate wind speeds can be comparable to the airspeed the vehicle is able to achieve, so that the wind may push the airship off its desired trajectory or exceed the available control authority. This sensitivity is one of the main challenges for autonomous airship operation, in particular during low-speed maneuvers such as hovering, station keeping, inspection tasks and precision landing, where the demands on tracking accuracy are highest.

Knowing the current wind is therefore valuable, but it is not directly available. A dedicated wind sensor, such as a pitot tube or an ultrasonic anemometer, can in principle measure the relative airflow, but it is often not a practical solution, especially for small autonomous airships. Additional sensors increase weight, cost and system complexity, and payload is scarce on a lighter-than-air vehicle. Pitot-based sensors also lose accuracy at the low airspeeds at which airships typically operate. Consequently, it is desirable to estimate the wind onboard from the measurements that are available anyway, such as velocity, attitude and angular rates. Such an estimator provides the wind information without additional hardware and can also be used to complement or validate a sensor if one is installed.

A reliable wind estimate enables more than observing the surrounding atmosphere. If it is fed into the flight controller, the aerodynamic disturbance can be compensated actively instead of being left for the feedback loop to reject after it has already caused a tracking error. A purely feedback-based controller reacts only to deviations that have already occurred, and for a slowly responding vehicle like an airship this delay is significant. Wind compensation has the potential to improve trajectory tracking, reduce the control effort and increase the robustness of autonomous flight under varying environmental conditions.

Achieving accurate wind estimation and effective compensation is, however, challenging. The airship dynamics are strongly nonlinear and are governed by the interaction of gravitational, buoyancy, propulsion and aerodynamic forces, while the aerodynamic effects depend on the unknown wind itself. Both the estimator and the controller must therefore take the nonlinear dynamics and the coupling between vehicle motion and surrounding airflow into account. Model-based estimators address this by exploiting a physical description of the vehicle, but their accuracy depends on the quality of the model. Data-driven methods, in contrast, learn the relation between the measurements and the wind directly from data and do not require an explicit wind model, although they depend on the available training data and on how well they generalize to conditions not seen during training.

This motivates the objectives of this work, which are outlined in the following section.

1.1. Aim of this work

- Objectives - Scope - Evaluation scenarios

The aim of this work is to develop and implement model-based and data-driven algorithms for wind estimation and wind compensation for an unmanned airship. The proposed approach shall reliably estimate the ambient wind using only the available onboard measurements and the nonlinear dynamic model of the vehicle, without requiring dedicated wind sensors.

The wind estimation algorithm is designed to operate under conditions representative of realistic flight scenarios. These include dynamic wind conditions with gusts of up to 30 km/h, as well as different flight maneuvers such as forward and sideward flight, rotation about the vertical axis, and coordinated turning during forward flight. The estimated wind shall remain accurate and robust despite the nonlinear vehicle dynamics and changing operating conditions.

Based on the estimated wind, a model-based compensation strategy is implemented to directly counteract the aerodynamic disturbances acting on the airship. By incorporating the estimated wind into the control system, the influence of wind on the vehicle motion is reduced as directly as possible, thereby improving trajectory tracking performance and the overall robustness of the flight controller.

To evaluate the proposed approach, the complete estimation and compensation framework is implemented in a MATLAB/Simulink simulation environment using a comprehensive nonlinear airship model. The performance of the estimator and the effectiveness of the compensation strategy are assessed under various wind conditions and flight maneuvers representative of realistic operating scenarios.

1.2. State of the art / Related work

- Airship dynamics - Wind estimation - Wind compensation / disturbance observers - Positioning this thesis

1.3. Airship squid

This work is carried out on the airship Squid, an airship developed and built by roboloon in Stuttgart. Squid features a novel propulsion system that enables omnidirectional maneuvering and is therefore well suited for the investigation of wind estimation and compensation. An overview of the airship and its main components is shown in Figure 1.1.

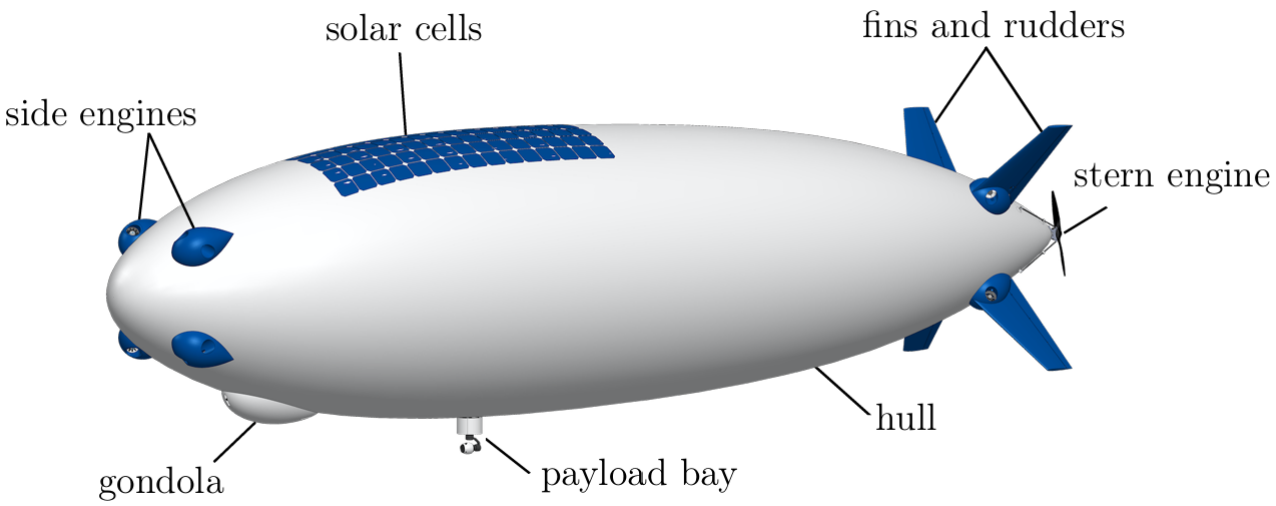


Figure 1.1.: CAD model of the Airship Squid and its components.

1.3.1. Airship Configuration

The airship consists of an approximately $8\,\mathrm{m}$ long elastic hull with a volume of $16.3\,\mathrm{m}^3$, which is filled with helium to provide the required buoyant lift. The hull is equipped with an integrated safety valve for controlled release of helium when required. The center of gravity (COG) is positioned below the center of volume (COV), while the mass is distributed approximately evenly along the longitudinal and lateral axes. This configuration provides a naturally level orientation of the airship during hovering without requiring additional control effort.

The propulsion system of the airship consists of a stern propeller, four rudders, and eight impellers with four impellers at the front and four in the back. They arranged such that their combined actuation can generate motion in all translational and rotational directions. By simultaneously actuating selected engines, the individual thrust forces can be combined to produce a desired resultant force and moment acting on the airship. This propulsion concept results in an overactuated system, as multiple combinations of actuator commands can be used to generate the same desired generalized force.

The airship is equipped with several onboard measurement systems that provide the information required for state estimation and control. The central flight-control hardware is a Cube Orange (Pixhawk 2.1), which integrates two processing units, three accelerometers, three gyroscopes, two barometers, and a compass [**Cube**]. In addition, an external GPS unit is connected to the flight controller and provides position and velocity measurements. The combination of the inertial, barometric, magnetic, and GPS measurements provides information on the airship's position, translational velocity, attitude, and angular rates, which form the basis for the state estimation used in the control system.

For the wind estimation investigated in this work, the airspeed measurement provides useful information on the relative wind velocity. The airship is therefore equipped with a one-dimensional Pitot tube that measures the airspeed along the body-fixed x-axis. The measurement is only available for airspeeds above approximately $2\,\mathrm{m/s}$. Below this threshold, no valid airspeed measurement is provided. This limitation is particularly relevant for low-speed flight and hovering conditions and is therefore taken into account in the design and evaluation of the wind estimation method.

The flight-control hardware is housed together with the battery and the remaining electrical components in the gondola located below the hull. The gondola additionally provides space for a camera and further payload. Additional sensors can be installed in the payload bay for specific applications.

1.3.2. Control Architecture

The airship is controlled using a PID-based control structure. The controller receives a desired position and generates the corresponding desired forces and moments acting on the airship. The PID controller is tuned to provide robust behavior across the considered flight envelope and therefore serves as the baseline controller for this work.

The control allocator converts the desired forces and moments into individual actuator commands. Due to the overactuated propulsion system, multiple combinations of actuator inputs can produce the same desired generalized forces. The control allocator therefore determines a suitable actuator configuration while accounting for the available actuator constraints. The design and implementation of the PID controller and control allocator, including their mathematical formulation and derivation, are described in detail in [**Ahmed2022Controller**].

Before being implemented on the physical airship, all algorithms developed in this work are first evaluated in simulation. The simulation environment contains the airship model derived in Chapter 2.2, together with the implemented PID controller and control allocator. In addition to the nominal airship dynamics, uncertainties in the force and moment model are considered to account for discrepancies between the model and the physical system.

After successful validation in simulation, the algorithms are evaluated in real-world flight tests on the Airship Squid. This two-stage procedure allows the developed estimation and compensation methods to be investigated under controlled and reproducible simulation conditions before their performance is assessed on the physical airship.

2. Fundamentals

This chapter introduces the theoretical foundations and key concepts required for the remainder of this thesis. It presents the coordinate systems used throughout this work, the mathematical models of the airship and the wind, the fundamentals of Kalman filtering, and the control concepts underlying the proposed wind compensation approach. Finally, the data-driven methods considered in this work are introduced.

2.1. Coordinate Systems

To describe the motion and dynamics of the airship, as well as the wind velocity acting on the vehicle, two coordinate systems (CS) are considered throughout this work: the body-fixed coordinate system and the earth-fixed coordinate system. Their definitions and the associated coordinate transformations follow [**Fichter2009Flugmechanik**] and are briefly reviewed in the following.

2.1.1. Body-Fixed Coordinate System

The body-fixed coordinate system B is attached to the airship and therefore translates and rotates together with the vehicle. Its origin is located at the Center of Gravity (COG) of the airship, and the coordinate axes are defined according to the standard body-axis convention, with the x_B-axis points towards the nose of the airship, the y_B-axis points towards the right side, and the z_B-axis points downwards.

The body-fixed coordinate system is used to describe quantities directly related to the vehicle dynamics, such as translational and rotational velocities, aerodynamic forces, propulsion forces, and moments. Since the equations of motion are formulated in the body-fixed frame, this coordinate system represents the primary reference frame for the dynamic model. A quantity expressed in the body-fixed coordinate system is indicated by a preceding frame identifier, denoted as ${}_B(\cdot)$.

2.1.2. Earth-Fixed Coordinate System

The earth-fixed coordinate system E is introduced as an inertial reference frame for describing the position of the airship and the wind velocity, and serves as the reference frame relative to which the airship's orientation is defined. For simplicity, a flat-earth assumption is made, which neglects the curvature and rotation of the Earth. This assumption is justified by the relatively short flight distances considered in this work.

The origin of the earth-fixed coordinate system is chosen as an arbitrary point on the Earth's surface. The axes are aligned according to the North-East-Down (NED) convention, where the north and east axes span the horizontal plane and the down axis points vertically downward. Under the flat-Earth assumption, the earth-fixed coordinate system is considered an inertial reference frame, neglecting translational acceleration and rotation. A quantity expressed in the earth-fixed coordinate system is indicated by a preceding frame identifier, denoted as ${}_E(\cdot)$.

Figure 2.1 illustrates the body-fixed and earth-fixed coordinate systems and their orientations relative to the airship.

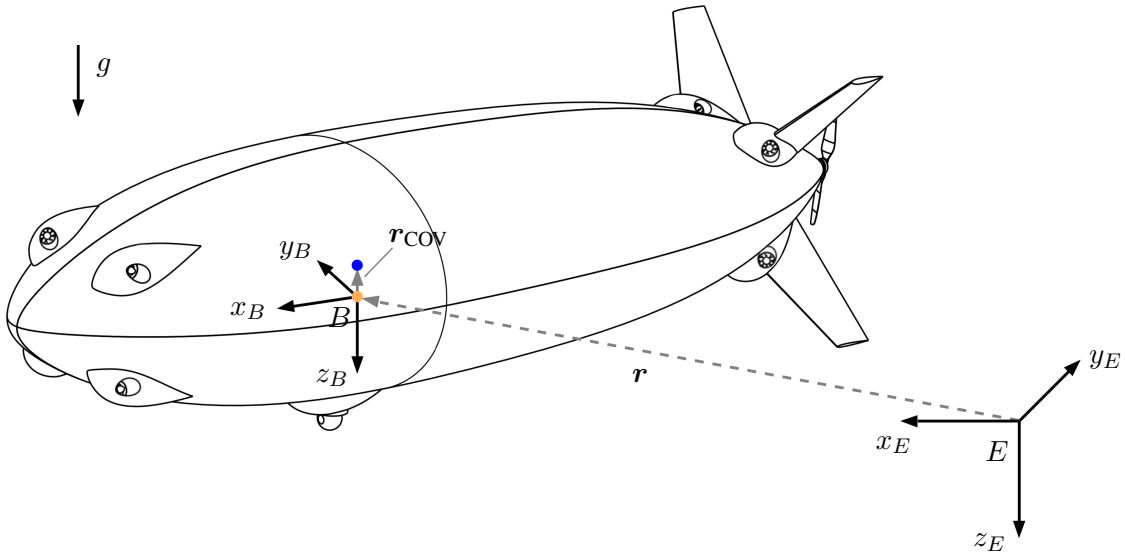


Figure 2.1.: Airship with body-fixed coordinate system B and earth-fixed coordinate system E. The orange dot marks the Center of Gravity and the blue dot marks the Center of Volume (COV), connected by the position vector $\boldsymbol{r}_{COV}$.

The transformation from the earth-fixed coordinate system to the body-fixed coordinate system is described by the rotation matrix [**Fossen2021Handbook**]

$$\boldsymbol{R}_{BE} = \begin{bmatrix} c_\psi c_\theta & s_\psi c_\theta & -s_\theta \\ c_\psi s_\theta s_\phi - s_\psi c_\phi & s_\psi s_\theta s_\phi + c_\psi c_\phi & c_\theta s_\phi \\ c_\psi s_\theta c_\phi + s_\psi s_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi & c_\theta c_\phi \end{bmatrix}, \tag{2.1}$$

which maps vectors expressed in the earth-fixed coordinate system into the body-fixed coordinate system. Here ϕ, θ, and ψ denote the roll, pitch, and yaw angles, describing the orientation of B relative to E via the ZYX Euler angle sequence and $s_{(\cdot)}$ and $c_{(\cdot)}$ denote $\sin(\cdot)$ and $\cos(\cdot)$, respectively.

The inverse transformation is given by the transpose of the rotation matrix

$$\boldsymbol{R}_{EB} = (\boldsymbol{R}_{BE})^T, \tag{2.2}$$

and describes the transformation from body-fixed CS to earth-fixed CS.

2.2. Airship Model

The airship is modeled using a six degree-of-freedom (6DOF) Newton Euler approach [**Fossen2021Handbook**]. The airship is assumed to be a rigid body with constant mass, and the translational and rotational dynamics are modeled accordingly. The equations of motion are expressed in body-fixed coordinates, allowing for a constant inertia matrix. Unless stated otherwise, all quantities are expressed in the body-fixed coordinate

frame, and all forces and moments are assumed to act at the center of gravity (COG). The corresponding frame identifier ${}_B(\cdot)$ and point-of-application identifier $(\cdot)^{\text{COG}}$ are omitted throughout the remainder of this section for improved readability.

The three-dimensional **translational dynamics** in body-coordinates are given by the linear momentum theorem as

$$m\boldsymbol{I}_{3\times3}\,\boldsymbol{a}_B = \boldsymbol{F}_{\text{ext}}, \tag{2.3}$$

where m denotes the mass of the airship, $\boldsymbol{I}_{3\times3} \in \mathbb{R}^{3\times3}$ denotes the identity matrix, $\boldsymbol{a}_B \in \mathbb{R}^3$ describes the acceleration of the body (airship) in body-fixed coordinates and $\boldsymbol{F}_{\text{ext}} \in \mathbb{R}^3$ represents the resultant external force acting at the COG, expressed in body-fixed coordinates.

To relate the body acceleration $\boldsymbol{a}_B$ to the body velocity $\boldsymbol{v}_B \in \mathbb{R}^3$, the rotation of the body-fixed coordinate system relative to the earth-fixed coordinate system must be taken into account. Applying the transport theorem yields [**Greenwood2003Dynamics**]:

$$\boldsymbol{a}_B = \dot{\boldsymbol{v}}_B + (\boldsymbol{\omega}_{EB} \times \boldsymbol{v}_B) = \dot{\boldsymbol{v}}_B + \boldsymbol{a}_{\text{cor}}, \tag{2.4}$$

where $\boldsymbol{a}_{\text{cor}} \in \mathbb{R}^3$ denotes the Coriolis acceleration as a result of the rotating body-fixed CS and $\boldsymbol{\omega}_{EB} \in \mathbb{R}^3$ describes the rotational velocity of the body-fixed CS against the earth-fixed CS. As the earth-fixed CS is inertial (non-rotating) this is equal to the angular velocity of the airship along its three axes, $\boldsymbol{\omega}_B \in \mathbb{R}^3$.

Inserting Eq. (2.4) in Eq. (2.3) and defining the Coriolis force as

$$\boldsymbol{F}_{\text{cor}} = m\boldsymbol{I}_{3\times3}\,\boldsymbol{a}_{\text{cor}}, \tag{2.5}$$

yields the dynamics for the translational motion of the airship in body-fixed coordinates:

$$m\boldsymbol{I}_{3\times3}\,\dot{\boldsymbol{v}}_B = \boldsymbol{F}_{\text{ext}} - \boldsymbol{F}_{\text{cor}}. \tag{2.6}$$

The three-dimensional **rotational dynamics** in body-fixed coordinates are given by the angular momentum theorem as

$$\frac{d}{dt}\left(\boldsymbol{I}_B\,\boldsymbol{\omega}_B\right) = \boldsymbol{M}_{\text{ext}}, \tag{2.7}$$

where $\boldsymbol{I}_B \in \mathbb{R}^{3\times3}$ denotes the inertia matrix of the airship expressed in body-fixed coordinates and $\boldsymbol{M}_{\text{ext}} \in \mathbb{R}^3$ represents the resultant external moment acting about the COG, expressed in body-fixed coordinates.

To relate the change of angular momentum to the angular acceleration of the airship, the rotation of the body-fixed coordinate system relative to the earth-fixed coordinate system must be considered. Applying the transport theorem to the angular momentum vector yields:

$$\frac{d}{dt}\left(\boldsymbol{I}_B\,\boldsymbol{\omega}_B\right) = \boldsymbol{I}_B\,\dot{\boldsymbol{\omega}}_B + \left(\boldsymbol{\omega}_B \times \boldsymbol{I}_B\,\boldsymbol{\omega}_B\right), \tag{2.8}$$

where the second term represents the gyroscopic moment caused by the rotation of the body-fixed coordinate system. Since the inertia matrix is constant when expressed in body-fixed coordinates, the time derivative only applies to the angular velocity.

Defining the gyroscopic moment as

$$\boldsymbol{M}_{\text{gyro}} = \left(\boldsymbol{\omega}_B \times \boldsymbol{I}_B\, \boldsymbol{\omega}_B\right), \tag{2.9}$$

and rearranging Eq. (2.8) yields the rotational dynamics of the airship in body-fixed coordinates:

$$\boldsymbol{I}_B\, \dot{\boldsymbol{\omega}}_B = \boldsymbol{M}_{\text{ext}} - \boldsymbol{M}_{\text{gyro}}. \tag{2.10}$$

The translational dynamics (2.6) and the rotational dynamics (2.10) can be combined into one equation obtaining the Newton Euler Equations for the airship:

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} = \boldsymbol{\mathcal{F}}_{\text{ext}} - \boldsymbol{\mathcal{F}}_{\text{cor}}, \tag{2.11}$$

with the generalized velocity vector $\boldsymbol{\nu}$ and mass matrix $\boldsymbol{M}$ defined as

$$\boldsymbol{\nu} = \begin{bmatrix} \boldsymbol{v}_B \\ \boldsymbol{\omega}_B \end{bmatrix} \in \mathbb{R}^6, \qquad \boldsymbol{M} = \begin{bmatrix} m\boldsymbol{I}_{3\times3} & \boldsymbol{0}_{3\times3} \\ \boldsymbol{0}_{3\times3} & \boldsymbol{I}_B \end{bmatrix} \in \mathbb{R}^{6\times6}. \tag{2.12}$$

The generalized external and Coriolis force vectors are defined as

$$\boldsymbol{\mathcal{F}}_{\text{ext}} = \begin{bmatrix} \boldsymbol{F}_{\text{ext}} \\ \boldsymbol{M}_{\text{ext}} \end{bmatrix} \in \mathbb{R}^6, \qquad \boldsymbol{\mathcal{F}}_{\text{cor}} = \begin{bmatrix} \boldsymbol{F}_{\text{cor}} \\ \boldsymbol{M}_{\text{cor}} \end{bmatrix} \in \mathbb{R}^6. \tag{2.13}$$

The generalized external forces acting on the airship can be further classified based on the available knowledge of the airship system. Since the airship is modeled as a rigid body, the internal deformations and distributed effects of the structure are neglected, which represents a significant simplification of the real system. Consequently, the accuracy of the airship model depends strongly on the precise representation of the external forces and moments acting on the vehicle. To capture the relevant physical behavior of the airship, these contributions must therefore be modeled as accurately as possible.

However, due to the complexity of the real environment and the limitations of the available model knowledge, not all forces and moments acting on the airship can be fully accounted for. The resulting discrepancies between modeled and actual system behavior are considered model uncertainties. These uncertainties are not explicitly modeled within the presented equations of motion but are addressed in the subsequent estimation and control design.

The external forces and moments acting on the airship are divided into four main categories: gravitational, buoyancy, aerodynamic, and propulsion,

$$\boldsymbol{\mathcal{F}}_{\text{ext}} = \boldsymbol{\mathcal{F}}_{\text{grav}} + \boldsymbol{\mathcal{F}}_{\text{buoy}} + \boldsymbol{\mathcal{F}}_{\text{aero}} + \boldsymbol{\mathcal{F}}_{\text{prop}}, \tag{2.14}$$

resulting in the final EOM:

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} = \boldsymbol{\mathcal{F}}_{\text{grav}} + \boldsymbol{\mathcal{F}}_{\text{buoy}} + \boldsymbol{\mathcal{F}}_{\text{aero}} + \boldsymbol{\mathcal{F}}_{\text{prop}} - \boldsymbol{\mathcal{F}}_{\text{cor}}. \tag{2.15}$$

The generalized external forces are derived in the following.

2.2.1. Gravitation and Buoyancy

The gravitational force computes to

$$\boldsymbol{F}_{\text{grav}} = m\boldsymbol{g} = m\boldsymbol{R}_{BE}\,{}_{E}\boldsymbol{g} \qquad \text{with} \qquad {}_{E}\boldsymbol{g} = \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix}, \tag{2.16}$$

where ${}_{E}\boldsymbol{g}$ represents the constant gravity vector in the inertial earth reference frame following the NED convention and $\boldsymbol{R}_{BE}$ denotes the transformation matrix from earth-fixed CS to body-fixed CS.

As the gravitational force acts at the center of gravity, no gravitational moment about the COG is induced, thus

$$\boldsymbol{\mathcal{F}}_{\text{grav}} = \begin{bmatrix} \boldsymbol{F}_{\text{grav}} \\ \boldsymbol{0}_{3\times 1} \end{bmatrix}. \tag{2.17}$$

The buoyancy depends on the density of the surrounding air ρ_{air} and the lifting gas contained within the hull ρ_{gas} as well as the volume of the airship V, which is assumed constant for simplicity. Assuming a homogeneous distribution, the resulting buoyancy force acts at the center of volume (COV) and is given by

$$\boldsymbol{F}_{\text{buoy}}^{\text{COV}} = -V\boldsymbol{g}(\rho_{\text{air}} - \rho_{\text{gas}}). \tag{2.18}$$

In case of the airship Squid, the volume is chosen such that $m > V(\rho_{\text{air}} - \rho_{\text{gas}})$, resulting in a downward-directed net force that prevents the airship from ascending without external input. To express the buoyancy contribution with respect to the center of gravity, the moment induced by the offset between the COV and COG is considered:

$$\boldsymbol{M}_{\text{buoy}} = \boldsymbol{r}_{\text{COV}} \times \boldsymbol{F}_{\text{buoy}}^{\text{COV}}, \tag{2.19}$$

where $\boldsymbol{r}_{\text{COV}} \in \mathbb{R}^3$ denotes the vector from the COG to COV. Due to the design of the airship *Squid*, $\boldsymbol{r}_{\text{COV}}$ only contains a component along the z_B-direction. Consequently, the generalized buoyant force expressed at the COG is given by

$$\boldsymbol{\mathcal{F}}_{\text{buoy}} = \begin{bmatrix} \boldsymbol{F}_{\text{buoy}} \\ \boldsymbol{M}_{\text{buoy}} \end{bmatrix} \qquad \text{with} \qquad \boldsymbol{F}_{\text{buoy}} = \boldsymbol{F}_{\text{buoy}}^{\text{COV}}. \tag{2.20}$$

2.2.2. Aerodynamics

The aerodynamic forces and moments play a central role in the airship dynamics, as they directly depend on the aerodynamic velocity. The aerodynamic velocity is defined as

$$\boldsymbol{v}_A = \boldsymbol{v}_B - \boldsymbol{v}_W, \tag{2.21}$$

and denotes the relative velocity between the airship and the surrounding air (commonly referred to as airspeed), where $\boldsymbol{v}_W \in \mathbb{R}^3$ corresponds to the wind velocity expressed in body-fixed coordinates. As a result, the aerodynamical forces and moments contain information about the current wind conditions. An accurate aerodynamic model is therefore essential and forms the basis for the wind estimation approach presented in this work.

As described in [**Li2007AirshipDynamics**], the aerodynamic forces and moments can be classified into several contributions, of which drag, added mass, viscous effects, and fin forces are the dominant ones. These are therefore considered in the present model:

$$\boldsymbol{\mathcal{F}}_{\text{aero}} = \boldsymbol{\mathcal{F}}_{\text{drag}} + \boldsymbol{\mathcal{F}}_{\text{am}} + \boldsymbol{\mathcal{F}}_{\text{visc}} + \boldsymbol{\mathcal{F}}_{\text{fins}}. \tag{2.22}$$

Rather than reproducing the full derivation, this section presents the mathematical expressions for the individual aerodynamic force and moment contributions in detail. The underlying derivations are provided in [**Li2007AirshipDynamics**]. The aerodynamic forces act at the center of volume. Therefore, the offset between the COV and the COG results in an induced moment, which is considered in the aerodynamic moment contribution.

Aerodynamic Drag

The aerodynamic drag force opposes the relative motion between the airship and the surrounding air and therefore acts in the opposite direction of the aerodynamic velocity. In this work, the drag force is modeled independently along each body-fixed axis using a quadratic drag formulation based on the conventional drag equation [**Anderson2017Fundamentals**]. The resulting aerodynamic drag force is given by

$$\boldsymbol{F}_{\text{drag}}^{\text{COV}} = -\frac{1}{2}\rho_{\text{air}}\,\text{sgn}(\boldsymbol{v}_A) \odot \boldsymbol{v}_A^2 \odot \begin{bmatrix} c_{\text{D,ax}} \\ c_{\text{D,lat}} \\ c_{\text{D,lat}} \end{bmatrix}, \tag{2.23}$$

where $\odot$ represents the element-wise (Hadamard) product, and $\text{sgn}(\cdot)$ is applied element-wise. The parameters $c_{\text{D,ax}}$ and $c_{\text{D,lat}}$ are the axial and lateral drag coefficients, respectively. Since the airship is assumed to be rotationally symmetric about its longitudinal axis, the same drag coefficient is used for the lateral and vertical directions.

Since the aerodynamic drag force acts at the COV, it must be transformed to the COG to obtain the corresponding generalized aerodynamic force as

$$\boldsymbol{\mathcal{F}}_{\text{drag}} = \begin{bmatrix} \boldsymbol{F}_{\text{drag}} \\ \boldsymbol{M}_{\text{drag}} \end{bmatrix} \qquad \text{with} \qquad \boldsymbol{M}_{\text{drag}} = \boldsymbol{r}_{\text{COV}} \times \boldsymbol{F}_{\text{drag}}^{\text{COV}}, \qquad \boldsymbol{F}_{\text{drag}} = \boldsymbol{F}_{\text{drag}}^{\text{COV}}. \tag{2.24}$$

Added Mass

For an airship, the added mass can be interpreted as the effective mass of the surrounding air that is accelerated together with the vehicle [**Newman1977Hydrodynamics**]. The associated added-mass forces and moments arise from the inertia of this accelerated air and contribute to the equations of motion in two ways: through a constant added-mass matrix and through a velocity-dependent coupling term:

$$\boldsymbol{\mathcal{F}}_{\text{am}}^{\text{COV}} = -\begin{bmatrix} \boldsymbol{M}_{11} & \boldsymbol{M}_{12} \\ \boldsymbol{M}_{21} & \boldsymbol{M}_{22} \end{bmatrix} \begin{bmatrix} \dot{\boldsymbol{v}}_B \\ \dot{\boldsymbol{\omega}}_B \end{bmatrix} - \begin{bmatrix} \boldsymbol{\omega}_B \times (\boldsymbol{M}_{11}\boldsymbol{v}_A + \boldsymbol{M}_{12}\boldsymbol{\omega}_B) \\ \boldsymbol{v}_A \times (\boldsymbol{M}_{11}\boldsymbol{v}_A + \boldsymbol{M}_{12}\boldsymbol{\omega}_B) + \boldsymbol{\omega}_B \times (\boldsymbol{M}_{21}\boldsymbol{v}_A + \boldsymbol{M}_{22}\boldsymbol{\omega}_B) \end{bmatrix}. \tag{2.25}$$

For steady translation ($\boldsymbol{\omega}_B = \boldsymbol{0}$), the coupling term reduces to the classical *Munk moment*, $-\boldsymbol{v}_A \times (\boldsymbol{M}_{11}\boldsymbol{v}_A)$, which is a destabilizing moment characteristic of elongated bodies first described in [**Munk1924Aerodynamic**].

The entries of the added-mass matrix are computed using the analytical expressions given by [**Li2007AirshipDynamics**].

Since the generalized added-mass force acts at the COV, it is transformed to the COG as

$$\boldsymbol{\mathcal{F}}_{\text{am}} = \begin{bmatrix} \boldsymbol{F}_{\text{am}} \\ \boldsymbol{M}_{\text{am}}^{\text{COV}} + \boldsymbol{r}_{\text{COV}} \times \boldsymbol{F}_{\text{am}} \end{bmatrix} \qquad \text{with} \qquad \boldsymbol{F}_{\text{am}} = \boldsymbol{F}_{\text{am}}^{\text{COV}}. \tag{2.26}$$

Viscous Aerodynamic Effects

In contrast to the added-mass forces, which are associated with the acceleration of the surrounding air, the viscous forces and moments originate from the interaction between the airship surface and the surrounding airflow. These effects arise from viscous shear stresses and pressure differences and are described in detail by [**Anderson2017Fundamentals**].

The magnitude of the viscous force and moment acting normal to the hull centerline are given by

$$F_{\text{visc,N}}^{\text{COV}} = q_0 \left(-C_{\text{F,hif}} \sin 2\gamma + C_{\text{F,hvf}} \sin^2 \gamma \right), \tag{2.27}$$

$$M_{\text{visc,N}}^{\text{COV}} = q_0 \left(-C_{\text{M,hif}} \sin 2\gamma + C_{\text{M,hvf}} \sin^2 \gamma \right), \tag{2.28}$$

where q_0 describes the dynamic pressure associated by the local aerodynamic velocity $\boldsymbol{v}_{A,0}$ and γ denotes the angle between the hull centerline and $\boldsymbol{v}_{A,0}$:

$$q_0 = \frac{1}{2}\rho_{\text{air}} \left\| \boldsymbol{v}_{A,0} \right\|_2^2 \qquad \text{and} \qquad \gamma = \text{atan2}\Big(\sqrt{v_{A,0}^2 + w_{A,0}^2} / u_{A,0} \Big). \tag{2.29}$$

Here $\boldsymbol{v}_{A,0} = [u_{A,0},\, v_{A,0},\, w_{A,0}]$ denotes the aerodynamic velocity at ε_0, which marks the point along the hull, measured from the nose, beyond which correction for real flow behavior is necessary. The coefficient $C_{\cdot,\text{hif}}$ accounts for the additional drag caused by the pressure difference between the front and rear of the hull after flow separation, whereas $C_{\cdot,\text{hvf}}$ represents the crossflow drag caused by the separated flow, modeled using a cylinder-drag analogy.

The viscous force and moment are then decomposed into their body-axis components. Since both act normal to the hull centerline $F_{\text{visc},x}^{\text{COV}} = M_{\text{visc},x}^{\text{COV}} = 0$ and

$$F_{\text{visc},y}^{\text{COV}} = -F_{\text{visc,N}}^{\text{COV}} \frac{v_{A,0}}{\sqrt{v_{A,0}^2 + w_{A,0}^2}}, \qquad F_{\text{visc},z}^{\text{COV}} = -F_{\text{visc,N}}^{\text{COV}} \frac{w_{A,0}}{\sqrt{v_{A,0}^2 + w_{A,0}^2}}, \tag{2.30}$$

and the corresponding body-axis moment components are

$$M_{\text{visc},y}^{\text{COV}} = M_{\text{visc,N}}^{\text{COV}} \frac{v_{A,0}}{\sqrt{v_{A,0}^2 + w_{A,0}^2}}, \qquad M_{\text{visc},z}^{\text{COV}} = -M_{\text{visc,N}}^{\text{COV}} \frac{w_{A,0}}{\sqrt{v_{A,0}^2 + w_{A,0}^2}}. \tag{2.31}$$

Again, we have to transform the force and moment to the COG:

$$\boldsymbol{\mathcal{F}}_{\text{visc}} = \begin{bmatrix} \boldsymbol{F}_{\text{visc}} \\ \boldsymbol{M}_{\text{visc}}^{\text{COV}} + \boldsymbol{r}_{\text{COV}} \times \boldsymbol{F}_{\text{visc}} \end{bmatrix} \qquad \text{with} \qquad \boldsymbol{F}_{\text{visc}} = \boldsymbol{F}_{\text{visc}}^{\text{COV}}. \tag{2.32}$$

Fin Forces

The fin forces are computed using a lumped approach, in which each of the four fins is treated as a single lifting surface. The contribution of each fin depends on the local aerodynamic velocity and the corresponding angle of attack. The total fin force and moment are then obtained by summing the individual fin contributions.

For each fin $i = 1, \ldots, 4$, located at position $\boldsymbol{r}_{\text{fin},i}$ relative to the COV, the local aerodynamic velocity follows from the rigid-body velocity field:

$$\boldsymbol{v}_{A,\text{fin},i} = \boldsymbol{v}_A + \boldsymbol{\omega}_B \times \boldsymbol{r}_{\text{fin},i}, \tag{2.33}$$

from which the local dynamic pressure is obtained as

$$q_{0,\text{fin},i} = \frac{1}{2} \rho_{\text{air}} \left\| \boldsymbol{v}_{A,\text{fin},i} \right\|_2^2 . \tag{2.34}$$

The geometric angle of attack of each fin is computed from the local velocity component perpendicular to the fin surface $v_{\perp,\text{fin},i}$ and the local velocity component along the hull centerline $u_{\text{fin},i}$ as

$$\alpha_{\text{fin},i} = \text{atan2}(v_{\perp,\text{fin},i},\, u_{\text{fin},i}) \,, \tag{2.35}$$

and is saturated at the stall angle α_{stall}.

The resulting normal force on each fin computes to

$$F_{\text{fin},N,i} = -q_{0,\text{fin},i}\, C_{\text{F,fnf}}\, \alpha_{\text{fin},i}, \tag{2.36}$$

where $C_{\text{F,fnf}}$ is the empirical fin normal-force coefficient that is assumed to capture all aerodynamic effects of the fins, including geometric and interference effects, in a single fitted parameter.

The total fin force and moment about the COV are obtained by summing the individual fin contributions:

$$\boldsymbol{F}_{\text{fins}}^{\text{COV}} = \sum_{i=1}^{4} \boldsymbol{F}_{\text{fin},i}, \qquad \boldsymbol{M}_{\text{fins}}^{\text{COV}} = \sum_{i=1}^{4} \boldsymbol{r}_{\text{fin},i} \times \boldsymbol{F}_{\text{fin},i}, \tag{2.37}$$

where $\boldsymbol{F}_{\text{fin},i}$ denotes the fin force vector obtained by resolving the fin normal force $F_{\text{fin},N,i}$ into its body-fixed components according to the orientation of the respective fin.

As with the previous contributions, the resulting force and moment are transformed to the COG to obtain the generalized fin forces:

$$\boldsymbol{\mathcal{F}}_{\text{fins}} = \begin{bmatrix} \boldsymbol{F}_{\text{fins}}^{\text{COV}} \\ \boldsymbol{M}_{\text{fins}}^{\text{COV}} + \boldsymbol{r}_{\text{COV}} \times \boldsymbol{F}_{\text{fins}}^{\text{COV}} \end{bmatrix} \qquad \text{with} \qquad \boldsymbol{F}_{\text{fins}} = \boldsymbol{F}_{\text{fins}}^{\text{COV}}. \tag{2.38}$$

2.2.3. Propulsion

The propulsion forces and moments resulting from the command input $\boldsymbol{u} \in \mathbb{R}^9$ are computed in two steps. First, the thrust generated by each individual motor $F_{\text{motor},i}$ is determined from the corresponding motor command u_i using a nonlinear motor model. Due to the different characteristics of the impellers and the stern propeller, separate parameter sets are used for the two actuator types. The impeller commands u_i, $i = 1, \ldots, 8$, are normalized to $u_i \in [0, 1]$, reflecting their unidirectional operation, while the stern propeller command u_9 is normalized to $u_9 \in [-1, 1]$ to allow for both forward and reverse thrust. The resulting motor thrust is therefore given by

$$F_{\text{motor},i} = \begin{cases} \alpha_{\text{imp}} \left(k_{\text{imp}}\, u_i^2 + (1 - k_{\text{imp}}) u_i \right), & i = 1, \ldots, 8 \\ \alpha_{\text{stern}}\, \text{sgn}(u_i) \left(k_{\text{stern}}\, u_i^2 + (1 - k_{\text{stern}}) u_i \right), & i = 9. \end{cases} \tag{2.39}$$

The stern propeller exhibits different thrust characteristics in forward and reverse operation. To account for this asymmetry, separate parameter sets $(\alpha_{\text{stern}}, k_{\text{stern}})$ are used for positive and negative commands u_9. The individual motor thrusts are then combined into the resulting generalized force vector by considering the orientation and position of each motor. This transformation is described by the motor effectiveness matrix $\boldsymbol{B}_{\text{eff}}^{\text{COG}} \in \mathbb{R}^{6\times 9}$, which maps the individual motor thrusts to the generalized force vector acting at the COG. The matrix incorporates the orientation of each motor as well as the moment contributions resulting from their respective positions relative to the COG:

$$\boldsymbol{\mathcal{F}}_{\text{prop}} = \boldsymbol{B}_{\text{eff}}^{\text{COG}} \boldsymbol{F}_{\text{motor}}, \tag{2.40}$$

with

$$\boldsymbol{B}_{\text{eff}}^{\text{COG}} = \begin{bmatrix} \boldsymbol{d}_1 & \cdots & \boldsymbol{d}_9 \\ \boldsymbol{r}_1 \times \boldsymbol{d}_1 & \cdots & b\boldsymbol{r}_9 \times \boldsymbol{d}_9 \end{bmatrix}, \tag{2.41}$$

where $\boldsymbol{r}_i \in \mathbb{R}^3$ denotes the position vector of the i-th motor relative to the COG, and $\boldsymbol{d}_i \in \mathbb{R}^3$ denotes the corresponding fixed unit thrust direction vector.

2.3. Wind Model

To ensure that the simulation results are representative of real flight conditions, the wind model needs to accurately capture the dominant characteristics of atmospheric wind while remaining computationally efficient. In particular, a state-space realization is required so that the wind model can be incorporated into the airship dynamics, providing the necessary states for wind estimation within the Kalman filter framework. Since atmospheric wind generally consists of a slowly varying mean component combined with rapidly changing turbulence, the wind in this work is modeled as the sum of a constant mean wind and a stochastic turbulence component [**Etkin1995Dynamics**]:

$$\boldsymbol{v}_W = \boldsymbol{v}_{W,\text{const}} + \boldsymbol{v}_{W,\text{turb}}. \tag{2.42}$$

Stochastic turbulence models are widely used in aerospace applications because they reproduce the statistical properties of atmospheric turbulence without requiring an explicit description of the underlying turbulent wind field. A simple stochastic model is the random walk, in which white noise is integrated to

generate a randomly varying signal. However, to reproduce the characteristic properties of atmospheric turbulence, more sophisticated models are required. Two commonly used models in aerospace applications are the Dryden and von Kármán turbulence models. While the von Kármán model generally provides a more accurate representation of the atmospheric turbulence spectrum, it describes turbulence using non-rational functions and therefore cannot be represented exactly by a finite-dimensional state-space model. In contrast, the Dryden turbulence model approximates atmospheric turbulence by passing white noise through linear shaping filters, resulting in rational transfer functions that are realizable in state-space form [**Hoblit1988Gust**, **MILHDBK1797**].

Since an exact finite-dimensional state-space representation is required for integration into the Kalman filter framework, and the Dryden model still describes atmospheric turbulence sufficiently well for this purpose, it is adopted throughout this work. Figure 2.2 compares wind turbulences generated by a random walk model, the Dryden turbulence model and the von Kármán turbulence model. While the random walk exhibits an unrealistic long-term drift, the Dryden model produces bounded, correlated wind disturbances similar to the von Kármán model.

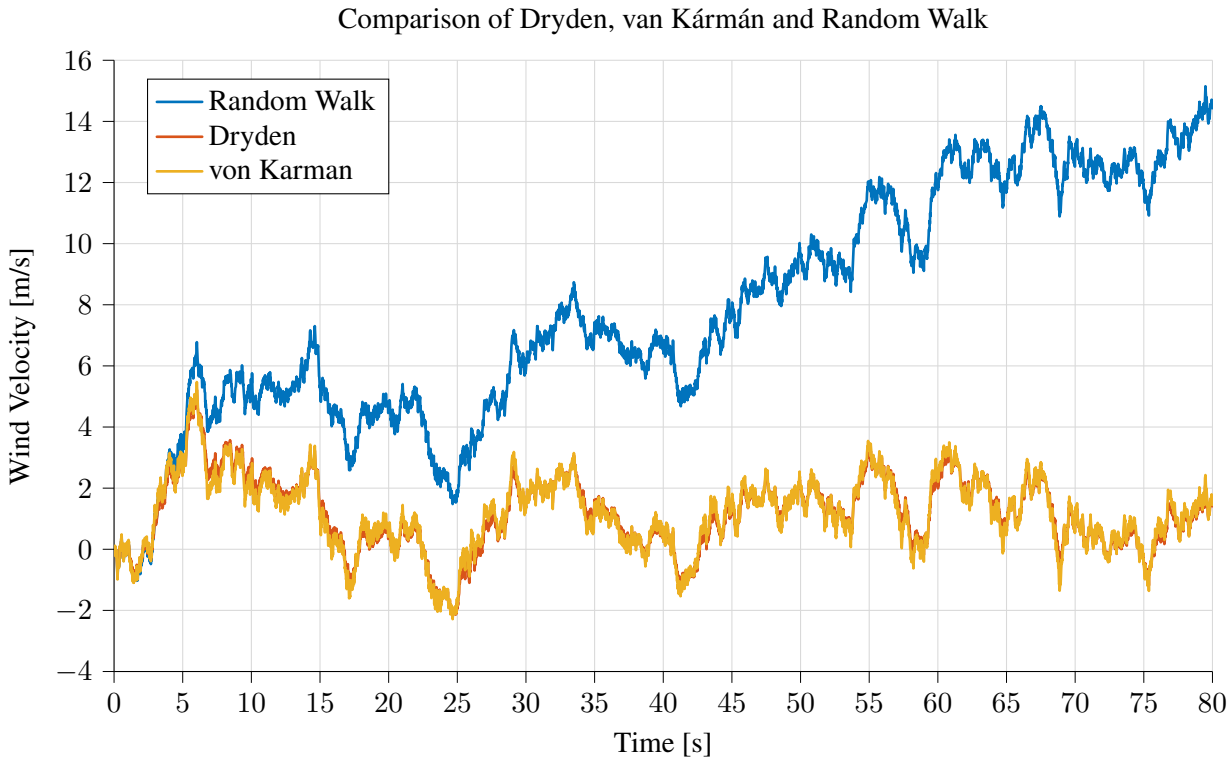


Figure 2.2.: Comparison of wind turbulence created by the random walk, von Kármán and Dryden model for identical white noise input.

2.3.1. Dryden Turbulence Model

The Dryden turbulence model, named after the American aerodynamicist Hugh L. Dryden, is a stochastic model for continuous atmospheric turbulence. The model is based on statistical descriptions of turbulence and was first formulated in terms of power spectral densities by Liepmann in 1952 [**Liepmann1952Dryden**].

Rather than generating turbulent wind directly in the time domain, the Dryden model characterizes atmospheric turbulence by its power spectral density and synthesizes the corresponding wind components by filtering white noise. As illustrated in Figure 2.3, three independent white Gaussian noise processes

are passed through shaping filters for the longitudinal, lateral, and vertical directions, resulting in the stochastic turbulence velocity vector $\boldsymbol{v}_{W,\text{turb}} = [u_{W,\text{turb}},\, v_{W,\text{turb}},\, w_{W,\text{turb}}]^T$. A white Gaussian noise process describes a sequence of independent Gaussian random variables indexed in time. Each sample is normally distributed with zero mean and constant variance, while the values at different time instants are uncorrelated.

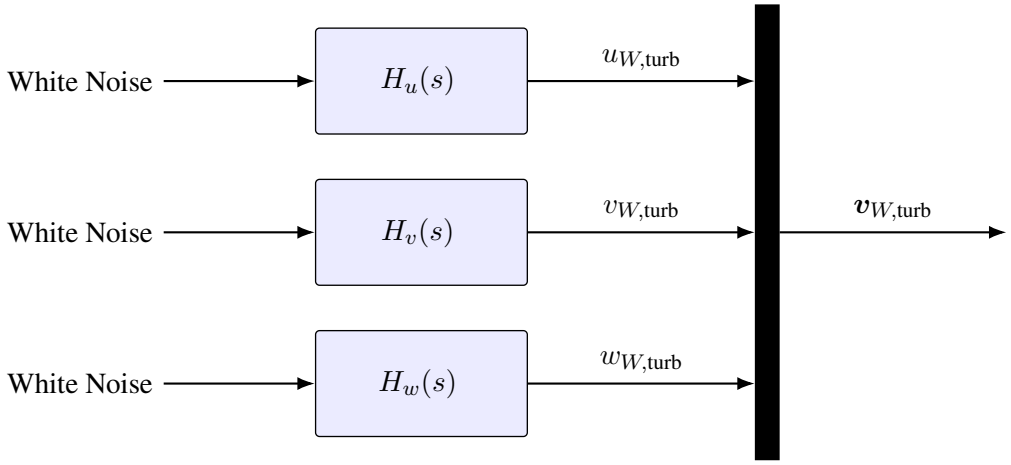


Figure 2.3.: Structure of the Dryden Turbulence model.

For this work, the low-altitude variant of the Dryden model (valid for altitudes below 300 m, as specified in MIL-F-8785C / MIL-HDBK-1797 [**MILF8785C**, **MILHDBK1797**]) is used, consistent with the near-ground operational envelope of the airship. In this formulation, the Dryden shaping filters generate the turbulence components in a coordinate system aligned with the mean wind. The longitudinal component $u_{W,\text{turb}}$ is defined along the horizontal mean wind direction, the lateral component $v_{W,\text{turb}}$ lies perpendicular to it in the horizontal plane, and the vertical component $w_{W,\text{turb}}$ is aligned with the z-axis of the Earth-fixed coordinate system. Consequently, the filter outputs are not expressed in the vehicle's body-fixed coordinate system. The symbols u, v, w are nevertheless maintained because they represent the standard notation for the Dryden turbulence components.

Since no onboard measurement of the true mean wind direction is available and only the total wind, rather than its mean and turbulent components, is estimated, the orientation of the mean-wind coordinate system is unknown. Therefore, for simplicity, the mean-wind coordinate system is assumed to coincide with the Earth-fixed coordinate system, with the mean wind aligned with the positive Earth x-axis. Under this assumption, the turbulence vector $\boldsymbol{v}_{W,\text{turb}}$ is given directly in the Earth-fixed frame.

The corresponding shaping filters, which reproduce the characteristic Dryden turbulence spectra, are defined by the following transfer functions [**MILHDBK1797**]:

$$H_u(s) = \sigma_u \sqrt{\frac{2L_u}{\pi V}} \frac{1}{1 + \frac{L_u}{V} s}, \tag{2.43}$$

$$H_v(s) = \sigma_v \sqrt{\frac{L_v}{\pi V}} \frac{1 + \frac{\sqrt{3} L_v}{V} s}{\left(1 + \frac{L_v}{V} s\right)^2}, \tag{2.44}$$

$$H_w(s) = \sigma_w \sqrt{\frac{L_w}{\pi V}} \frac{1 + \frac{\sqrt{3} L_w}{V} s}{\left(1 + \frac{L_w}{V} s\right)^2}, \tag{2.45}$$

where $V = \|\boldsymbol{v}_B\|_2$ denotes the absolute velocity of the airship and σ, L are the Dryden turbulence parameters. The longitudinal turbulence component is represented by a first-order transfer function, whereas the lateral and vertical components require second-order transfer functions due to their different spectral characteristics. A detailed derivation of the Dryden transfer functions can be found in [**MILHDBK1797**, **Yeager1998Turbulence**]. It is worth noting that for $V = 0$, such as during hovering, the generated Dryden turbulence computes to zero. The corresponding filter parameters, including the turbulence intensities σ and length scales L, are discussed in the following.

Dryden Model Parameters

The Dryden shaping filters are parameterized by the turbulence intensities σ_u, σ_v, σ_w and the turbulence scale lengths L_u, L_v, and L_w. These parameters determine the strength and spatial correlation of the generated turbulence and are specified in [**MILF8785C**]. Although the original specification defines the parameters using imperial units, the equivalent expressions in SI units are employed throughout this work.

For flight altitudes below 300 m, the turbulence scale lengths are defined as

$$L_w = h, \tag{2.46}$$

$$L_u = L_v = \frac{h}{(0.177 + 0.0027\,h)^{1.2}}, \tag{2.47}$$

where h denotes the altitude above ground in meters. The corresponding turbulence intensities are computed as

$$\sigma_w = 0.1\,W_{20}, \tag{2.48}$$

$$\sigma_u = \sigma_v = \frac{\sigma_w}{(0.177 + 0.0027\,h)^{0.4}}, \tag{2.49}$$

where W_{20} denotes the reference wind speed at an altitude of 6 m (20 ft). Consequently, the characteristics of the generated turbulence are determined by only two parameters: the flight altitude h and the reference wind speed W_{20}.

Since the turbulence intensities σ scale linearly with W_{20}, this parameter directly determines the magnitude of the generated turbulence. Therefore, an increase of W_{20} results in a proportional increase in the generated turbulence intensity. Conversely, for $W_{20} = 0$, the turbulence intensities become zero, and the Dryden model produces no turbulence contribution.

2.4. Kalman Filter

The Kalman filter (KF), introduced by Kalman in 1960 [**Kalman1960KF**], is one of the most widely used algorithms for state estimation in dynamic systems [**Maybeck1979KF**, **Särkkä2013KF**]. Its objective is to estimate the state of a system from noisy or incomplete sensor measurements by recursively combining

an uncertain mathematical model of the system with incoming measurements. At each time step, the filter first predicts the current state based on the system dynamics and then refines this prediction using the latest measurement. The prediction and measurement are weighted according to their respective uncertainties, allowing the filter to balance the information provided by the system model and the sensor measurements.

Under the assumptions of linear system dynamics and Gaussian process and measurement noise, the Kalman filter yields the optimal state estimate in the minimum mean-square error (MMSE) sense [**Simon2006KF**]. More specifically, at each time step k, it computes the state estimate that minimizes the expected squared estimation error conditioned on all available measurements:

$$\hat{\boldsymbol{x}}_k = \arg\min_{\tilde{\boldsymbol{x}}} \mathbb{E}\left[\|\boldsymbol{x}_k - \tilde{\boldsymbol{x}}\|_2^2 \,\middle|\, \boldsymbol{y}_{1:k} \right], \tag{2.50}$$

where $\boldsymbol{y}_{1:k} = \{\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_k\}$ denotes the set of all measurements available up to time step k.

Besides this optimality property, the Kalman filter allows to incorporate stochastic system models, account for uncertainties in both the system dynamics and measurements, and recursively updates the state estimate in a computationally efficient manner. This makes the Kalman filter well suited for the estimation problem considered in this thesis. In particular, atmospheric turbulence, as captured by the Dryden model (Section 2.3.1), is described as a stochastic process driven by white noise. Therefore, the Dryden turbulence model can be naturally integrated into the Kalman filtering framework to estimate the unknown wind acting on the airship.

The Kalman filter framework treats the system dynamics and measurement process as stochastic rather than deterministic. Within this framework, the state $\boldsymbol{x}_k$ and measurement $\boldsymbol{y}_k$ at each time step k are modeled as Gaussian random vectors rather than deterministic quantities. A Gaussian random vector is fully characterized by its mean and covariance matrix. The mean represents the expected value of the random vector and is defined as

$$\bar{\boldsymbol{x}} = \mathbb{E}[\boldsymbol{x}], \tag{2.51}$$

while the covariance matrix describes its deviation from the mean and is defined as

$$\boldsymbol{P}_{xx} = \mathbb{E}\left[(\boldsymbol{x} - \bar{\boldsymbol{x}})(\boldsymbol{x} - \bar{\boldsymbol{x}})^T \right]. \tag{2.52}$$

By construction, $\boldsymbol{P}_{xx}$ is symmetric and positive semidefinite, i.e., $\boldsymbol{v}^T \boldsymbol{P}_{xx} \boldsymbol{v} \geq 0$ for all $\boldsymbol{v} \in \mathbb{R}^n$, which is required for $\boldsymbol{P}_{xx}$ to represent a valid covariance matrix.

The covariance matrix serves as a measure of the uncertainty associated with a random vector. The diagonal entries of $\boldsymbol{P}_{xx}$ represent the variances of the individual components and quantify their spread around the mean, where larger variances indicate greater uncertainty. The off-diagonal entries describe the correlation between the components and indicate how the uncertainty of one component is related to the uncertainty of another.

To describe the evolution of the state over time and how the measurements relate to the system state, the Kalman filter uses a discrete-time probabilistic state-space model. To account for modeling inaccuracies and measurement uncertainties, as well as the stochastic wind, the process and measurement models are augmented by additive random vectors representing process noise and measurement noise, respectively:

$$\boldsymbol{x}_k = \boldsymbol{A}_{k-1}\boldsymbol{x}_{k-1} + \boldsymbol{B}_{k-1}\boldsymbol{u}_{k-1} + \boldsymbol{q}_{k-1}, \tag{2.53}$$

$$\boldsymbol{y}_k = \boldsymbol{C}_k\boldsymbol{x}_k + \boldsymbol{r}_k, \tag{2.54}$$

where $\boldsymbol{x}_k \in \mathbb{R}^n$ denotes the system state, $\boldsymbol{y}_k \in \mathbb{R}^m$ the measurement, $\boldsymbol{u}_k \in \mathbb{R}^p$ the known deterministic input, $\boldsymbol{A}_{k-1} \in \mathbb{R}^{n\times n}$ the state transition matrix, $\boldsymbol{B}_{k-1} \in \mathbb{R}^{n\times p}$ the input matrix, and $\boldsymbol{C}_k \in \mathbb{R}^{m\times n}$ the measurement matrix. The system matrices may be time-varying. The random vectors $\boldsymbol{q}_{k-1} \in \mathbb{R}^n$ and $\boldsymbol{r}_k \in \mathbb{R}^m$ represent the process and measurement noise, respectively. Both are modeled as zero-mean, white Gaussian noise processes with covariance matrices $\boldsymbol{Q}_{k-1} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{R}_k \in \mathbb{R}^{m\times m}$:

$$\mathbb{E}[\boldsymbol{q}_k] = 0, \quad \mathbb{E}[\boldsymbol{q}_k \boldsymbol{q}_k^T] = \boldsymbol{Q}_k \qquad \text{and} \qquad \mathbb{E}[\boldsymbol{r}_k] = 0, \quad \mathbb{E}[\boldsymbol{r}_k \boldsymbol{r}_k^T] = \boldsymbol{R}_k \qquad \forall k. \tag{2.55}$$

As a consequence of this formulation, the state and the measurement are themselves random vectors rather than deterministic quantities. Therefore, the initial condition is not a fixed value but an initial distribution,

$$\boldsymbol{x}_0 \sim \mathcal{N}(\hat{\boldsymbol{x}}_0, \boldsymbol{P}_0), \tag{2.56}$$

with initial state estimate $\hat{\boldsymbol{x}}_0 \in \mathbb{R}^n$ and covariance $\boldsymbol{P}_0 \in \mathbb{R}^{n\times n}$ assumed given.

The state estimate at any time step is defined as the mean of the state's probability distribution,

$$\hat{\boldsymbol{x}}_k = \mathbb{E}[\boldsymbol{x}_k] = \bar{\boldsymbol{x}}_k. \tag{2.57}$$

Because the initial state $\boldsymbol{x}_0$ is Gaussian, the process and measurement noise are Gaussian, and both the prediction and update steps introduced below act linearly on Gaussian random vectors, the conditional distribution of the state remains Gaussian at every time step [**Särkkä2013KF**], both before and after a measurement is incorporated:

$$\boldsymbol{x}_k \mid \boldsymbol{y}_{1:k-1} \sim \mathcal{N}(\hat{\boldsymbol{x}}_k^-, \boldsymbol{P}_k^-), \qquad \boldsymbol{x}_k \mid \boldsymbol{y}_{1:k} \sim \mathcal{N}(\hat{\boldsymbol{x}}_k, \boldsymbol{P}_k). \tag{2.58}$$

Here the superscript $(\cdot)^-$ denotes *a priori* quantities, computed before incorporating the measurement at time step k, while quantities without superscript denote *a posteriori* quantities, computed after incorporating the measurement. In both cases, the state estimate is given by the conditional mean of the state, whereas the associated covariance quantifies the remaining uncertainty of that estimate. For notational simplicity, the explicit conditioning on the available measurements is omitted in the remainder of this section.

Starting from the initial estimate $\hat{\boldsymbol{x}}_0$, the Kalman filter recursively computes the state estimate at each time step k by alternating between two steps: a prediction step and an update step. In the prediction step, the previous *a posteriori* state estimate $\hat{\boldsymbol{x}}_{k-1}$ and the system model are used to propagate the estimate and its associated uncertainty forward in time, yielding the *a priori* estimate $\hat{\boldsymbol{x}}_k^-$ and covariance $\boldsymbol{P}_k^-$. Additionally, the predicted measurement, $\hat{\boldsymbol{y}}_k^-$, corresponding to $\hat{\boldsymbol{x}}_k^-$ is calculated.

In the subsequent update step, the latest measurement is incorporated to correct the *a priori* estimate $\hat{\boldsymbol{x}}_k^-$. The correction is based on the so-called innovation, i.e., the difference between the actual and predicted measurement, and is weighted according to the relative uncertainties of the prediction and the measurement. The resulting *a posteriori* estimate $\hat{\boldsymbol{x}}_k$ incorporates all available information up to time step k. The corresponding equations are introduced in the following [**Särkkä2013KF**].

Prediction

By taking the expectation of Eq. (2.53) and using $\mathbb{E}[\boldsymbol{q}_{k-1}] = 0$, the *a priori* estimate computes to

$$\hat{\boldsymbol{x}}_k^- = \mathbb{E}[\boldsymbol{x}_k] = \boldsymbol{A}_{k-1}\hat{\boldsymbol{x}}_{k-1} + \boldsymbol{B}_{k-1}\boldsymbol{u}_{k-1}, \tag{2.59}$$

which can be computed for all k recursively given the input sequence and initial estimate $\hat{\boldsymbol{x}}_0$.

The *a priori* covariance at time step k is defined as

$$\boldsymbol{P}_k^- = \mathbb{E}\big[\,\big(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-\big)\,\big(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-\big)^T\,\big]. \tag{2.60}$$

Subtracting Eq. (2.59) from Eq. (2.53), the known input term $\boldsymbol{B}_{k-1}\boldsymbol{u}_{k-1}$ cancels, leaving the propagated estimation error,

$$\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^- = \boldsymbol{A}_{k-1}\left(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1}\right) + \boldsymbol{q}_{k-1}. \tag{2.61}$$

Substituting Eq. (2.61) into the covariance definition (2.60) yields:

$$\begin{aligned}
\boldsymbol{P}_k^- &= \mathbb{E}\Big[\big(\boldsymbol{A}_{k-1}(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1}) + \boldsymbol{q}_{k-1}\big)\big(\boldsymbol{A}_{k-1}(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1}) + \boldsymbol{q}_{k-1}\big)^T\Big] \\
&= \underbrace{\mathbb{E}\Big[\boldsymbol{A}_{k-1}(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1})(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1})^T\boldsymbol{A}_{k-1}^T\Big]}_{\boldsymbol{A}_{k-1}\boldsymbol{P}_{k-1}\boldsymbol{A}_{k-1}^T} + \underbrace{\mathbb{E}\Big[\boldsymbol{q}_{k-1}\boldsymbol{q}_{k-1}^T\Big]}_{\boldsymbol{Q}_{k-1}} \\
&\quad + 2\underbrace{\mathbb{E}\Big[\boldsymbol{A}_{k-1}(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1})\boldsymbol{q}_{k-1}^T\Big]}_{=\boldsymbol{0}} \\
&= \boldsymbol{A}_{k-1}\boldsymbol{P}_{k-1}\boldsymbol{A}_{k-1}^T + \boldsymbol{Q}_{k-1}.
\end{aligned} \tag{2.62}$$

The cross term vanishes because the process noise $\boldsymbol{q}_{k-1}$ is assumed to be uncorrelated with the state estimation error $(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1})$. Note that $\boldsymbol{P}_k^-$ in Eq. (2.62) is guaranteed to be positive semidefinite: $\boldsymbol{A}_{k-1}\boldsymbol{P}_{k-1}\boldsymbol{A}_{k-1}^T$ is PSD as a congruence transformation of the PSD matrix $\boldsymbol{P}_{k-1}$, and $\boldsymbol{Q}_{k-1}$ is PSD by definition, so their sum is PSD. Equations (2.59) and (2.62) together describe how the *a priori* mean and covariance of the state evolve under the linear system model. Accordingly, the predicted output computes to

$$\hat{\boldsymbol{y}}_k^- = \mathbb{E}[\boldsymbol{y}_k] = \boldsymbol{C}_k\hat{\boldsymbol{x}}_k^-. \tag{2.63}$$

This concludes the prediction step.

Update

In the update step, the *a priori* estimate $\hat{\boldsymbol{x}}_k^-$ obtained from the prediction step is corrected using the latest measurement $\boldsymbol{y}_k$, resulting in the *a posteriori* estimate $\hat{\boldsymbol{x}}_k$. The correction is expressed as a linear combination of the *a priori* estimate and the innovation, weighted by the Kalman gain $\boldsymbol{K}_k$:

$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^- + \boldsymbol{K}_k\tilde{\boldsymbol{y}}_k, \tag{2.64}$$

where the innovation $\tilde{\boldsymbol{y}}_k$ is defined as the difference between the actual and predicted measurement,

$$\tilde{\boldsymbol{y}}_k = \boldsymbol{y}_k - \hat{\boldsymbol{y}}_k^- = \boldsymbol{C}_k(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-) + \boldsymbol{r}_k. \tag{2.65}$$

The Kalman gain $\boldsymbol{K}_k \in \mathbb{R}^{n\times m}$ is chosen such that the *a posteriori* estimate $\hat{\boldsymbol{x}}_k$ in Eq. (2.64) minimizes the trace of the *a posteriori* covariance $\boldsymbol{P}_k$, consistent with the MMSE criterion in Eq. (2.50). The resulting optimal gain is given by [**Särkkä2013KF**]

$$\boldsymbol{K}_k = \boldsymbol{P}_k^{xy}\boldsymbol{S}_k^{-1}, \tag{2.66}$$

where $\boldsymbol{S}_k \in \mathbb{R}^{m\times m}$ denotes the innovation covariance, which quantifies the uncertainty associated with the innovation. Assuming that the measurement noise $\boldsymbol{r}_k$ is uncorrelated with the *a priori* estimation error $(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-)$, the innovation covariance can be computed using Eqs. (2.54) and (2.63) as

$$\boldsymbol{S}_k = \mathbb{E}[\tilde{\boldsymbol{y}}_k\tilde{\boldsymbol{y}}_k^T] = \underbrace{\mathbb{E}\big[\boldsymbol{C}_k(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-)(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-)^T\boldsymbol{C}_k^T\big]}_{\boldsymbol{C}_k\boldsymbol{P}_k^-\boldsymbol{C}_k^T} + \underbrace{\mathbb{E}[\boldsymbol{r}_k\boldsymbol{r}_k^T]}_{\boldsymbol{R}_k} = \boldsymbol{C}_k\boldsymbol{P}_k^-\boldsymbol{C}_k^T + \boldsymbol{R}_k, \tag{2.67}$$

and $\boldsymbol{P}_k^{xy} \in \mathbb{R}^{n\times m}$ denotes the cross-covariance between the *a priori* state estimation error and the innovation, given by

$$\boldsymbol{P}_k^{xy} = \mathbb{E}\big[(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-)\tilde{\boldsymbol{y}}_k^T\big] = \mathbb{E}\big[(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-)(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-)^T\big]\boldsymbol{C}_k^T = \boldsymbol{P}_k^-\boldsymbol{C}_k^T. \tag{2.68}$$

The *a posteriori* covariance follows by substituting Eqs. (2.65) and (2.64) into the definition of the estimation error,

$$\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k = (\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-) - \boldsymbol{K}_k\tilde{\boldsymbol{y}}_k = (\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{C}_k)(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-) - \boldsymbol{K}_k\boldsymbol{r}_k, \tag{2.69}$$

so that, following the same expansion as in Eq. (2.67):

$$\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{C}_k)\boldsymbol{P}_k^-(\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{C}_k)^T + \boldsymbol{K}_k\boldsymbol{R}_k\boldsymbol{K}_k^T. \tag{2.70}$$

Substituting the optimal gain from Eq. (2.66) simplifies this expression to:

$$\boldsymbol{P}_k = \boldsymbol{P}_k^- - \boldsymbol{K}_k\boldsymbol{S}_k\boldsymbol{K}_k^T = (\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{C}_k)\boldsymbol{P}_k^-. \tag{2.71}$$

This concludes the update step. Together, the prediction and update steps in Eqs. (2.59), (2.62), (2.64), and (2.71) form the recursive Kalman filter algorithm, alternately propagating the *a priori* estimate forward in time and correcting it into an *a posteriori* estimate as new measurements become available.

The recursive state estimate can be summarized compactly as

$$\hat{\boldsymbol{x}}_k = \underbrace{\boldsymbol{A}_{k-1}\hat{\boldsymbol{x}}_{k-1} + \boldsymbol{B}_{k-1}\boldsymbol{u}_{k-1}}_{\text{Prediction}} + \underbrace{\boldsymbol{K}_k\left(\boldsymbol{y}_k - \hat{\boldsymbol{y}}_k^-\right)}_{\text{Update}}, \tag{2.72}$$

where the first term predicts the state based on the system model, while the second term corrects this prediction using the innovation weighted by the Kalman gain.

This predictor-corrector structure is common to many state estimation methods, including deterministic observers such as the Luenberger observer [**Luenberger1964**, **Luenberger1966**]. The distinguishing feature of the Kalman filter is that it not only estimates the system state but also recursively propagates the associated estimation uncertainty through the error covariance matrix.

The presented equations describe the Kalman filter formulation for linear systems. In many real-world applications, however, the system dynamics and measurement models are inherently nonlinear. This also applies to the airship model considered in this work, whose nonlinear equations of motion require a nonlinear state estimation approach.

Several methods have been proposed to extend the Kalman filter framework to nonlinear systems, including the extended Kalman filter (EKF) and the unscented Kalman filter (UKF) [**Maybeck1979KF**, **Simon2006KF**, **Julier1997UKF**]. Both methods retain the recursive prediction-update structure of the classical Kalman filter while employing different strategies to handle nonlinearities. These two approaches are introduced in the following.

2.4.1. Extended Kalman Filter

The nonlinear discrete-time probabilistic state-space model is given by

$$\boldsymbol{x}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_{k-1}) + \boldsymbol{q}_{k-1}, \tag{2.73}$$

$$\boldsymbol{y}_k = \boldsymbol{h}(\boldsymbol{x}_k) + \boldsymbol{r}_k, \tag{2.74}$$

where $\boldsymbol{f}$ and $\boldsymbol{h}$ denote the, possibly time-varying, nonlinear state transition and measurement functions, respectively, replacing the matrices $\boldsymbol{A}_{k-1}$, $\boldsymbol{B}_{k-1}$, and $\boldsymbol{C}_k$ of the linear model in Eq. (2.53). The process noise $\boldsymbol{q}_{k-1}$ and measurement noise $\boldsymbol{r}_k$ retain the same statistical properties as in the linear case.

The extended Kalman filter (EKF) extends the classical Kalman filter to nonlinear systems by linearizing the nonlinear system around the latest available state estimate [**Simon2006**]. This results in a locally linear system representation, allowing the recursive prediction and update structure of the Kalman filter to be applied. Specifically, the nonlinear state transition function $\boldsymbol{f}$ is linearized around the latest *a posteriori* state estimate $\hat{\boldsymbol{x}}_{k-1}$ during the prediction step, while the nonlinear measurement function $\boldsymbol{h}$ is linearized around the latest *a priori* state estimate $\hat{\boldsymbol{x}}_k^-$ during the update step. The resulting first-order approximations are given by [**Särkkä2013KF**]

$$\boldsymbol{x}_k \approx \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_{k-1}) + \boldsymbol{F}_{k-1}(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1}) + \boldsymbol{q}_{k-1}, \tag{2.75}$$

$$\boldsymbol{y}_k \approx \boldsymbol{h}(\hat{\boldsymbol{x}}_k^-) + \boldsymbol{H}_k(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-) + \boldsymbol{r}_k, \tag{2.76}$$

where $\boldsymbol{F}_{k-1} \in \mathbb{R}^{n \times n}$ denotes the Jacobian of $\boldsymbol{f}$ with respect to the state, evaluated at $(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_{k-1})$, and $\boldsymbol{H}_k \in \mathbb{R}^{m \times n}$ denotes the Jacobian of $\boldsymbol{h}$ with respect to the state, evaluated at $\hat{\boldsymbol{x}}_k^-$:

$$\boldsymbol{F}_{k-1} = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_{k-1}} \quad \text{and} \quad \boldsymbol{H}_k = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\right|_{\hat{\boldsymbol{x}}_k^-} \tag{2.77}$$

Prediction

Analogously to the classical Kalman filter, the *a priori* state estimate is obtained by propagating the previous *a posteriori* estimate through the system model. Taking the expectation of the linearized state equation in Eq. (2.75) yields:

$$\begin{aligned}\hat{\boldsymbol{x}}_k^- &= \mathbb{E}[\boldsymbol{x}_k] = \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_{k-1}) + \boldsymbol{F}_{k-1}\underbrace{\mathbb{E}[\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1}]}_{=\mathbf{0}} + \underbrace{\mathbb{E}[\boldsymbol{q}_{k-1}]}_{=\mathbf{0}} \\ &= \boldsymbol{f}(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_{k-1}),\end{aligned} \tag{2.78}$$

where the second term computes to zero because the expected value of the state estimation error computes to zero: $\mathbb{E}[\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1}] = \hat{\boldsymbol{x}}_{k-1} - \hat{\boldsymbol{x}}_{k-1} = \mathbf{0}$.

The *a priori* covariance follows from the state estimation error,

$$\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^- = \boldsymbol{F}_{k-1}\left(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1}\right) + \boldsymbol{q}_{k-1}. \tag{2.79}$$

Equation (2.79) has the same structure as in the linear case in Eq. (2.61), with $\boldsymbol{A}_{k-1}$ replaced by the Jacobian $\boldsymbol{F}_{k-1}$. Applying the linear covariance propagation result from Eq. (2.62) to this locally linearized system gives the *a priori* covariance,

$$\boldsymbol{P}_k^- = \boldsymbol{F}_{k-1}\boldsymbol{P}_{k-1}\boldsymbol{F}_{k-1}^T + \boldsymbol{Q}_{k-1}. \tag{2.80}$$

The predicted measurement follows analogously by evaluating the nonlinear measurement function at the *a priori* estimate,

$$\hat{\boldsymbol{y}}_k^- = \boldsymbol{h}(\hat{\boldsymbol{x}}_k^-). \tag{2.81}$$

This concludes the prediction step.

Update

As in the linear case, the *a posteriori* estimate is obtained by correcting the *a priori* estimate with the innovation:

$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^- + \boldsymbol{K}_k\tilde{\boldsymbol{y}}_k, \tag{2.82}$$

where the innovation is given by

$$\tilde{\boldsymbol{y}}_k = \boldsymbol{y}_k - \hat{\boldsymbol{y}}_k^- = \boldsymbol{H}_k(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k^-) + \boldsymbol{r}_k. \tag{2.83}$$

Equation (2.83) has the same structure as in the linear case, with $\boldsymbol{C}_k$ replaced by the Jacobian $\boldsymbol{H}_k$. Applying the linear update equations from Eqs. (2.66)–(2.71) to this locally linearized system yields the innovation covariance, Kalman gain, and *a posteriori* covariance:

$$\boldsymbol{S}_k = \boldsymbol{H}_k\boldsymbol{P}_k^-\boldsymbol{H}_k^T + \boldsymbol{R}_k, \tag{2.84}$$

$$\boldsymbol{K}_k = \boldsymbol{P}_k^- \boldsymbol{H}_k^T \boldsymbol{S}_k^{-1}, \tag{2.85}$$

$$\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k) \boldsymbol{P}_k^-. \tag{2.86}$$

This concludes the update step.

The EKF approximates the nonlinear state transition and measurement functions by neglecting higher-order terms of the Taylor expansion and propagating only the first-order statistics of the state distribution. Consequently, the accuracy of the filter depends on the degree of nonlinearity of the system and the validity of the local linear approximation [**Simon2006**]. In contrast to the linear Kalman filter, the Jacobians $\boldsymbol{F}_{k-1}$ and $\boldsymbol{H}_k$ depend on the current state estimate and must therefore be recomputed at every time step, increasing the computational cost. Although this approach sacrifices the exact optimality guarantees of the linear Kalman filter, the EKF remains one of the most widely used nonlinear estimation methods due to its conceptual simplicity and comparatively low computational overhead relative to alternative nonlinear estimation techniques.

2.4.2. Unscented Kalman Filter

The unscented Kalman filter (UKF) avoids the local linearization employed by the EKF by propagating a deterministically chosen set of sample points, referred to as sigma points, through the nonlinear process and measurement functions $\boldsymbol{f}$ and $\boldsymbol{h}$ [**Julier2004UKF**]. The transformed sigma points are then used to reconstruct the mean and covariance of the propagated probability distribution using the so-called unscented transform. Since the nonlinear functions are evaluated directly rather than approximated by a first-order Taylor expansion, the unscented transform captures the mean and covariance of the transformed distribution with at least second-order accuracy for arbitrary nonlinear functions and third-order accuracy for Gaussian distributions, while avoiding the computation of Jacobians [**Julier2004UKF**].

Sigma Points and the Unscented Transform

The unscented transform is a method for approximating the mean and covariance of a random variable after undergoing a nonlinear transformation $\boldsymbol{y} = \boldsymbol{g}(\boldsymbol{x})$. The probability distribution of the n-dimensional Gaussian random vector $\boldsymbol{x}$ is represented by a set of $2n+1$ sigma points $\boldsymbol{\mathcal{X}}_i \in \mathbb{R}^n$, which are deterministically generated from the current mean $\bar{\boldsymbol{x}}$ and covariance $\boldsymbol{P}$ as [**Julier1997UKF**]

$$\begin{aligned}
\boldsymbol{\mathcal{X}}_0 &= \bar{\boldsymbol{x}}, \\
\boldsymbol{\mathcal{X}}_i &= \bar{\boldsymbol{x}} + \sqrt{(n+\lambda)} \left[\sqrt{\boldsymbol{P}}\right]_i, \qquad i = 1, \dots, n, \\
\boldsymbol{\mathcal{X}}_{i+n} &= \bar{\boldsymbol{x}} - \sqrt{(n+\lambda)} \left[\sqrt{\boldsymbol{P}}\right]_i, \qquad i = 1, \dots, n,
\end{aligned} \tag{2.87}$$

where $[\sqrt{\boldsymbol{P}}]_i$ denotes the i-th column of a matrix square root satisfying $\sqrt{\boldsymbol{P}}\sqrt{\boldsymbol{P}}^T = \boldsymbol{P}$ and λ is a scaling parameter defined as

$$\lambda = \alpha^2(n + \kappa) - n. \tag{2.88}$$

The parameters α and κ determine the spread of the sigma points around the mean, while β incorporates prior knowledge of the distribution shape into the covariance weighting. A common default choice is $\kappa = 0$ with α chosen small and positive to keep the sigma points close to $\bar{\boldsymbol{x}}$. For Gaussian random vectors $\beta = 2$ is optimal [**Wan2001UKF**].

The sigma points $\boldsymbol{\mathcal{X}}_i$ are then propagated through the nonlinear function:

$$\boldsymbol{\mathcal{Y}}_i = \boldsymbol{g}(\boldsymbol{\mathcal{X}}_i), \qquad i = 0, \ldots, 2n \tag{2.89}$$

to obtain the set of transformed sigma points $\boldsymbol{\mathcal{Y}}_i$. The mean and covariance of the transformed distribution $\boldsymbol{y}$ are reconstructed as the weighted sample mean and covariance of the propagated sigma points:

$$\bar{\boldsymbol{y}} = \sum_{i=0}^{2n} W_i^m \, \boldsymbol{\mathcal{Y}}_i, \tag{2.90}$$

$$\boldsymbol{P}_{yy} = \sum_{i=0}^{2n} W_i^c \, (\boldsymbol{\mathcal{Y}}_i - \bar{\boldsymbol{y}}) \, (\boldsymbol{\mathcal{Y}}_i - \bar{\boldsymbol{y}})^T \,, \tag{2.91}$$

with weights for mean W_i^m and covariance W_i^c defined as [**Wan2001UKF**]

$$\begin{aligned} W_0^m &= \frac{\lambda}{n+\lambda}, \\ W_0^c &= \frac{\lambda}{n+\lambda} + (1 - \alpha^2 + \beta), \\ W_i^m &= W_i^c = \frac{1}{2(n+\lambda)}, \qquad i = 1, \ldots, 2n. \end{aligned} \tag{2.92}$$

Since the weights depend only on the state dimension n and the scaling parameters α, β, and κ, they remain constant throughout the filtering process and can therefore be computed once prior to filter execution. In the UKF algorithm, the unscented transform is applied separately during the prediction and update steps by propagating the sigma points through the nonlinear state transition function $\boldsymbol{f}$ and the nonlinear measurement function $\boldsymbol{h}$, respectively.

Prediction

A set of sigma points $\boldsymbol{\mathcal{X}}_{k-1,i}$ is generated according to Eq. (2.87) from the previous *a posteriori* estimate $\hat{\boldsymbol{x}}_{k-1}$ and covariance $\boldsymbol{P}_{k-1}$. The sigma points are then propagated individually through the nonlinear state transition function:

$$\boldsymbol{\mathcal{X}}_{k,i}^- = \boldsymbol{f}(\boldsymbol{\mathcal{X}}_{k-1,i}, \boldsymbol{u}_{k-1}), \qquad i = 0, \ldots, 2n. \tag{2.93}$$

Applying the unscented transform in Eqs. (2.90) and (2.91) to the propagated sigma points yields the *a priori* estimate and covariance [**Wan2001UKF**],

$$\hat{\boldsymbol{x}}_k^- = \sum_{i=0}^{2n} W_i^m \boldsymbol{\mathcal{X}}_{k,i}^-, \tag{2.94}$$

$$\boldsymbol{P}_k^- = \sum_{i=0}^{2n} W_i^c \left(\boldsymbol{\mathcal{X}}_{k,i}^- - \hat{\boldsymbol{x}}_k^-\right)\left(\boldsymbol{\mathcal{X}}_{k,i}^- - \hat{\boldsymbol{x}}_k^-\right)^T + \boldsymbol{Q}_{k-1}, \tag{2.95}$$

where the process noise covariance $\boldsymbol{Q}_{k-1}$ is added directly, consistent with the additive noise formulation in Eq. (2.73).

To incorporate the computed *a priori* estimate into the output prediction, a new set of sigma points $\boldsymbol{\mathcal{X}}_{k,i}$ is generated according to Eq. (2.87) from the *a priori* estimate $\hat{\boldsymbol{x}}_k^-$ and covariance $\boldsymbol{P}_k^-$. The predicted measurement is obtained by propagating the redrawn sigma points through the nonlinear measurement function and applying the unscented transform (2.90):

$$\boldsymbol{\mathcal{Y}}_{k,i} = \boldsymbol{h}(\boldsymbol{\mathcal{X}}_{k,i}), \qquad i = 0, \dots, 2n, \tag{2.96}$$

$$\hat{\boldsymbol{y}}_k^- = \sum_{i=0}^{2n} W_i^m \boldsymbol{\mathcal{Y}}_{k,i}. \tag{2.97}$$

This concludes the prediction step.

Update

As in the linear Kalman filter and the EKF, the *a posteriori* estimate is obtained by correcting the *a priori* estimate with the innovation:

$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^- + \boldsymbol{K}_k \tilde{\boldsymbol{y}}_k, \tag{2.98}$$

The innovation covariance and the cross-covariance between the *a priori* state and the predicted measurement are reconstructed from the propagated sigma points using the unscented transform:

$$\boldsymbol{S}_k = \sum_{i=0}^{2n} W_i^c \left(\boldsymbol{\mathcal{Y}}_{k,i} - \hat{\boldsymbol{y}}_k^-\right)\left(\boldsymbol{\mathcal{Y}}_{k,i} - \hat{\boldsymbol{y}}_k^-\right)^T + \boldsymbol{R}_k, \tag{2.99}$$

$$\boldsymbol{P}_k^{xy} = \sum_{i=0}^{2n} W_i^c \left(\boldsymbol{\mathcal{X}}_{k,i}^- - \hat{\boldsymbol{x}}_k^-\right)\left(\boldsymbol{\mathcal{Y}}_{k,i} - \hat{\boldsymbol{y}}_k^-\right)^T, \tag{2.100}$$

where the measurement noise covariance $\boldsymbol{R}_k$ is added directly to $\boldsymbol{S}_k$, analogous to $\boldsymbol{Q}_{k-1}$ in Eq. (2.95). As in the linear Kalman filter, the Kalman gain and *a posteriori* covariance follow from Eqs. (2.66) and (2.71),

$$\boldsymbol{K}_k = \boldsymbol{P}_k^{xy} \boldsymbol{S}_k^{-1}, \tag{2.101}$$

$$\boldsymbol{P}_k = \boldsymbol{P}_k^- - \boldsymbol{K}_k \boldsymbol{S}_k \boldsymbol{K}_k^T. \tag{2.102}$$

This concludes the update step.

The UKF approximates the propagation of the state distribution by deterministically sampling a set of sigma points and propagating them through the nonlinear process and measurement functions. Consequently, it captures the effect of nonlinear transformations on the mean and covariance more accurately

than the EKF, while avoiding the first-order linearization error associated with Taylor series approximations [**Julier2004UKF**]. In contrast to the EKF, no Jacobians of $\boldsymbol{f}$ or $\boldsymbol{h}$ are required. Instead, the nonlinear functions are evaluated directly at $2n + 1$ sigma points, resulting in a computational cost that scales with the state dimension. Although the UKF still relies on a finite set of sample points and therefore does not provide the exact optimality guarantees of the linear Kalman filter, its improved accuracy relative to the EKF at comparable computational cost has made it a widely used alternative for nonlinear state estimation.

2.5. Disturbance Compensation

Compensation methods aim to improve the disturbance rejection and tracking performance of a control system by reducing the influence of known or estimated disturbances and compensating for modeled system dynamics. Rather than relying solely on feedback to correct errors after they occur, compensation methods utilize additional information, such as system models, disturbance estimates, or reference dynamics, to generate corrective control actions proactively. The effectiveness of such approaches depends strongly on the accuracy of the system model and the available disturbance information. Consequently, compensation methods are typically combined with a feedback controller, which compensates for remaining modeling errors and unmeasured disturbances while ensuring closed-loop stability [**Astrom2010FeedbackSystems**, **Lunze2013Regelungstechnik2**].

The general structure of a compensation-based control approach is depicted in Fig. 2.4.

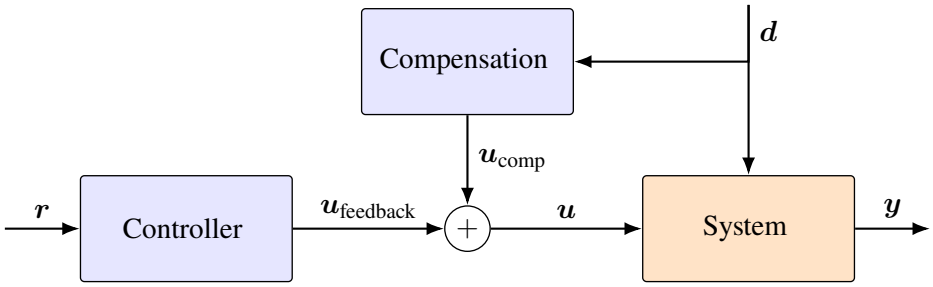


Figure 2.4.: Structure of a compensation-based control approach. Both the controller and the compensation module may additionally use available system outputs of the airship. For clarity, the corresponding signal paths are omitted.

2.5.1. Disturbance Feedforward

Disturbance feedforward (DFF) is an explicit model-based disturbance compensation strategy that utilizes measurements or estimates of external disturbances to generate a compensating control input before the disturbance affects the controlled output.

Consider the linear time-invariant (LTI) system,

$$\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{u} + \boldsymbol{E}\boldsymbol{d}, \tag{2.103}$$

where $\boldsymbol{x} \in \mathbb{R}^n$ denotes the system state, $\boldsymbol{u} \in \mathbb{R}^p$ the control input, and $\boldsymbol{d} \in \mathbb{R}^q$ a measurable or estimated disturbance acting on the system through the disturbance input matrix $\boldsymbol{E} \in \mathbb{R}^{n\times q}$.

Now the feedback control law is augmented with a disturbance feedforward compensation term:

$$\boldsymbol{u} = \boldsymbol{u}_{\text{feedback}} + \boldsymbol{u}_{\text{comp}}, \tag{2.104}$$

where $\boldsymbol{u}_{\text{feedback}}$ denotes the nominal feedback control input and $\boldsymbol{u}_{\text{comp}}$ the additional compensation input designed to counteract the disturbance influence. The compensation input can be selected as [**Lunze2013Regelungstechnik2**]

$$\boldsymbol{u}_{\text{comp}} = -\boldsymbol{B}^{\dagger}\boldsymbol{E}\boldsymbol{d}, \tag{2.105}$$

where $\boldsymbol{B}^{\dagger}$ denotes the Moore-Penrose pseudoinverse of the input matrix. For a square and invertible input matrix, the pseudoinverse reduces to the ordinary inverse, i.e., $\boldsymbol{B}^{\dagger} = \boldsymbol{B}^{-1}$. In this case, the compensated closed-loop dynamics become

$$\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{u}_{\text{feedback}}, \tag{2.106}$$

which are independent of the disturbance.

The same principle can be extended to nonlinear systems. Consider the nonlinear input affine dynamics [**Isidori1995NonlinearControlSystems**],

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{g}(\boldsymbol{x})\boldsymbol{u} + \boldsymbol{g}_d(\boldsymbol{x})\boldsymbol{d}, \tag{2.107}$$

where $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^n$, $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^{n\times p}$, and $\boldsymbol{g}_d : \mathbb{R}^n \to \mathbb{R}^{n\times q}$ denote the nonlinear system dynamics, the input distribution matrix, and the disturbance distribution matrix, respectively. As for the linear case, the control input is augmented with a compensation feedforward term:

$$\boldsymbol{u}_{\text{comp}} = -\boldsymbol{g}^{\dagger}(\boldsymbol{x})\boldsymbol{g}_d(\boldsymbol{x})\boldsymbol{d}. \tag{2.108}$$

For an invertible input matrix $\boldsymbol{g}(\boldsymbol{x})$, this results in the compensated dynamics

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{g}(\boldsymbol{x})\boldsymbol{u}_{\text{feedback}}, \tag{2.109}$$

which are independent of the disturbance.

2.5.2. Nonlinear Dynamic Inversion

Nonlinear dynamic inversion (NDI) is a model-based control strategy that applies the principles of feedback linearization. NDI exploits an accurate nonlinear model of the system to algebraically cancel its known nonlinear dynamics and transform the nonlinear system into an approximately linear input-output behavior [**Stevens2015AircraftControl**, **Slotine1991AppliedNC**]. In contrast to disturbance feedforward, NDI does not only compensate the effect of external disturbances but incorporates them into the nonlinear system model and computes the control input required to compensate the complete known system dynamics.

The nonlinear dynamics are described by

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{d}) + \boldsymbol{g}(\boldsymbol{x})\boldsymbol{u}, \tag{2.110}$$

where $\boldsymbol{f} : \mathbb{R}^n \times \mathbb{R}^q \to \mathbb{R}^n$ represents the known nonlinear system dynamics including the influence of the disturbance and $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^{n\times p}$ denotes the input distribution matrix.

The control input is augmented as

$$\boldsymbol{u} = \boldsymbol{u}_{\text{feedback}} + \boldsymbol{u}_{\text{comp}}, \tag{2.111}$$

where the feedback term is responsible for defining the desired closed-loop behavior, whereas the compensation term is designed to cancel the known nonlinear system dynamics and disturbance effects. The compensation term is therefore selected as

$$\boldsymbol{u}_{\text{comp}} = -\boldsymbol{g}^{\dagger}(\boldsymbol{x})\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{d}). \tag{2.112}$$

Substituting the compensation term into the system dynamics (2.110) results in the compensated dynamics, assuming that the nonlinear dynamics can be exactly inverted through the input distribution matrix:

$$\dot{\boldsymbol{x}} = \boldsymbol{g}(\boldsymbol{x})\boldsymbol{u}_{\text{feedback}}. \tag{2.113}$$

Thus, the nonlinearities and known disturbance effects included in $\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{d})$ are removed from the system description, leaving the feedback controller to determine the remaining closed-loop behavior.

2.6. Artificial Neural Networks

Artificial neural networks (ANNs) are a class of nonlinear function approximators. Formally, an ANN realizes a parameterized mapping

$$\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m, \quad \hat{\boldsymbol{y}} = \boldsymbol{f}(\boldsymbol{x}; \boldsymbol{\theta}), \tag{2.114}$$

from an input $\boldsymbol{x} \in \mathbb{R}^n$ to an approximated output $\hat{\boldsymbol{y}} \in \mathbb{R}^m$, where $\boldsymbol{\theta}$ denotes the set of trainable parameters. Rather than encoding an explicit analytical relationship, ANNs learn this mapping directly from data, making them attractive for capturing complex relationships at the cost of interpretability and formal guarantees on generalization. These properties have earned such models the label black box.

ANNs with a sufficiently wide hidden layer can, in principle, approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary accuracy, as formalized by the universal approximation theorem [**Hornik1989ApprThm**]. While this result establishes the theoretical expressive power of neural networks, it gives no guarantee on the number of neurons required, nor on whether a network of practical size can be trained to realize such an approximation.

The basic computational unit of an ANN is the neuron. Given an input vector $\boldsymbol{x} \in \mathbb{R}^n$, a single neuron computes a weighted sum of its inputs, adds a bias term, and applies a nonlinear activation function to compute the so-called activation $a \in \mathbb{R}$:

$$a = \varphi\left(\boldsymbol{w}^T\boldsymbol{x} + b\right), \tag{2.115}$$

where $\boldsymbol{w} \in \mathbb{R}^n$ denotes the weight vector and $b \in \mathbb{R}$ the bias, together constituting the neuron's trainable parameters $\boldsymbol{\theta}$. Common choices for the activation function $\varphi(\cdot)$ include the sigmoid function, the hyperbolic tangent, and the rectified linear unit (ReLU) [**Goodfellow2016DeepLearning**]. The nonlinearity introduced by the activation function is essential: without it, any composition of layers reduces to a single affine transformation, and the network could not represent nonlinear input–output relationships.

Neurons are organized into layers, with each layer's outputs serving as the next layer's inputs. A network typically consists of an input layer, one or more hidden layers, and an output layer. The number of layers is referred to as the network's depth, and the number of neurons per layer as its width.

The ANN is fully determined by its architecture (depth and width) and its parameters, collectively the weights and biases of all neurons. The parameters of the network are adapted through a training process starting from initial values, typically drawn at random. Given a dataset of input-output pairs $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{N}$, training aims at minimizing a loss function $\mathcal{L}(\boldsymbol{\theta})$ that quantifies the discrepancy between the network's predictions $\hat{\boldsymbol{y}}_i = \boldsymbol{f}(\boldsymbol{x}_i; \boldsymbol{\theta})$ and the target outputs $\boldsymbol{y}_i$. For regression tasks – as considered in this work – a common choice is the mean squared error (MSE):

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \| \boldsymbol{f}(\boldsymbol{x}_i; \boldsymbol{\theta}) - \boldsymbol{y}_i \|_2^2 \,. \tag{2.116}$$

To minimize the loss function $\mathcal{L}(\boldsymbol{\theta})$, the network parameters $\boldsymbol{\theta}$ are updated via gradient-based optimization: the gradient $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ is computed efficiently for all layers using backpropagation, which applies the chain rule to propagate the error signal from the output layer backward through the network [**Rumelhart1986Backprop**]. The parameters are then updated in the direction of steepest descent,

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \, \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_k), \tag{2.117}$$

where η is the learning rate, a hyperparameter governing the step size of each update. In practice, plain gradient descent as in (2.117) converges slowly and is sensitive to the choice of η. Optimizers, such as Adam [**Kingma2014Adam**], extend this basic update rule with momentum and adaptive per-parameter learning rates to improve convergence speed and stability.

The available dataset is typically partitioned into a training set, used to compute $\mathcal{L}(\boldsymbol{\theta})$ and update the parameters, and a validation set, held out from training and used to monitor the network's performance on unseen data. Since a sufficiently large network can fit the training data arbitrarily well without generalizing – a phenomenon known as overfitting – validation performance provides a more meaningful indicator of the model's predictive capability.

One full pass through the training dataset is referred to as an epoch. Training typically requires many epochs for the loss to converge. Rather than computing the loss and gradient over the entire dataset at once, the training data is commonly divided into smaller mini-batches, with a parameter update performed after each.

The following sections introduce two network architectures relevant to this thesis: the multi-layer perceptron and the long short-term memory network.

2.6.1. Multi-Layer Perceptron

The multi-layer perceptron (MLP) is the most basic feedforward neural network architecture [**Goodfellow2016DeepLearning**] in which information flows strictly in one direction, from the input layer through one or more hidden layers to the output layer, with no feedback connections. Every neuron in a given layer is connected to every neuron in the subsequent layer. Such layers are referred to as fully connected, or dense, layers. Figure 2.5 illustrates the structure of the MLP.

For a network with L layers, the output of layer l is computed from the output of the preceding layer as

$$\boldsymbol{a}^{[l]} = \varphi^{[l]} \left(\boldsymbol{W}^{[l]} \boldsymbol{a}^{[l-1]} + \boldsymbol{b}^{[l]} \right), \qquad l = 1, \ldots, L, \tag{2.118}$$

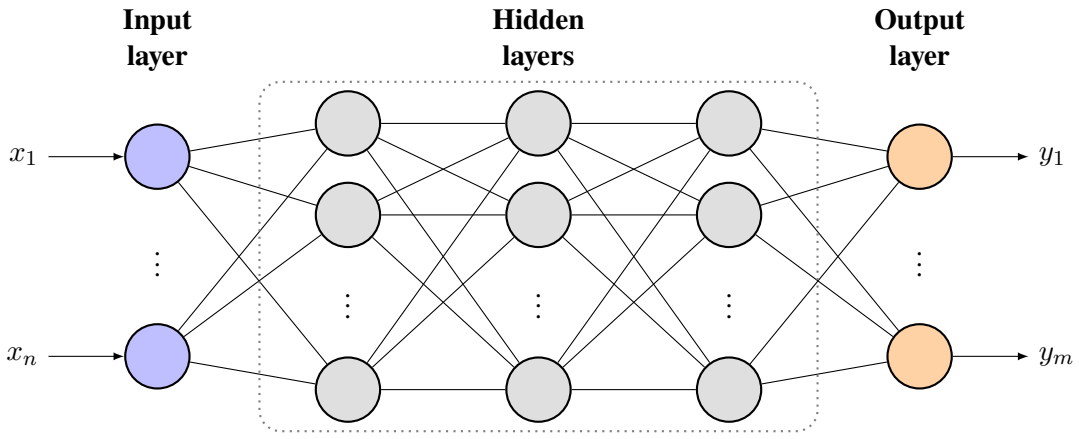


Figure 2.5.: Schematic illustration of a multi-layer perceptron with n inputs, three fully connected hidden layers, and m outputs. Each circle represents a single neuron, computing a weighted sum of its inputs followed by a nonlinear activation, as defined in (2.115).

where $\boldsymbol{a}^{[0]} = \boldsymbol{x}$ denotes the network input, $\boldsymbol{W}^{[l]}$ and $\boldsymbol{b}^{[l]}$ the weight matrix and bias vector of layer l, $\varphi^{[l]}(\cdot)$ the (possibly layer-dependent) activation function, and $\hat{\boldsymbol{y}} = \boldsymbol{a}^{[L]}$ the network output. Each row of $\boldsymbol{W}^{[l]}$ contains the weight vector of a single neuron in layer l, so that (2.118) applies (2.115) to every neuron in the layer simultaneously. Evaluating (2.118) successively for $l = 1, \ldots, L$ constitutes the forward pass of the network, mapping an input $\boldsymbol{x}$ to a prediction $\hat{\boldsymbol{y}}$.

Hidden layers typically employ a nonlinear activation function $\varphi^{[l]}(\cdot)$, enabling the network to represent nonlinear input-output relationships. The output layer's activation function is chosen depending on the task: for regression problems, as considered in this work, it is common to use a linear activation, i.e., $\varphi^{[L]}(\boldsymbol{z}) = \boldsymbol{z}$, allowing the network output to take any real value.

Owing to its purely feedforward structure, the MLP has no internal notion of time or sequence order: each input $\boldsymbol{x}$ is mapped to an output independently of any previous inputs. This makes the MLP well suited to static, memoryless input-output mappings, but limits its applicability to problems involving temporal dependencies. This motivates the recurrent architecture introduced in the following section.

2.6.2. Long Short-Term Memory Networks

Unlike the MLP, recurrent neural networks (RNNs) are designed to process sequential data by introducing feedback connections. The network's output at a given time step depends not only on the current input, but also on information from previous time steps. This is achieved through a hidden state $\boldsymbol{h}_t$, which is updated at every time step t and passed forward to influence the processing of subsequent inputs.

A well-known limitation of simple RNNs is their difficulty in learning long-range dependencies. As the temporal distance between a relevant input and the point at which it affects the output grows, simple RNNs become increasingly unable to retain and exploit that information [**Bengio1994DownsideRNN**]. The underlying reason is the vanishing/exploding gradient problem: during training, gradients propagated backward through many time steps are repeatedly scaled by the same recurrent weight matrix and activation derivatives, causing them to shrink toward zero or grow unboundedly as the sequence length increases [**Hochreiter1991ExplGradients**]. The long short-term memory (LSTM) network was proposed to address this limitation by introducing a more elaborate recurrent unit, capable of retaining and modeling long-range dependencies [**Hochreiter1997LSTM**].

The core idea of the LSTM is the cell state $\boldsymbol{c}_t$, which acts as an internal memory that is propagated from one time step to the next with only minor, additive modifications, allowing information to persist over many time steps without being repeatedly transformed. Access to and modification of the cell state are regulated by three gates, the forget gate [**Gers2000forget**], input gate, and output gate, each implemented as a small feedforward layer with a sigmoid activation, producing values between 0 and 1 that determine the extent to which information is retained, added, or exposed.

The LSTM processes a sequence of inputs $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T$, where T denotes the sequence length, or time horizon, over which the network operates. Unlike the network depth or width, T is not a property of the LSTM architecture itself, but a choice made when constructing the training data, determining how much past information is made available to the network at each prediction.

At each time step t, given the current input $\boldsymbol{x}_t$, the previous hidden state $\boldsymbol{h}_{t-1}$, and the previous cell state $\boldsymbol{c}_{t-1}$, the LSTM computes

$$\boldsymbol{f}_t = \sigma\left(\boldsymbol{W}_f \boldsymbol{x}_t + \boldsymbol{U}_f \boldsymbol{h}_{t-1} + \boldsymbol{b}_f\right), \tag{2.119}$$

$$\boldsymbol{i}_t = \sigma\left(\boldsymbol{W}_i \boldsymbol{x}_t + \boldsymbol{U}_i \boldsymbol{h}_{t-1} + \boldsymbol{b}_i\right), \tag{2.120}$$

$$\boldsymbol{o}_t = \sigma\left(\boldsymbol{W}_o \boldsymbol{x}_t + \boldsymbol{U}_o \boldsymbol{h}_{t-1} + \boldsymbol{b}_o\right), \tag{2.121}$$

$$\tilde{\boldsymbol{c}}_t = \tanh\left(\boldsymbol{W}_c \boldsymbol{x}_t + \boldsymbol{U}_c \boldsymbol{h}_{t-1} + \boldsymbol{b}_c\right), \tag{2.122}$$

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t, \tag{2.123}$$

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t), \tag{2.124}$$

where $\boldsymbol{f}_t$, $\boldsymbol{i}_t$, and $\boldsymbol{o}_t$ denote the forget, input, and output gate activations, $\tilde{\boldsymbol{c}}_t$ a candidate cell state, $\sigma(\cdot)$ the sigmoid function, and $\odot$ the element-wise (Hadamard) product. The matrices $\boldsymbol{W}_{(\cdot)}$ and $\boldsymbol{U}_{(\cdot)}$ and the vectors $\boldsymbol{b}_{(\cdot)}$ are the trainable weights and biases associated with each gate and the candidate cell state, collectively constituting the LSTM's parameters $\boldsymbol{\theta}$.

The forget gate (2.119) determines which parts of the previous cell state $\boldsymbol{c}_{t-1}$ are retained, the input gate (2.120) determines which parts of the candidate cell state $\tilde{\boldsymbol{c}}_t$ are added, and the output gate (2.121) determines which parts of the cell state are exposed as the new hidden state $\boldsymbol{h}_t$. Because the cell state update in (2.123) is additive rather than a repeated nonlinear transformation, gradients can propagate through many time steps largely unimpeded, mitigating the vanishing gradient problem observed in simple RNNs.

As with the MLP, the LSTM's parameters are trained by minimizing a loss function via backpropagation and gradient-based optimization, as described in Section 2.6. In the recurrent setting, this is commonly referred to as backpropagation through time, since gradients are propagated backward not only through the network's layers but also through the sequence of time steps [**Werbos1990BPTT**]. Owing to their ability to retain and selectively update information over extended sequences, LSTMs are well suited to modeling temporal dependencies in sequential data, such as the wind and motion signals considered in this thesis.

3. model-based wind estimation and compensation

This chapter presents the implementation of the proposed model-based wind estimation and compensation algorithms. The first part focuses on the implementation of the Kalman filter used for wind estimation, including the required model formulation and discretization. In the second part, the estimated wind velocity is utilized to implement the disturbance compensation methods.

3.1. Kalman Filter

The implementation of the Kalman filter requires a discrete-time state-space model of the airship dynamics that captures the influence of wind. Starting from the nonlinear airship model, the Dryden wind model is incorporated to obtain an augmented state-space representation. Subsequently, the continuous-time system is discretized and the corresponding process and measurement noise covariance matrices are derived. Based on this discrete-time model, the extended Kalman filter (EKF) and the unscented Kalman filter (UKF) are implemented for recursive wind estimation.

3.1.1. State-Space Description

A valid and complete state-space description is the fundamental prerequisite for implementing a Kalman filter. While the filter algorithm itself follows a well-defined prediction and correction procedure, its performance depends heavily on an accurate mathematical representation of the system dynamics and the available measurements [**Särkka2013KF**]. Consequently, deriving an appropriate state-space description is the first step toward implementing the wind estimation and compensation strategy presented in this thesis.

The airship model derived in Section 2.2 describes the translational and rotational dynamics using the generalized velocity vector,

$$ {}_B\boldsymbol{\nu} = \begin{bmatrix} {}_B\boldsymbol{v}_B \\ {}_B\boldsymbol{\omega}_B \end{bmatrix}, \tag{3.1} $$

where ${}_B\boldsymbol{v}_B$ and ${}_B\boldsymbol{\omega}_B$ denote the translational and angular velocities expressed in the body-fixed coordinate system. The rigid-body dynamics depend on the generalized forces acting at the COG and have been derived as

$$ \boldsymbol{M}\,{}_B\dot{\boldsymbol{\nu}} = {}_B\boldsymbol{\mathcal{F}}^{\mathrm{COG}}_{\mathrm{grav}} + {}_B\boldsymbol{\mathcal{F}}^{\mathrm{COG}}_{\mathrm{buoy}} + {}_B\boldsymbol{\mathcal{F}}^{\mathrm{COG}}_{\mathrm{aero}} + {}_B\boldsymbol{\mathcal{F}}^{\mathrm{COG}}_{\mathrm{prop}} - {}_B\boldsymbol{\mathcal{F}}^{\mathrm{COG}}_{\mathrm{cor}}. \tag{3.2} $$

Although Eq. (3.2) describes the motion of the airship, it does not represent a complete state-space description yet.

First, the generalized forces acting on the airship do not depend on $\boldsymbol{\nu}$ alone. For example, $\boldsymbol{\mathcal{F}}_{\text{grav}}$ and $\boldsymbol{\mathcal{F}}_{\text{buoy}}$ must be transformed into the body-fixed coordinate system and thus depend on the attitude $\boldsymbol{\Theta}$ of the airship. To obtain a state-space model whose state evolution depends only on the current state and inputs, the attitude must be included in the state vector.

Furthermore, the state-space model requires an output equation relating the state to the available onboard sensor measurements. As described in 1.3.1, the airship is equipped with several onboard measurement systems, providing measurements of its position, attitude, translational velocity, and angular velocity. Therefore, to express the measurements as a function of the state, both position $\boldsymbol{p}$ and attitude $\boldsymbol{\Theta}$ must be included in the state.

Consequently, the state vector is defined as

$$\boldsymbol{x} = \begin{bmatrix} {}_E\boldsymbol{p} \\ \boldsymbol{\Theta} \\ {}_B\boldsymbol{v}_B \\ {}_B\boldsymbol{\omega}_B \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ \phi \\ \theta \\ \psi \\ u \\ v \\ w \\ p \\ q \\ r \end{bmatrix}, \tag{3.3}$$

where ${}_E\boldsymbol{p}$ represents the position in the earth-fixed coordinate system, whose kinematics are given by

$${}_E\dot{\boldsymbol{p}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = \boldsymbol{R}_{EB}\,{}_B\boldsymbol{v}_B, \tag{3.4}$$

and $\boldsymbol{\Theta}$ contains the attitude angles, with respective kinematics [**Fossen2021Handbook**]:

$$\dot{\boldsymbol{\Theta}} = \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \dfrac{\sin\phi}{\cos\theta} & \dfrac{\cos\phi}{\cos\theta} \end{bmatrix} {}_B\boldsymbol{\omega}_B = \boldsymbol{J}(\boldsymbol{\Theta})\,{}_B\boldsymbol{\omega}_B. \tag{3.5}$$

Combining the rigid-body dynamics in Eq. (3.2) with the kinematic equations for position and attitude yields the following nonlinear state-space model. Since the onboard sensor suite provides measurements of all twelve state variables, the output equation reduces to the identity mapping:

$$\dot{\boldsymbol{x}} = \begin{bmatrix} {}_E\dot{\boldsymbol{p}} \\ \dot{\boldsymbol{\Theta}} \\ {}_B\dot{\boldsymbol{v}}_B \\ {}_B\dot{\boldsymbol{\omega}}_B \end{bmatrix} = \begin{bmatrix} \boldsymbol{R}_{EB}\,{}_B\boldsymbol{v}_B \\ \boldsymbol{J}(\boldsymbol{\Theta})\,{}_B\boldsymbol{\omega}_B \\ \boldsymbol{M}^{-1}\left(\boldsymbol{\mathcal{F}}_{\text{grav}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{buoy}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{aero}}(\boldsymbol{x}, {}_B\boldsymbol{v}_A) + \boldsymbol{\mathcal{F}}_{\text{prop}}(\boldsymbol{u}) - \boldsymbol{\mathcal{F}}_{\text{cor}}(\boldsymbol{x})\right) \end{bmatrix}, \tag{3.6}$$

$$\boldsymbol{y} = \boldsymbol{h}(\boldsymbol{x}) = \boldsymbol{x}. \tag{3.7}$$

However, the aerodynamic forces and moments $\boldsymbol{\mathcal{F}}_{\text{aero}}$ depend explicitly on the aerodynamic velocity, which is influenced by the wind velocity. Therefore, the wind dynamics must be incorporated into the state representation to obtain a complete model suitable for wind estimation. The incorporation of the wind model and the resulting augmented state-space description are presented in Section 3.1.2.

The state-space model derived above represents a deterministic description of the airship dynamics. In practice, however, both the system model and the sensor measurements are subject to uncertainty. Modeling inaccuracies arise primarily from the aerodynamic and propulsion models due to parameter uncertainties, simplifications, and unmodeled effects, while sensor measurements are affected by measurement noise. To account for these uncertainties, the system and measurement equations are augmented with stochastic process and measurement noise, respectively.

The process noise is modeled as additive zero-mean Gaussian noise. Since the airship dynamics are formulated on a force and moment level, the process noise is introduced at the same level. This preserves the physical structure of the equations of motion while directly representing uncertainties in the modeled aerodynamic and propulsion forces and moments. The generalized force and moment vector is therefore modeled as

$$\boldsymbol{\mathcal{F}} = \boldsymbol{\mathcal{F}}_{\text{model}} + \boldsymbol{\zeta} \qquad \text{with} \qquad \boldsymbol{\zeta} \sim \mathcal{N}\left(\mathbf{0}, \boldsymbol{\Sigma}_{\zeta}\right), \tag{3.8}$$

where $\boldsymbol{\mathcal{F}}_{\text{model}}$ denotes the nominal generalized force and moment vector obtained from the airship model, and $\boldsymbol{\zeta} \in \mathbb{R}^6$ represents the modeling uncertainty associated with the aerodynamic and propulsion components. To reflect the assumption that modeling errors increase with the magnitude of the acting forces and moments, the standard deviation is chosen proportional to the corresponding aerodynamic and propulsion generalized forces,

$$\boldsymbol{\sigma}_{\zeta} = \alpha_{\text{aero}} \left|\boldsymbol{\mathcal{F}}_{\text{aero}}\right| + \alpha_{\text{prop}} \left|\boldsymbol{\mathcal{F}}_{\text{prop}}\right|, \tag{3.9}$$

where $|\cdot|$ denotes the element-wise absolute value and α_{aero} and α_{prop} are tuning parameters that determine the relative uncertainty associated with the aerodynamic and propulsion models. They are not fixed a priori and have to be optimized for the application at hand. For the simulation tests in Chapter 5, both parameters are set to $\alpha_{\text{aero}} = \alpha_{\text{prop}} = 0.1$, corresponding to a standard deviation of each uncertain force and moment component equal to $10\,\%$ of its magnitude. For real flight tests, different values will be assigned.

The covariance matrix used for the process noise is then obtained as

$$\boldsymbol{\Sigma}_{\zeta} = \text{diag}\left(\boldsymbol{\sigma}_{\zeta}^2\right). \tag{3.10}$$

Similarly, the measurement noise $\boldsymbol{\xi} \in \mathbb{R}^{12}$ is modeled as additive zero-mean Gaussian noise acting on the measurement equation:

$$\boldsymbol{y} = \boldsymbol{h}(\boldsymbol{x}) + \boldsymbol{\xi} \qquad \text{with} \qquad \boldsymbol{\xi} \sim \mathcal{N}\left(\mathbf{0}, \boldsymbol{\Sigma}_{\xi}\right). \tag{3.11}$$

Assuming uncorrelated noise across the individual sensor channels, the measurement noise covariance follows directly from the selected standard deviations,

$$\boldsymbol{\Sigma}_{\xi} = \text{diag}\left(\boldsymbol{\sigma}_{\xi}^2\right), \tag{3.12}$$

where $\boldsymbol{\sigma}_{\xi}$ contains conservative estimates for the standard deviations of the individual measured quantities. The chosen measurement noise standard deviations are summarized in Appendix A.

3.1.2. Including the Wind Model

As stated in Section 2.3, the wind is modeled as the sum of constant wind and turbulence described by the Dryden turbulence model:

$$ {}_E\boldsymbol{v}_W = {}_E\boldsymbol{v}_{W,\text{const}} + {}_E\boldsymbol{v}_{W,\text{turb}}. \tag{3.13} $$

To incorporate the wind model into the airship state-space description, a state-space realization of the wind dynamics is required. The dynamics of the constant wind component are simply given by

$$ {}_E\dot{\boldsymbol{v}}_{\text{const}} = \mathbf{0}. \tag{3.14} $$

Hence, the dynamic part of the wind model is entirely described by the Dryden turbulence model. The turbulent wind component is generated by passing white Gaussian noise through the Dryden shaping filters introduced in Section 2.3.1. Consequently, state-space realizations of the transfer functions in Eqs. (2.43), (2.44), and (2.45) are required. Using the observable canonical form [**Lunze2012Regelungstechnik1**] and introducing the white Gaussian noise $\eta_i \sim \mathcal{N}(0,1)$, the first-order transfer function for the longitudinal wind component results in:

$$ \dot{x}_u = \underbrace{-\frac{V}{L_u}}_{=:\boldsymbol{A}_u} x_u + \underbrace{\sigma_u\sqrt{\frac{2V}{\pi L_u}}}_{=:\boldsymbol{B}_u} \eta_u, \tag{3.15} $$

$$ {}_E u_W = x_u. \tag{3.16} $$

The lateral and vertical shaping filters share the same state-space structure. As both are of second order their state-space representation is of dimension two. For the generic wind component $i \in \{v, w\}$, the realization is given by

$$ \dot{\boldsymbol{x}}_i = \underbrace{\begin{bmatrix} 0 & -\dfrac{V^2}{L_i^2} \\ 1 & -\dfrac{2V}{L_i} \end{bmatrix}}_{=:\boldsymbol{A}_i} \boldsymbol{x}_i + \underbrace{\begin{bmatrix} \sigma_i \dfrac{V}{L_i}\sqrt{\dfrac{V}{\pi L_i}} \\ \sigma_i\sqrt{\dfrac{3V}{\pi L_i}} \end{bmatrix}}_{=:\boldsymbol{B}_i} \eta_i, \tag{3.17} $$

$$ {}_E i_W = \begin{bmatrix} 0 & 1 \end{bmatrix} \boldsymbol{x}_i. \tag{3.18} $$

As defined in Section 2.3.1, the scale lengths L_i depend on the flight altitude $h = -z$. Since the reference wind speed W_{20} cannot be measured directly during operation, a constant value of $W_{20} = 10\,\text{m/s}$ is assumed, allowing the turbulence intensities σ_i to be precomputed. This assumption is justified in Section 5.1.1. Furthermore, since $V = \|{}_B\boldsymbol{v}_B\|_2$, the system matrices $\boldsymbol{A}_i$ and $\boldsymbol{B}_i$ depend on the airship state.

Combining the state-space realization for longitudinal, lateral and vertical wind direction yields the probabilistic state-space description:

$$\dot{\boldsymbol{x}}_W = \underbrace{\begin{bmatrix} \boldsymbol{A}_u & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{A}_v & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{A}_w \end{bmatrix}}_{=:\boldsymbol{A}_W(\boldsymbol{x})} \boldsymbol{x}_W + \underbrace{\begin{bmatrix} \boldsymbol{B}_u & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{B}_v & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{B}_w \end{bmatrix}}_{=:\boldsymbol{B}_W(\boldsymbol{x})} \boldsymbol{\eta}, \tag{3.19}$$

$${}_E\boldsymbol{v}_W = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}}_{=:\boldsymbol{C}_W} \boldsymbol{x}_W, \tag{3.20}$$

where $\boldsymbol{A}_W(\boldsymbol{x}) \in \mathbb{R}^{5\times5}$ and $\boldsymbol{B}_W(\boldsymbol{x}) \in \mathbb{R}^{3\times5}$ depend on the state $\boldsymbol{x}$ and $\boldsymbol{\eta} = [\eta_u,\, \eta_v,\, \eta_w]^T$. The advantage of choosing the observable canonical form becomes apparent here, as the estimated wind velocity can be obtained directly from the Dryden states, without requiring an additional state transformation. The wind model can now be appended to the airship state-space model, yielding the augmented state vector,

$$\tilde{\boldsymbol{x}} = \begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{x}_W \end{bmatrix}. \tag{3.21}$$

Although the complete wind model consists of a constant and a turbulent component, only the Dryden filter states are appended to the system state, as they describe the wind dynamics. Consequently, the augmented model contains only the five Dryden states, representing the internal states of the Dryden shaping filters. These states can be used to describe the wind velocity according to Eq. (3.20). The aerodynamic velocity is therefore given by

$${}_B\boldsymbol{v}_A = {}_B\boldsymbol{v}_B - {}_B\boldsymbol{v}_W = {}_B\boldsymbol{v}_B - \boldsymbol{R}_{BE}\boldsymbol{C}_W\boldsymbol{x}_W, \tag{3.22}$$

which allows all aerodynamic forces and moments to be expressed as functions of the augmented state. Although the Dryden states are nominally zero-mean by construction, their estimates within the Kalman filter are not constrained to this property. Since the constant and turbulent wind components affect the aerodynamic velocity in the same way, the filter cannot distinguish their physical origin from the available measurements. It therefore adjusts the Dryden state estimates to minimize the overall measurement residual. If the process noise covariance associated with the Dryden states is sufficiently large, the resulting Kalman gain allows the estimates to track not only the turbulent fluctuations but also slowly varying or constant offsets. Thus, the constant wind component can effectively be captured by the Dryden state estimates, despite not being explicitly represented by a separate state.

This completes the nonlinear state-space description of the airship, which is now given in the structure:

$$\dot{\tilde{\boldsymbol{x}}} = \begin{bmatrix} \dot{\boldsymbol{x}} \\ \dot{\boldsymbol{x}}_W \end{bmatrix} = \begin{bmatrix} \boldsymbol{f}(\tilde{\boldsymbol{x}}, \boldsymbol{u}) \\ \boldsymbol{A}_W\boldsymbol{x}_W \end{bmatrix} + \begin{bmatrix} \begin{bmatrix} \boldsymbol{0}_{6\times1} \\ \boldsymbol{M}^{-1}\boldsymbol{\zeta} \end{bmatrix} \\ \boldsymbol{B}_W\boldsymbol{\eta} \end{bmatrix}, \tag{3.23}$$

$$\boldsymbol{y} = \boldsymbol{h}(\tilde{\boldsymbol{x}}) + \boldsymbol{\xi}, \tag{3.24}$$

where $\boldsymbol{f}$ denotes the nonlinear airship dynamics from Eq. (3.6).

3.1.3. Airspeed Sensor

The airship is equipped with a one-dimensional pitot tube, which measures the longitudinal component of the relative airspeed in the body-fixed x_B direction. With the state vector augmented by the wind states, this measurement can be expressed as a function of the augmented state, allowing it to be incorporated explicitly into the measurement model:

$$v_{A,x_B} = u - \begin{bmatrix} c_\theta c_\psi & c_\theta s_\psi & -s_\theta \end{bmatrix} \boldsymbol{C}_W \boldsymbol{x}_W, \tag{3.25}$$

where the first term represents the body-fixed longitudinal velocity of the airship and the second term accounts for the x_B component of the body-fixed wind velocity, obtained by projecting the earth-fixed wind velocity onto the body-fixed x_B-axis using the first row of the rotation matrix $\boldsymbol{R}_{BE}$. The resulting measurement is added to the measurement vector $\boldsymbol{y}$.

A limitation of the installed pitot tube sensor is that reliable measurements are only available for body-fixed longitudinal airspeeds above $v_{A,x_B} > 2\,\mathrm{m/s}$. Below this threshold, the measurement is considered unreliable and should not influence the state estimate. This behavior is implemented by introducing a time-varying measurement noise variance,

$$R_{k,\text{pitot}} = \begin{cases} R_{\text{pitot}}, & v_{A,x_B} > 2\,\mathrm{m/s}, \\ \infty, & v_{A,x_B} \leq 2\,\mathrm{m/s}, \end{cases} \tag{3.26}$$

where R_{pitot} denotes the nominal measurement noise variance of the sensor. The predicted relative airspeed is used for the switching condition, since the true airspeed is unknown at the time measurement update is performed.

The effect of this modification can be understood by considering the Kalman gain,

$$\boldsymbol{K}_k = \boldsymbol{P}_k^- \boldsymbol{C}_k^T \left(\boldsymbol{C}_k \boldsymbol{P}_k^- \boldsymbol{C}_k^T + \boldsymbol{R}_k \right)^{-1}, \tag{3.27}$$

where $\boldsymbol{R}_k$ denotes the full measurement noise covariance matrix, of which $R_{k,\text{pitot}}$ forms the diagonal entry corresponding to the pitot tube channel. As $R_{k,\text{pitot}} \to \infty$, the corresponding column of the Kalman gain $\boldsymbol{K}_k$ tends to zero. Consequently, the innovation of the pitot-tube measurement no longer affects the state correction in Eq. (2.64). The *a posteriori* state estimate therefore becomes equal to the *a priori* prediction with respect to this measurement channel. Similarly, in the covariance update in Eq. (2.71), the contribution of the pitot-tube channel vanishes, since the unavailable measurement provides no additional information.

Thus, increasing the measurement uncertainty effectively disables the pitot-tube correction while maintaining a consistent measurement model for all flight conditions. This approach allows the airspeed sensor to be incorporated into the filter without requiring a separate measurement model for different flight regimes. In practice, an infinite variance cannot be represented numerically; therefore, $R_{k,\text{pitot}}$ is replaced by a sufficiently large finite value.

3.1.4. Discretization

The Kalman filter framework requires a discrete-time state transition function and measurement function. Therefore, the continuous-time nonlinear state-space model derived in the previous sections is transformed into the discrete form:

$$\tilde{\boldsymbol{x}}_k = \boldsymbol{f}_d(\tilde{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_{k-1}) + \boldsymbol{q}_{k-1}, \tag{3.28}$$

$$\boldsymbol{y}_k = \boldsymbol{h}_d(\tilde{\boldsymbol{x}}_k) + \boldsymbol{r}_k, \tag{3.29}$$

where the discrete process and measurement noise are assumed to be zero-mean Gaussian stochastic processes:

$$\boldsymbol{q}_{k-1} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{Q}_{k-1}), \qquad \boldsymbol{r}_k \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}_k). \tag{3.30}$$

The process noise covariance combines the uncertainties of the nonlinear airship model and the discretized Dryden wind model,

$$\boldsymbol{Q}_{k-1} = \text{blkdiag}\left(\boldsymbol{Q}_{k-1,\text{model}}, \boldsymbol{Q}_{k-1,\text{wind}}\right). \tag{3.31}$$

Since the augmented system consists of a nonlinear airship model and a linear wind model, both subsystems are discretized separately.

An exact analytical discretization of the nonlinear airship dynamics is generally not available. Therefore, the state transition is approximated using the forward Euler method:

$$\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + T_s \boldsymbol{f}(\tilde{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_{k-1}) + \boldsymbol{q}_{k-1,\text{model}}, \tag{3.32}$$

where $T_s = 0.01\,\text{s}$ denotes the sampling time. The forward Euler method exhibits a global error of order $\mathcal{O}(T_s)$. Although higher-order integration methods, such as the classical fourth-order Runge-Kutta method, provide greater numerical accuracy, they require multiple evaluations of the nonlinear dynamics per sampling step. Given the small sampling time and the presence of model uncertainties represented by the process noise, the additional computational effort is not justified for the present application.

The continuous-time force and moment uncertainty introduced in Section 3.1.1 is transformed into an equivalent discrete-time covariance. Following [**BarShalom2001Discretization**] and approximating $e^{\boldsymbol{A}T_s} \approx \boldsymbol{I}$, the discrete covariance is obtained as

$$\boldsymbol{Q}_{k-1,\text{model}} = \boldsymbol{M}^{-1} \boldsymbol{\Sigma}_\zeta(\tilde{\boldsymbol{x}}_{k-1}) \left(\boldsymbol{M}^{-1}\right)^T. \tag{3.33}$$

For fixed airship states, the Dryden shaping filters constitute a linear state-space model. Consequently, the wind dynamics can be discretized exactly:

$$\boldsymbol{x}_{W,k} = \boldsymbol{A}_{W,k-1} \boldsymbol{x}_{W,k-1} + \boldsymbol{q}_{k-1,\text{wind}}, \tag{3.34}$$

with the discrete state transition matrix [**Astrom1997Discretization**],

$$\boldsymbol{A}_{W,k-1} = e^{\boldsymbol{A}_W T_s}\big|_{\boldsymbol{x}_{k-1}}. \tag{3.35}$$

The corresponding discrete process noise covariance is given by [**BarShalom2001Discretization**]

$$\boldsymbol{Q}_{k-1,\text{wind}} = \int_0^{T_s} e^{\boldsymbol{A}_W \tau} \boldsymbol{B}_W \boldsymbol{\Sigma}_\eta \boldsymbol{B}_W^T e^{\boldsymbol{A}_W^T \tau}\, d\tau \Big|_{\boldsymbol{x}_{k-1}}, \tag{3.36}$$

where $\boldsymbol{\Sigma}_\eta = \boldsymbol{I}_{3\times3}$, since the driving inputs η_i are independent standard Gaussian white noise processes. The integral is evaluated using the van Loan method [**VanLoan1978**]. To mitigate numerical asymmetries arising from finite-precision arithmetic, $\boldsymbol{Q}_{k-1,\text{wind}}$ is subsequently symmetrized as

$$\boldsymbol{Q}_{k-1,\text{wind}} \leftarrow \frac{1}{2}\left(\boldsymbol{Q}_{k-1,\text{wind}} + \boldsymbol{Q}_{k-1,\text{wind}}^T\right). \tag{3.37}$$

Since covariance matrices must be positive semi-definite by definition, an eigenvalue decomposition is performed, and any negative eigenvalues resulting from numerical errors are clipped to zero. Finally, the covariance matrix is reconstructed using the modified eigenvalues.

Finally, the discretized airship dynamics and the discretized Dryden wind model are combined to obtain the complete discrete-time state transition function,

$$\boldsymbol{f}_d(\tilde{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_{k-1}) = \begin{bmatrix} \boldsymbol{x}_{k-1} + T_s\, \boldsymbol{f}(\tilde{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_{k-1}) \\ \boldsymbol{A}_{W,k-1}\, \boldsymbol{x}_{W,k-1} \end{bmatrix}, \tag{3.38}$$

with process noise covariance $\boldsymbol{Q}_{k-1}$ as given above. Since the measurement model introduced in Section 3.1.1 is a static, memoryless function of the state, its discretization is trivial, $\boldsymbol{h}_d(\tilde{\boldsymbol{x}}_k) = \boldsymbol{h}(\tilde{\boldsymbol{x}}_k)$, and no separate treatment is required. As the sensor measurements are obtained at discrete sampling instants, the measurement noise covariance $\boldsymbol{R}_k$ is directly determined from the constant noise characteristics of the individual sensors, which were specified in Section 3.1.1. With the airspeed sensor included, the measurement noise covariance becomes:

$$\boldsymbol{R}_k = \text{blkdiag}\left(\boldsymbol{\Sigma}_\xi, R_{k,\text{pitot}}\right). \tag{3.39}$$

3.1.5. Extended Kalman Filter Implementation

The implementation of the EKF directly follows the prediction and update equations introduced in Section 2.4.1. Rather than reiterating the standard algorithm, this section discusses the implementation choices made in this work. In particular, it addresses the computation of the Jacobian matrices and a numerically robust formulation of the covariance update.

Computation of Jacobians

The EKF requires the Jacobians $\boldsymbol{F}_k$ and $\boldsymbol{H}_k$ of the discrete-time state transition function $\boldsymbol{f}_d$ and measurement function $\boldsymbol{h}_d$ with respect to the augmented state $\tilde{\boldsymbol{x}}$. Both Jacobians are evaluated at each sampling instant using the current state estimate. An analytical derivation of the state transition Jacobian $\boldsymbol{F}_k$ is not practical for the considered system. The nonlinear aerodynamic force and moment models, together with their coupling to the wind states through the relative airspeed, result in lengthy expressions that are impractical to derive symbolically and computationally expensive to evaluate online in the Simulink environment. Instead, $\boldsymbol{F}_k$ is approximated numerically using a forward finite-difference scheme:

$$[\boldsymbol{F}_k]_i \approx \frac{\boldsymbol{f}_d\left(\hat{\tilde{\boldsymbol{x}}}_{k-1} + \varepsilon \boldsymbol{e}_i, \boldsymbol{u}_{k-1}\right) - \boldsymbol{f}_d\left(\hat{\tilde{\boldsymbol{x}}}_{k-1}, \boldsymbol{u}_{k-1}\right)}{\varepsilon}, \tag{3.40}$$

where $\boldsymbol{e}_i$ denotes the i-th unit vector and $[\boldsymbol{F}_k]_i$ the i-th column of $\boldsymbol{F}_k$. A constant perturbation size of $\varepsilon = 10^{-6}$ is used. A forward finite-difference approximation is chosen due to its lower computational cost compared with central differencing while still providing sufficient accuracy for the present application.

In contrast, the measurement Jacobian $\boldsymbol{H}_k$ is computed analytically from the measurement model. The comparatively simple structure of the measurement model allows for a straightforward analytical derivation, making a numerical approximation unnecessary.

Robust Covariance Update

The conventional EKF covariance update given in (2.86),

$$\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k)\, \boldsymbol{P}_k^-, \tag{3.41}$$

can be sensitive to numerical errors. Inaccuracies in the Kalman gain $\boldsymbol{K}_k$, resulting for example from the numerical approximation of the state transition Jacobian, finite-precision arithmetic, or an ill-conditioned innovation covariance matrix, may cause the covariance matrix $\boldsymbol{P}_k$ to lose symmetry or positive semi-definiteness during simulation. If left unaddressed, these numerical effects can accumulate over time and may lead to filter divergence.

To improve numerical robustness, the covariance update is implemented using the Joseph form [**Simon2006KF**]:

$$\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k)\, \boldsymbol{P}_k^- \, (\boldsymbol{I} - \boldsymbol{K}_k \boldsymbol{H}_k)^T + \boldsymbol{K}_k \boldsymbol{R}_k \boldsymbol{K}_k^T, \tag{3.42}$$

which better preserves the symmetry and positive semi-definiteness of the covariance matrix in finite-precision arithmetic. This improved numerical stability comes at the cost of additional matrix multiplications compared with the conventional covariance update.

3.1.6. Unscented Kalman Filter Implementation

The implementation of the UKF directly follows the prediction and update equations introduced in Section 2.4.2. Therefore, this section focuses on the implementation choices specific to the augmented airship-wind model, in particular the selection of the sigma point scaling parameters and the computation of the matrix square root.

Sigma Point Parameters

The unscented transform requires the selection of three scaling parameters: α, β, and κ. The parameter α determines the spread of the sigma points around the estimated mean, β incorporates prior knowledge about the distribution and is optimal at $\beta = 2$ for Gaussian distributions, while κ serves as an additional scaling parameter.

Following the scaled unscented transform formulation in [**Julier2004Unscented**], κ is set to $\kappa = 0$, which represents a common practical choice. The parameter β is set to $\beta = 2$, consistent with the Gaussian state distributions assumed in this work.

The choice of α has a significant influence on the sigma point distribution and therefore on the filter performance. Instead of selecting this parameter manually, its value is determined through a Bayesian optimization procedure [**Snoek2012Bayesian**] using the wind estimation performance as the optimization objective. Based on this optimization, $\alpha = 0.01$ is selected for the UKF implementation.

Matrix Square Root Computation

The unscented transform requires the computation of the matrix square root of the state covariance matrix in order to generate the sigma points. This operation is performed using a Cholesky decomposition, which provides an efficient and numerically stable factorization for a positive definite covariance matrix:

$$\boldsymbol{P}_k = \boldsymbol{L}_k \boldsymbol{L}_k^T. \tag{3.43}$$

However, due to numerical errors introduced during the covariance propagation and update steps, the covariance matrix may no longer be exactly symmetric or positive semi-definite in finite-precision arithmetic. Since the Cholesky decomposition requires a positive definite matrix, these small numerical deviations can cause the decomposition to fail.

To improve numerical robustness, the covariance matrix is therefore regularized prior to the Cholesky decomposition. First, symmetry is enforced by averaging the covariance matrix with its transpose:

$$\boldsymbol{P}_k \leftarrow \frac{1}{2}\left(\boldsymbol{P}_k + \boldsymbol{P}_k^T\right). \tag{3.44}$$

Subsequently, an eigenvalue decomposition is performed, and negative or numerically insignificant eigenvalues are replaced by a small positive tolerance. The covariance matrix is then reconstructed using the modified eigenvalues and symmetrized again before applying the Cholesky decomposition. This procedure ensures that the covariance matrix remains suitable for the sigma point generation while preserving the original covariance structure as closely as possible.

3.1.7. Filter Initialization

Both filters are initialized with a zero state estimate and a small diagonal initial covariance matrix,

$$\hat{\tilde{\boldsymbol{x}}}_0 = \boldsymbol{0}, \qquad \boldsymbol{P}_0 = 10^{-3}\boldsymbol{I}_{n_{\tilde{x}} \times n_{\tilde{x}}}, \tag{3.45}$$

where $\boldsymbol{I}_{n_{\tilde{x}} \times n_{\tilde{x}}}$ denotes the identity matrix with dimension equal to that of the augmented state vector. The zero initial state estimate eliminates the need for prior knowledge of the true initial condition. A small initial covariance is selected to limit large corrections and thus ensure a smooth convergence behavior during the first filter iterations. The influence of the initial estimate and covariance is mainly limited to the transient phase, during which the filter converges from the initial estimate toward the actual state trajectory. Therefore, the specific choice of $\hat{\tilde{\boldsymbol{x}}}_0$ and $\boldsymbol{P}_0$ has no significant influence on the long-term estimation performance.

The EKF and UKF estimate the complete augmented state vector. The wind estimate is subsequently obtained from the estimated wind states through the wind model output equation (3.20):

$$\boxed{{}_E\hat{\boldsymbol{v}}_W = \boldsymbol{C}_W \hat{\boldsymbol{x}}_W.} \tag{3.46}$$

3.2. Compensation

While wind is modeled as an additional system state in the Kalman filter implementation, it can also be considered as an external disturbance acting on the airship. Therefore, the wind estimation provided by the Kalman filter can be utilized to compensate for the effect of wind disturbances. Two compensation

approaches are implemented and evaluated: a nonlinear disturbance feedforward (DFF) controller and a nonlinear dynamic inversion (NDI) controller. Both approaches can be interpreted as an augmentation to the existing PID controller, adding an additional compensation term:

$$\boldsymbol{u} = \boldsymbol{u}_{\text{PID}} + \boldsymbol{u}_{\text{comp}}, \tag{3.47}$$

where $\boldsymbol{u}_{\text{PID}}$ is mainly responsible for setpoint tracking and stabilization, while $\boldsymbol{u}_{\text{comp}}$ uses model knowledge of the airship to counteract the wind-induced disturbances in case of DFF, or compensates for all acting forces and moments in case of the NDI.

To minimize the delay between wind disturbance estimation and compensation, the estimated disturbance is compensated directly at the force and moment level. Based on the estimated wind and the current airship state measurement, the required compensation force and moment vector,

$$\boldsymbol{\mathcal{F}}_{\text{comp}} = \begin{bmatrix} \boldsymbol{F}_{\text{comp}} \\ \boldsymbol{M}_{\text{comp}} \end{bmatrix} \in \mathbb{R}^6 \tag{3.48}$$

is calculated. The computation of $\boldsymbol{\mathcal{F}}_{\text{comp}}$ depends on the compensation approach and is derived in the sections 3.2.1 and 3.2.2.

The total desired force and moment vector is obtained by combining the force and moment vector generated by the PID controller, $\boldsymbol{\mathcal{F}}_{\text{PID}}$, with the compensation vector, $\boldsymbol{\mathcal{F}}_{\text{comp}}$:

$$\boldsymbol{\mathcal{F}}_{\text{des}} = \boldsymbol{\mathcal{F}}_{\text{PID}} + \boldsymbol{\mathcal{F}}_{\text{comp}}. \tag{3.49}$$

This force and moment vector is subsequently converted into individual motor commands by the control allocation algorithm, described in detail in [**Ahmed2022Controller**]. All forces and moments considered in this section are assumed to act at the COV, consistent with the propulsion model introduced in 2.2.3 and the input required by the control allocation algorithm. While $\boldsymbol{\mathcal{F}}_{\text{PID}}$ is acting at the COV by default, $\boldsymbol{\mathcal{F}}_{\text{comp}}$ is transformed accordingly. For improved readability, the corresponding point-of-application identifier $(\cdot)^{\text{COV}}$ is omitted throughout this section.

Performing the compensation directly at the force and moment level enables computationally efficient execution, as the mapping from forces and moments to motor commands $\boldsymbol{u}$ is performed only once per time step. At the same time, the underlying concept introduced in Eq. (3.47) is preserved, allowing disturbances to be compensated with a delay of only one control cycle.

$\boldsymbol{\mathcal{F}}_{\text{des}}$ denotes the desired force and moment to be generated by the propulsion system. However, due to the individual actuator limits of the motors, the achieved force and moment, $\boldsymbol{\mathcal{F}}_{\text{prop}}$, may differ from the desired value.

The resulting control structure, including the compensation strategy, is illustrated in Figure 3.1.

The original PID controller, without compensation, was designed with a strong emphasis on robustness against modeling uncertainties and wind disturbances. Since the compensation approaches explicitly account for a significant portion of wind-induced disturbances, the PID controller gains can be retuned to prioritize tracking performance. In Section 5.3, the effects of the proposed compensation methods on the control performance are evaluated. To investigate the influence of the controller tuning, multiple sets of PID gains are considered in combination with the different compensation approaches.

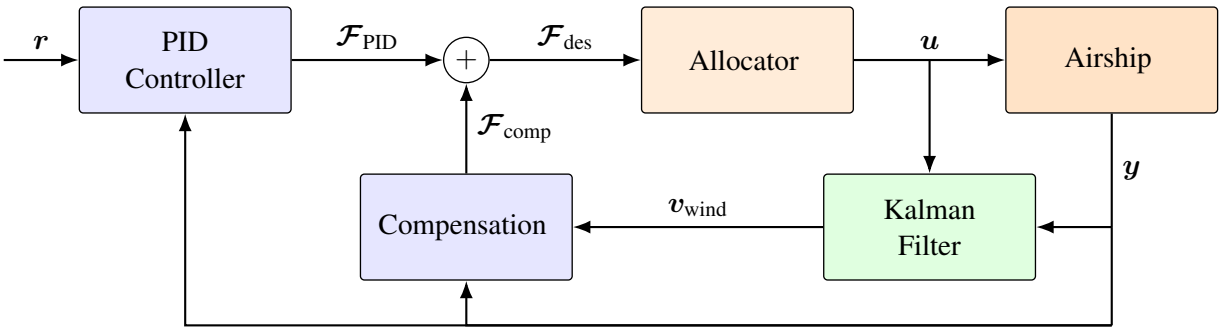


Figure 3.1.: Block diagram of the control structure, combining PID and compensation.

In the following sections, the two compensation approaches are presented in detail, and the corresponding compensation vector $\mathcal{F}_{\text{comp}}$ is derived. It is assumed that the airship operates under nominal flight conditions, such that the desired force and moment generated by the control system can be realized by the propulsion system without actuator saturation. Therefore,

$$\mathcal{F}_{\text{prop}} = \mathcal{F}_{\text{des}} = \mathcal{F}_{\text{PID}} + \mathcal{F}_{\text{comp}}. \tag{3.50}$$

In addition, we assume that the full state is available from the sensor suite of the airship, and that measurement noise is negligible.

3.2.1. Disturbance Feedforward

The disturbance feedforward approach compensates only for the additional forces and moments induced by wind disturbances. Therefore, the compensation vector $\mathcal{F}_{\text{comp}}$ is designed to counteract the change in aerodynamic forces and moments caused by the prevailing wind. To determine the wind-induced contribution, the equations of motion introduced in Section 2.2 are considered:

$$M\dot{\nu} = \mathcal{F}_{\text{grav}}(x) + \mathcal{F}_{\text{buoy}}(x) + \mathcal{F}_{\text{aero}}(x, v_A) + \mathcal{F}_{\text{prop}}(u) - \mathcal{F}_{\text{cor}}(x). \tag{3.51}$$

The wind affects the system dynamics only through the aerodynamic generalized forces, as these depend on the aerodynamic velocity,

$$v_A = v_B - v_W. \tag{3.52}$$

Consequently, the wind-induced forces and moments can be determined as the difference between the generalized aerodynamic forces in the presence of wind and the corresponding forces without wind. In the latter case, the aerodynamic velocity is equal to the body velocity, i.e., $v_A = v_B$:

$$\Delta\mathcal{F}_{\text{wind}} = \mathcal{F}_{\text{aero}}(x, v_A) - \mathcal{F}_{\text{aero}}(x, v_B). \tag{3.53}$$

Substituting Eqs. (3.53) and (3.50) into the equations of motion (3.51) yields:

$$M\dot{\nu} = \mathcal{F}_{\text{grav}}(x) + \mathcal{F}_{\text{buoy}}(x) + \mathcal{F}_{\text{aero}}(x, v_B) - \mathcal{F}_{\text{cor}}(x) + \Delta\mathcal{F}_{\text{wind}} + \mathcal{F}_{\text{PID}} + \mathcal{F}_{\text{comp}}. \tag{3.54}$$

By considering $\Delta\boldsymbol{\mathcal{F}}_{\text{wind}}$ as the disturbance and $\boldsymbol{\mathcal{F}}_{\text{comp}}$ as the compensation input, the system can be interpreted in the nonlinear input-affine form introduced in Eq. (2.107). Since the compensation is directly applied at the generalized force level, the input matrices $\boldsymbol{g}(\boldsymbol{x})$ and $\boldsymbol{g}_d(\boldsymbol{x})$ reduce to identity mappings. The ideal compensation is therefore given by

$$\boldsymbol{\mathcal{F}}_{\text{comp}} = -\Delta\boldsymbol{\mathcal{F}}_{\text{wind}}. \tag{3.55}$$

This results in the compensated dynamics:

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} = \boldsymbol{\mathcal{F}}_{\text{grav}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{buoy}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{aero}}(\boldsymbol{x}, \boldsymbol{v}_B) - \boldsymbol{\mathcal{F}}_{\text{cor}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{PID}}, \tag{3.56}$$

which, assuming perfect knowledge of the model and exact estimation of the wind, are independent of the wind-induced forces and moments.

In practice, exact disturbance compensation cannot be achieved due to two main sources of error. The first results from the fact that only an estimate of the wind velocity, $\hat{\boldsymbol{v}}_W$, is available from the Kalman filter. Consequently, the wind-induced forces and moments must be computed using the estimated aerodynamic velocity,

$$\hat{\boldsymbol{v}}_A = \boldsymbol{v}_B - \hat{\boldsymbol{v}}_W. \tag{3.57}$$

The second source of error originates from the aerodynamic model $\boldsymbol{\mathcal{F}}_{\text{aero}}$ itself, which is an approximation of the true aerodynamic forces and moments and does not take all physical effects into account. These two error sources directly affect the computation of $\Delta\boldsymbol{\mathcal{F}}_{\text{wind}}$. Therefore, the implemented compensation is given by

$$\boxed{\hat{\boldsymbol{\mathcal{F}}}_{\text{comp}} = -\Delta\hat{\boldsymbol{\mathcal{F}}}_{\text{wind}} = -\boldsymbol{\mathcal{F}}_{\text{aero}}(\boldsymbol{x}, \hat{\boldsymbol{v}}_A) + \boldsymbol{\mathcal{F}}_{\text{aero}}(\boldsymbol{x}, \boldsymbol{v}_B),} \tag{3.58}$$

rather than the ideal compensation $-\Delta\boldsymbol{\mathcal{F}}_{\text{wind}}$ derived above.

3.2.2. Nonlinear Dynamic Inversion

In contrast to the disturbance feedforward approach, which compensates only for the wind-induced forces and moments, the nonlinear dynamic inversion (NDI) approach compensates for the complete nonlinear system dynamics. Rather than isolating the disturbance contribution, NDI simultaneously cancels gravitational, buoyant, aerodynamic, and Coriolis forces to approximately transform the nonlinear system into a linear input-output behavior.

Starting again from the equations of motion introduced in Section 2.2:

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} = \boldsymbol{\mathcal{F}}_{\text{grav}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{buoy}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{aero}}(\boldsymbol{x}, \boldsymbol{v}_A) + \boldsymbol{\mathcal{F}}_{\text{prop}}(\boldsymbol{u}) - \boldsymbol{\mathcal{F}}_{\text{cor}}(\boldsymbol{x}), \tag{3.59}$$

and replacing the generalized propulsion force by the sum of the PID and compensation terms yields:

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} = \boldsymbol{\mathcal{F}}_{\text{grav}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{buoy}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{aero}}(\boldsymbol{x}, \boldsymbol{v}_A) - \boldsymbol{\mathcal{F}}_{\text{cor}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{PID}} + \boldsymbol{\mathcal{F}}_{\text{comp}}. \tag{3.60}$$

This has the same nonlinear input-affine structure as Eq. (2.110), with $\boldsymbol{g}(\boldsymbol{x})$ again reducing to an identity mapping. In contrast to the disturbance feedforward approach, the disturbance does not need to be isolated explicitly, but is treated as part of the nonlinear dynamics $\boldsymbol{f}(\boldsymbol{x}, \boldsymbol{d})$. The resulting ideal NDI compensation is therefore given by

$$\boldsymbol{\mathcal{F}}_{\text{comp}} = -\Big(\boldsymbol{\mathcal{F}}_{\text{grav}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{buoy}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{aero}}(\boldsymbol{x}, \boldsymbol{v}_A) - \boldsymbol{\mathcal{F}}_{\text{cor}}(\boldsymbol{x})\Big). \tag{3.61}$$

Substituting Eq. (3.61) into Eq. (3.60) exactly cancels the nonlinear dynamics term, leaving

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} = \boldsymbol{\mathcal{F}}_{\text{PID}}. \tag{3.62}$$

Under the assumption of perfect cancellation, Eq. (3.62) represents an exact feedback linearization of the airship dynamics, resulting in a linear closed-loop system. Consequently, the PID controller no longer needs to compensate for the underlying nonlinear dynamics or wind-induced disturbances. This considerably simplifies the control design: since the remaining system is linear, well-established linear design methods apply, and closed-loop stability and accurate setpoint tracking can be achieved with far less effort than for the original nonlinear system.

As with the disturbance feedforward approach, perfect cancellation cannot be achieved in practice due to wind estimation errors and inaccuracies in the underlying models. First, evaluating $\boldsymbol{\mathcal{F}}_{\text{aero}}(\boldsymbol{x}, \boldsymbol{v}_A)$ requires the aerodynamic velocity $\boldsymbol{v}_A$, whereas only an estimate $\hat{\boldsymbol{v}}_A$ is available obtained according to Eq. (3.57). Second, the models of the gravitational, buoyant, aerodynamic, and Coriolis forces are themselves approximations and do not capture all physical effects acting on the airship.

Combining these two error sources, the compensation actually applied is

$$\boxed{\hat{\boldsymbol{\mathcal{F}}}_{\text{comp}} = -\Big(\boldsymbol{\mathcal{F}}_{\text{grav}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{buoy}}(\boldsymbol{x}) + \boldsymbol{\mathcal{F}}_{\text{aero}}(\boldsymbol{x}, \hat{\boldsymbol{v}}_A) - \boldsymbol{\mathcal{F}}_{\text{cor}}(\boldsymbol{x})\Big),} \tag{3.63}$$

rather than the ideal compensation given in Eq. (3.61).

In contrast to the disturbance feedforward approach, which compensates only the wind-induced aerodynamic contribution, NDI relies on accurate models of the complete nonlinear dynamics. Consequently, NDI is considerably more sensitive to modeling errors and generally requires greater control authority, as the complete nonlinear force vector is cancelled at each time step rather than only the wind-induced component.

4. data-driven wind estimation and compensation

This chapter describes how the multi-layer perceptron (MLP) and the long short-term memory (LSTM) network introduced in Section 2.6 are implemented and trained for the wind estimation task considered in this thesis. It first defines the learning task, then describes the simulated data and its preparation, and finally presents the model architectures and the training procedure.

4.1. Problem Formulation

The task of the networks is to estimate the wind acting on the airship at a given time step t from the vehicle's state and sensor information at or up to this time step.

To achieve this, the networks are provided with the same information available to the Kalman filter from the onboard sensor suite described in Section 1.3.1, with one intentional exception. The position is left out, as the wind does not depend on the location of the airship. The position measurement therefore carries no information about the wind and could only serve to distinguish individual training runs, which promotes overfitting. The input vector is given by

$$\boldsymbol{x}_t = \begin{bmatrix} \boldsymbol{v}_t^T & \boldsymbol{\Theta}_t^T & \boldsymbol{\omega}_t^T & \boldsymbol{a}_t^T & \dot{\boldsymbol{\omega}}_t^T & \boldsymbol{u}_t^T \end{bmatrix}^T \in \mathbb{R}^{24}, \tag{4.1}$$

comprising velocity $\boldsymbol{v}$, attitude $\boldsymbol{\Theta}$, and angular rate $\boldsymbol{\omega}$, the measured linear accelerations $\boldsymbol{a}$ and angular accelerations $\dot{\boldsymbol{\omega}}$, and the motor inputs $\boldsymbol{u} \in \mathbb{R}^9$. The velocity, angular rate, and accelerations are expressed in body-fixed coordinates. This yields an input vector dimension of $n = 24$.

The airspeed measurement of the pitot tube is not included in the input vector for simplicity. The sensor only delivers valid readings above a cutoff airspeed and outputs zero below it, so the measurement is not continuously available. Feeding such an intermittent signal to the networks would require additional handling, e.g., an availability flag, which is beyond the scope of this work.

The network output is the wind velocity vector, $\hat{\boldsymbol{y}}_t = {}_B\hat{\boldsymbol{v}}_{W,t} \in \mathbb{R}^3$. In contrast to the KF, which estimates the wind in earth-fixed coordinates, the networks estimate the wind in body-fixed coordinates. This choice simplifies the learning task, as the wind affects the airship dynamics through the aerodynamic velocity, which is defined in the body frame. Hence, ${}_B\boldsymbol{v}_W$ is the quantity that directly enters the aerodynamic forces and moments. Therefore, by estimating the wind velocity in body-fixed coordinates the networks do not have to learn the coordinate transformation between the body and the earth frame.

With these definitions, both networks realize the mapping (2.114) with $n = 24$ and $m = 3$. The MLP maps the input $\boldsymbol{x}_t$ of a single time step to $\hat{\boldsymbol{y}}_t$, whereas the LSTM additionally processes a sequence of preceding inputs, as described in Section 4.2.3.

4.2. Data Set

Due to the lack of real flight data, the networks are trained and evaluated using simulated data. The data set consists of 486 simulation runs, obtained by combining six flight scenarios (Curve, Forward fast, Forward slow, Hover, Rotate, and Sideways) with 81 wind scenarios each. The flight scenarios cover characteristic operating conditions of the airship, ranging from stationary hovering to translational and rotational maneuvers at different speeds. Each wind scenario combines one of 27 distinct constant wind vectors with one of three turbulence levels, defined by $W_{20} \in \{0, 5, 10\}\mathrm{m/s}$, where $W_{20} = 0\ \mathrm{m/s}$ corresponds to the absence of turbulence.

Each simulation run lasts $100\,\mathrm{s}$, with signals recorded at a sampling frequency of $100\,\mathrm{Hz}$. In total, the data set covers $48\,600\,\mathrm{s}$, or about 13.5 hours, of simulated flight. For each run, the recorded data consist of the input vector $\boldsymbol{x}_t$ defined in (4.1) and the corresponding simulated wind velocity in body-fixed coordinates, which serves as the target vector $\boldsymbol{y}_t$.

4.2.1. Data Splitting

To evaluate the networks' generalization performance, the dataset is divided into separate training, validation, and test sets. The training set is used to update the network parameters, while the validation set is used to monitor training progress. The test set is reserved exclusively for the final evaluation. To prevent data leakage, each simulation run is assigned entirely to a single set.

Two different data-splitting strategies are considered to assess generalization to unseen simulation runs (Experiment A) and unseen flight scenarios (Experiment B).

For Experiment A, the data set is split into 342 training, 72 validation, and 72 test runs. The simulation runs are randomly assigned to the three sets separately for each combination of flight scenario and wind turbulence level. This ensures that all flight scenarios and turbulence levels are represented in each set.

For Experiment B, generalization to unseen flight scenarios is evaluated using six leave-one-scenario-out splits (B1–B6), each holding out one flight scenario entirely. All 81 runs of the held-out scenario form the test set. The runs from the remaining five scenarios are divided into 325 training and 80 validation runs, with the runs distributed as evenly as possible across the three turbulence levels. Since the same wind scenarios are used across all flight scenarios, the test scenario differs from the training scenarios only in the airship's motion, while the wind conditions remain identical.

Table 4.1 provides an overview of the data splits used in the two experiments.

Table 4.1.: Number of simulation runs in each data split. In Experiment B, the held-out flight scenario forms the test set.

Experiment	Training	Validation	Test	Test data
A (unseen runs)	342	72	72	all scenarios
B1–B6 (unseen scenario)	325	80	81	held-out scenario

4.2.2. Preprocessing

The sensor measurements are originally recorded at $100\,\mathrm{Hz}$. Consecutive samples at this rate are highly redundant, and the resulting amount of data increases the computational cost considerably without adding much information. Therefore, all signals are downsampled to $20\,\mathrm{Hz}$ by retaining every fifth sample.

Furthermore, the inputs and targets are standardized to zero mean and unit variance, which improves the conditioning of the optimization problem and can accelerate training [**Lecun2000EffBackProp**]. Each of the 24 components $x_{j,t}$ of the input vector $\boldsymbol{x}_t$ (e.g., a single velocity component or a single motor input) and each of the three components $y_{k,t}$ of the target vector $\boldsymbol{y}_t$ is standardized individually:

$$\tilde{x}_{j,t} = \frac{x_{j,t} - \mu_{x,j}}{\sigma_{x,j}}, \quad j = 1, \ldots, 24, \qquad \tilde{y}_{k,t} = \frac{y_{k,t} - \mu_{y,k}}{\sigma_{y,k}}, \quad k = 1, 2, 3, \tag{4.2}$$

where μ and σ denote the corresponding means and standard deviations. For each of the seven data splits, these statistics are computed using all time steps of all runs in the respective training set. They are then applied unchanged to the corresponding validation and test sets, preventing information from these sets from leaking into the training process.

The standardized input and target vectors are obtained by stacking their respective components:

$$\tilde{\boldsymbol{x}}_t = [\tilde{x}_{1,t}, \ldots, \tilde{x}_{24,t}]^T \qquad \tilde{\boldsymbol{y}}_t = [\tilde{y}_{1,t}, \tilde{y}_{2,t}, \tilde{y}_{3,t}]^T. \tag{4.3}$$

Both networks are trained using these standardized vectors. Standardizing the targets additionally prevents wind components with larger numerical magnitudes from dominating the loss function. For readability, the tilde is omitted in the following, and $\boldsymbol{x}_t$ and $\boldsymbol{y}_t$ denote the standardized quantities unless stated otherwise.

4.2.3. Input Generation

Based on the standardized and downsampled signals, the input-output pairs for the networks are assembled separately for each run. For the MLP, each input consists solely of the input vector $\boldsymbol{x}_t$ at the current time step, without incorporating information from previous time steps.

For the LSTM, the input consists of a sequence of the most recent $T = 40$ input vectors,

$$\boldsymbol{X}_t = \begin{bmatrix} \boldsymbol{x}_{t-T+1} & \ldots & \boldsymbol{x}_{t-1} & \boldsymbol{x}_t \end{bmatrix}^T \in \mathbb{R}^{T \times 24}, \tag{4.4}$$

which corresponds to a time window of 2 s at a sampling frequency of 20 Hz. The sequences are generated using a sliding window with a stride of one time step. Both networks predict the standardized wind $\hat{\boldsymbol{y}}_t$ at the current time step t. Each simulation run is treated as an independent time series, so the windows never cross run boundaries, and as the split is performed on the level of entire runs, no window overlaps two sets. For the LSTM, the first $T - 1$ time steps of each run, for which no complete history is available, are not used as prediction targets. A run with N time steps after downsampling therefore yields $N - T + 1$ sequences, whereas the MLP uses all N time steps of each run.

4.3. Model Architectures

This section describes the architectures of the two networks. Both map the input vectors defined in Section 4.2.3 to an estimate of the body-fixed wind velocity and use a linear output layer, as is common for regression tasks. They differ in how the input is processed: the MLP evaluates the input of a single time step, whereas the LSTM processes a sequence of $T = 40$ input vectors.

The architectures were selected manually rather than through a hyperparameter search. Since the networks are intended for real-time execution on onboard flight hardware, where computational resources and memory are limited, both are designed as moderately sized baseline models. Both achieve a low loss

on the validation data (see Chapter 4.4). However, since the validation data contain flight scenarios also represented in the training data, hyperparameter tuning based on the validation loss would primarily optimize performance on familiar scenarios. Its effect on generalization to unseen scenarios, as investigated in Experiment B, is uncertain. The results should therefore be interpreted as the performance of manually selected baseline architectures rather than as the best performance achievable with these network types.

Multi-Layer Perceptron

The MLP is a fully connected feedforward network as introduced in Section 2.6.1. It maps the input vector $\boldsymbol{x}_t \in \mathbb{R}^{24}$ at a single time step to the wind estimate $\hat{\boldsymbol{y}}_t \in \mathbb{R}^3$ through three hidden layers with 64, 64, and 32 neurons, i.e., $L = 4$ layers in the notation of (2.118). The layer dimensions are therefore

$$24 \to 64 \to 64 \to 32 \to 3. \tag{4.5}$$

All hidden layers use the ReLU activation function $\varphi(z) = \max(0, z)$, applied element-wise. The output layer is linear, i.e., $\varphi^{[4]}(\boldsymbol{z}) = \boldsymbol{z}$, so that the output can take any real value, as required for the regression of the standardized wind components.

Long Short-Term Memory Network

The LSTM network consists of two stacked LSTM layers with 64 hidden units each, followed by a fully connected output layer. Its input is the sequence $\boldsymbol{X}_t$ defined in (4.4). Let $\boldsymbol{h}_\tau^{[l]}$ denote the hidden state of LSTM layer $l \in \{1, 2\}$ after processing the τ-th element of the input sequence, $\tau = 1, \ldots, T$, computed according to (2.119)–(2.124). The first LSTM layer processes the input sequence and returns its hidden state at every time step. These hidden states form the input sequence of the second layer, which returns only its final hidden state at the last time step, $\boldsymbol{h}_T^{[2]}$, to the output layer,

$$\hat{\boldsymbol{y}}_t = \boldsymbol{W}_{\text{out}}\, \boldsymbol{h}_T^{[2]} + \boldsymbol{b}_{\text{out}}, \tag{4.6}$$

where $\boldsymbol{W}_{\text{out}} \in \mathbb{R}^{3\times 64}$ and $\boldsymbol{b}_{\text{out}} \in \mathbb{R}^3$. The final hidden state corresponds to the last input vector of the sequence, $\boldsymbol{x}_t$, and summarizes the entire window. The network therefore realizes a sequence-to-one mapping, in which a window of $T = 40$ input vectors yields a single wind estimate for the current time step. The hidden and cell states are initialized with zeros at the beginning of every sequence, so that all windows are processed independently of each other. As for the MLP, the output layer is linear.

The number of trainable parameters follows from the layer dimensions. A fully connected layer with n_{in} inputs and n_{out} outputs has $n_{\text{out}}(n_{\text{in}} + 1)$ parameters, including biases. An LSTM layer with n_{h} hidden units has $4n_{\text{h}}(n_{\text{in}} + n_{\text{h}} + 1)$ parameters, since each of the three gates and the candidate cell state in (2.119)–(2.122) has a weight matrix $\boldsymbol{W} \in \mathbb{R}^{n_{\text{h}}\times n_{\text{in}}}$, a recurrent weight matrix $\boldsymbol{U} \in \mathbb{R}^{n_{\text{h}}\times n_{\text{h}}}$, and a bias vector $\boldsymbol{b} \in \mathbb{R}^{n_{\text{h}}}$. Table 4.2 summarizes the architectures.

The LSTM has considerably more trainable parameters than the MLP. Additionally, it evaluates $T = 40$ recurrent steps for each prediction, whereas the MLP requires a single forward pass. These differences in model size and computational requirements have to be taken into account when comparing their predictive performance in Chapter 5.

Table 4.2.: Layer structure and number of trainable parameters of the two networks.

Model	Layer	Output size	Parameters
MLP	Dense, ReLU	64	1 600
	Dense, ReLU	64	4 160
	Dense, ReLU	32	2 080
	Dense, linear	3	99
	Total		7 939
LSTM	LSTM (sequence output)	$64 \times T$	22 784
	LSTM (last output)	64	33 024
	Dense, linear	3	195
	Total		56 003

4.4. Training Procedure

Both networks are implemented and trained using the Deep Learning Toolbox of MATLAB [**MathworksDeepLearning**]. They are trained as regression models on the standardized input and target vectors from Section 4.2.2, using the training set of the respective data split. Training is performed separately for each of the seven splits and both network types, which results in 14 trained models.

The networks are trained by minimizing the regression loss between the predicted and true standardized wind vectors. MATLAB uses a half-mean-squared-error loss for the MLP. For the LSTM's sequence-to-one regression task, the loss is one-half of the sum of squared errors across the three output components, without normalization by the number of components. Both loss definitions are proportional to the conventional mean squared error in Eq. (2.116) and therefore have the same minimizer. The gradients are computed by backpropagation and, for the LSTM, backpropagation through time over the $T = 40$ time steps of each sequence.

The parameters are updated with the Adam optimizer [**Kingma2014Adam**] using the default decay rates of the implementation. The learning rate starts at $\eta_0 = 10^{-3}$ and is halved every 20 epochs. The training data is divided into mini-batches and shuffled before every epoch. The mini-batch size is 1024 for the MLP and 128 for the LSTM to account for the greater computational and memory requirements of processing sequences of 40 time steps. Besides the L2 regularization of the implementation with its default factor of 10^{-4}, no further regularization is used. The weights are initialized with the default initializers of the implementation, i.e., the Glorot initialization for the input weights, orthogonal initialization for the recurrent weights of the LSTM, and zeros for the biases, except for the forget gate bias, which is initialized with one.

The loss on the validation set is evaluated every 50 iterations. Training stops after a maximum of 100 epochs, or earlier if the validation loss has not fallen below its smallest value so far in 30 consecutive evaluations. The network of the final iteration is used for the evaluation. Table 4.3 summarizes the settings. Figure 4.1 shows the training and validation loss for the MLP as an example.

The networks are trained using standardized inputs and targets. Consequently, their outputs are also standardized and must be transformed back into physical units by inverting the standardization defined in (4.2) to obtain the wind estimates. The performance of the trained networks is evaluated in Chapter 5.

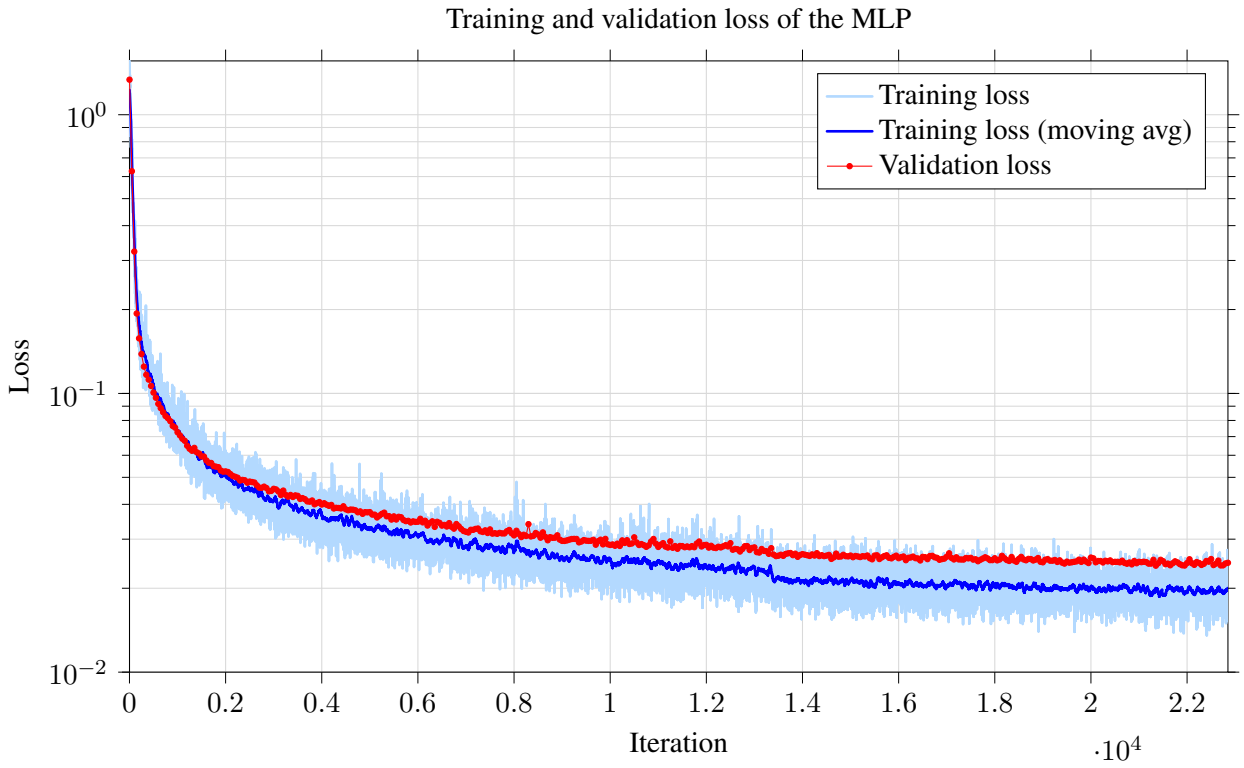


Figure 4.1.: Training and validation loss of the MLP on the Experiment A training set. The moving average training loss is computed over the last 50 samples for improved readability.

Table 4.3.: Training settings of the two networks.

	MLP	LSTM
Loss	Half mean squared error	Half sum of squared errors
Optimizer	Adam	
Initial learning rate	10^{-3}	
Learning rate schedule	Halved every 20 epochs	
L2 regularization	10^{-4}	
Maximum number of epochs	100	
Validation frequency	50 iterations	
Early stopping patience	30 validations	
Mini-batch size	1024	128

5. Evaluation and results

To evaluate the performance of the proposed wind estimation algorithms and the wind compensation strategies, a simulation-based test framework is established. For the wind estimation task, the model-based extended Kalman filter (EKF) and unscented Kalman filter (UKF) are compared with the data-driven neural network approaches, i.e., the multi-layer perceptron (MLP) and the long short-term memory (LSTM) network, under identical conditions. For wind compensation, disturbance feedforward (DFF) and nonlinear dynamic inversion (NDI) are evaluated with respect to their impact on tracking performance and compared to the baseline PID controller.

To ensure a representative evaluation under different operating conditions, six flight scenarios are considered. They cover translational motion, rotational motion, and their combination, as well as station keeping, and thus represent the main operating conditions of the airship:

- Slow forward flight ($2\,\mathrm{m/s}$)
- Fast forward flight ($4\,\mathrm{m/s}$)
- Sideward flight ($1\,\mathrm{m/s}$)
- Pure yaw rotation ($5\,^\circ/\mathrm{s}$)
- Curved flight with radius $15.9\,\mathrm{m}$: forward flight ($1\,\mathrm{m/s}$) combined with a yaw rotation ($3.6\,^\circ/\mathrm{s}$)
- Hover flight

Since the performance of the wind estimation and compensation methods depends strongly on the wind conditions, multiple wind scenarios are considered for each flight scenario. These include different constant wind velocities and varying turbulence intensities generated by varying the Dryden reference wind speed parameter W_{20}, as introduced in Section 2.3.

The root mean square error (RMSE) is used as the primary performance metric throughout this evaluation. It provides a quantitative measure of the deviation between a reference signal and its corresponding estimate or response. For a signal x with N samples, the RMSE is defined as [**Chai2014RMSE**]

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}(x_k - \hat{x}_k)^2}, \tag{5.1}$$

where x_k denotes the reference value and $\hat{x}_k$ the corresponding estimated or simulated value at sample k.

In the following evaluation, the RMSE is applied to assess both the wind estimation accuracy and the resulting state tracking performance. The wind RMSE quantifies the deviation between the estimated and actual wind components, while the state RMSE is used to evaluate the tracking error of the airship states.

The remainder of this chapter is structured as follows. In Section 5.1, the performance of the model-based wind estimation algorithms (EKF and UKF) is evaluated for the considered flight and wind scenarios. Section 5.2 then investigates the data-driven wind estimation using the MLP and LSTM networks. Subsequently, Section 5.3 investigates the effectiveness of the proposed wind compensation strategies and their impact on the overall control performance.

5.1. Model based Wind Estimation

The performance of the model-based wind estimation algorithms is evaluated first. As described in Section 2.3.1, the turbulence intensities σ_i of the Dryden model are determined by the reference wind speed W_{20}. Consequently, a suitable value for W_{20} must first be selected to fully define the wind model used by the Kalman filter. Section 5.1.1 investigates the influence of W_{20} and determines an appropriate value, which is subsequently used in Section 5.1.2 to compare the wind estimation performance of the EKF and UKF. Based on these results, the computational cost of both filters is analyzed, followed by an assessment of the influence of the airspeed sensor on the estimation accuracy. Finally, the computation of the mean wind from the estimated wind components is examined.

5.1.1. Selection of Reference Wind Speed

The reference wind speed W_{20} cannot be directly obtained from the system states, and its true value in the operating environment is unknown. In this work, it is represented by a constant parameter of the wind model, which must be specified beforehand. A suitable value of W_{20} should provide robust estimation performance across the expected range of wind conditions.

To determine an appropriate value for the reference wind speed W_{20}, the influence on the wind estimation performance is investigated. First, a constant reference wind speed $W_{20,\text{true}} \in \{0, 1.6, 5, 8.3\}\text{m/s}$ is specified for the Dryden model to generate the actual wind conditions in the simulation. The Kalman filter is then configured with varying assumed values $W_{20,\text{ass}}$ ranging from 0 to $15\,\text{m/s}$. The investigated range covers the expected wind conditions of up to approximately $30\,\text{km/h}$ ($\approx 8.3\,\text{m/s}$) and extends beyond this value to assess the impact of overestimating the turbulence intensity on the wind estimation performance.

For each combination of true value, $W_{20,\text{true}}$ and assumed value, $W_{20,\text{ass}}$, the same set of 27 wind scenarios is simulated to evaluate the estimation performance under different wind conditions. Each scenario is defined by a unique combination of constant wind component and Dryden turbulence model random seed, resulting in 27 distinct wind conditions. For each test case, the RMSE between the estimated and actual wind velocity is calculated individually for each axis. The resulting RMSE values are then averaged over all three axes and subsequently over all 27 wind scenarios to obtain a representative performance measure for each assumed value of $W_{20,\text{ass}}$.

For this analysis, the airspeed sensor is deliberately not considered, as its measurement would provide additional information that directly constrains the internal Dryden states. Consequently, the resulting wind estimate would become less sensitive to the assumed turbulence intensity, making it more difficult to quantify the influence of the assumed W_{20} value.

Figure 5.1 compares the averaged RMSE for different combinations of assumed and true reference wind speeds, $W_{20,\text{ass}}$ and $W_{20,\text{true}}$, respectively. Two observations are particularly relevant for the selection of $W_{20,\text{ass}}$.

First, even for wind conditions without turbulence ($W_{20,\text{true}} = 0$), assuming $W_{20,\text{ass}} = 0$ results in the largest RMSE among all tested values. This behavior follows from the resulting process noise covariance of the wind model, $\boldsymbol{Q}_{\text{wind}}$, derived in Section 3.1.4. For $W_{20,\text{ass}} = 0$, the input matrix of the Dryden model, $\boldsymbol{B}_W$, becomes the zero matrix, resulting in $\boldsymbol{Q}_{\text{wind}} = \boldsymbol{0}$. Consequently, no process uncertainty is assigned to the wind states, and their evolution is fully determined by the deterministic state matrix $\boldsymbol{A}_W$. Since the Dryden state matrix is stable, the wind states, and hence the estimated wind velocity, converge to zero regardless of the initial state. With the zero initial condition used in this work, the estimated wind velocity remains zero for the entire time. As the constant wind component is generally non-zero, this results in a large estimation error even though no turbulence is present.

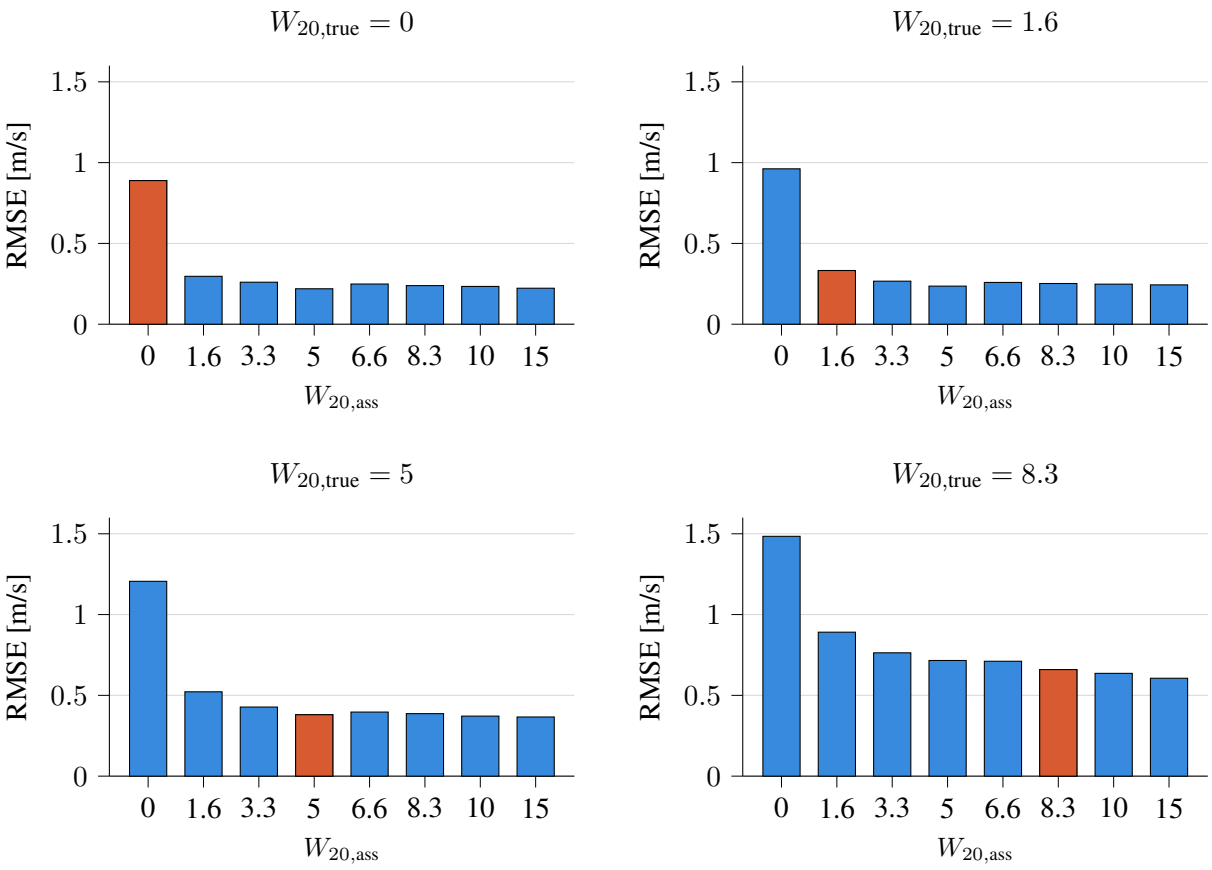


Figure 5.1.: Comparison of RMSE, averaged over three axes and 27 wind scenarios for varying assumed reference wind speeeds $W_{20,\text{ass}}$. The orange bar highlights the true configuration scenario where $W_{20,\text{ass}} = W_{20,\text{true}}$.

Second, matching the assumed and true reference wind speeds ($W_{20,\text{ass}} = W_{20,\text{true}}$) does generally not yield the lowest RMSE. Increasing $W_{20,\text{ass}}$ increases the process noise covariance $\boldsymbol{Q}_{\text{wind}}$, thereby allowing the estimated wind states to adapt more readily to rapid changes in the actual wind. Consequently, an assumed value that exceeds $W_{20,\text{true}}$ can provide better estimation performance than a perfectly matched value by allowing the filter greater flexibility to track the wind dynamics.

Both observations can be illustrated using a concrete example, shown in Figure 5.2.

This raises the question of why $W_{20,\text{ass}}$ should not simply be chosen as large as possible. Increasing $W_{20,\text{ass}}$ increases the process noise covariance $\boldsymbol{Q}_{\text{wind}}$, giving the wind states greater freedom to change in response to deviations between the measured and predicted system behavior. While this can improve the tracking of rapid wind changes, a large $W_{20,\text{ass}}$ can make the estimate overly sensitive to such deviations. As a result, the estimated wind becomes increasingly noisy and may exhibit peaks and oscillations that are not present in the actual wind. Such behavior is undesirable for the subsequent compensation strategy, as strong fluctuations in the wind estimate can directly translate into unnecessary or excessive compensation commands.

Across all tested true wind conditions, $W_{20,\text{ass}} = 10\,\text{m/s}$ consistently provides good estimation performance, with the resulting RMSE being close to or matching the best value within each panel. Although larger values of $W_{20,\text{ass}}$ can yield lower RMSE in individual scenarios, $W_{20,\text{ass}} = 10\,\text{m/s}$ provides a suit-

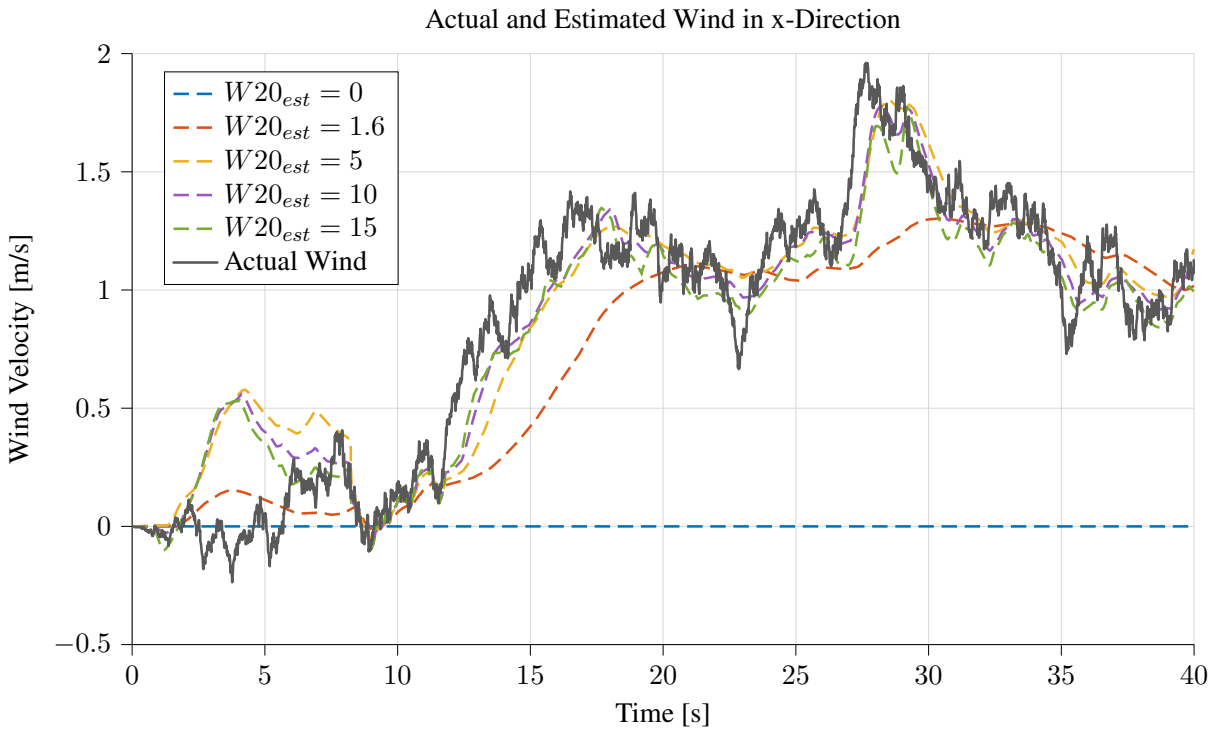


Figure 5.2.: Comparison of wind estimation performance for different assumed reference wind speeds $W_{20,\text{ass}}$ and $W_{20,\text{true}} = 5$.

able trade-off between responsiveness and estimation noise without introducing the potential effects associated with excessive process noise. It is therefore selected as the reference wind speed for the remainder of this work.

The consistent performance across the investigated wind conditions further indicates that a fixed value of W_{20} is sufficient to achieve robust estimation performance in practice. Consequently, no online adaptation of W_{20} based on the current flight or wind conditions is required.

5.1.2. UKF vs EKF

Both the extended Kalman filter (EKF) and the unscented Kalman filter (UKF) provide nonlinear extensions of the Kalman filter and are therefore suitable for the nonlinear augmented airship model considered in this work. The EKF accounts for the nonlinearities by locally linearizing the system and measurement models using their Jacobians. In contrast, the UKF propagates a set of deterministically selected sigma points through the nonlinear models, avoiding the explicit linearization required by the EKF.

To compare the wind estimation performance of the EKF and UKF, both filters are evaluated using the same test suite. The filters are configured with identical initial conditions as well as identical process and measurement noise covariances. For each filter, test runs covering the six flight scenarios described at the beginning of this section are carried out. For each flight scenario, the same set of 27 constant wind contributions and Dryden turbulence model random seeds is applied, while the true turbulence intensity is varied through $W_{20,\text{true}} \in \{0, 5, 10\}\text{m/s}$. This results in a total of $N = 6 \cdot 27 \cdot 3 = 486$ simulation runs for each filter. Using identical filter configurations, flight scenarios, and wind conditions ensures that any observed differences in estimation performance can be attributed to the respective filtering approach rather than to variations in the test conditions.

Figure 5.3 shows the actual wind together with the EKF and UKF estimates for a single representative test scenario.



Figure 5.3.: Actual and estimated wind for Curve flight scenario, constant wind component $\boldsymbol{v}_{W,\text{const}} = [2.5 - 1 - 0.5]\text{m/s}$ and reference wind speed $W_{20,\text{true}} = 10\,\text{m/s}$

Both filters exhibit a similar overall estimation behavior in this example. Following the initial convergence, the estimated wind components closely track the actual wind, with no significant difference between the two filtering approaches. Although the representative time histories suggest similar performance, a quantitative comparison across the complete test suite is required to determine whether this similarity holds across the full range of investigated flight and wind conditions.

For this purpose, the average RMSE along the three wind components is calculated individually for each test run for both the EKF and UKF. Since the initial estimation error is dominated by the mismatch between the initial state estimate and the actual state rather than by the steady-state tracking performance of the filter, the initial transient phase is excluded from the analysis. Specifically, the first 20s of each simulation are disregarded, and the RMSE is calculated only over the remaining part of the estimated wind trajectory. This provides a more representative measure of the wind estimation performance.

The resulting RMSE values are therefore obtained separately for each of the 486 simulation runs. The distributions of these per-run RMSE values for the EKF and UKF are shown in Fig. 5.4. This allows the estimation performance of both filters to be compared not only in terms of their average error, but also with respect to the spread and occurrence of larger estimation errors across the complete test suite.

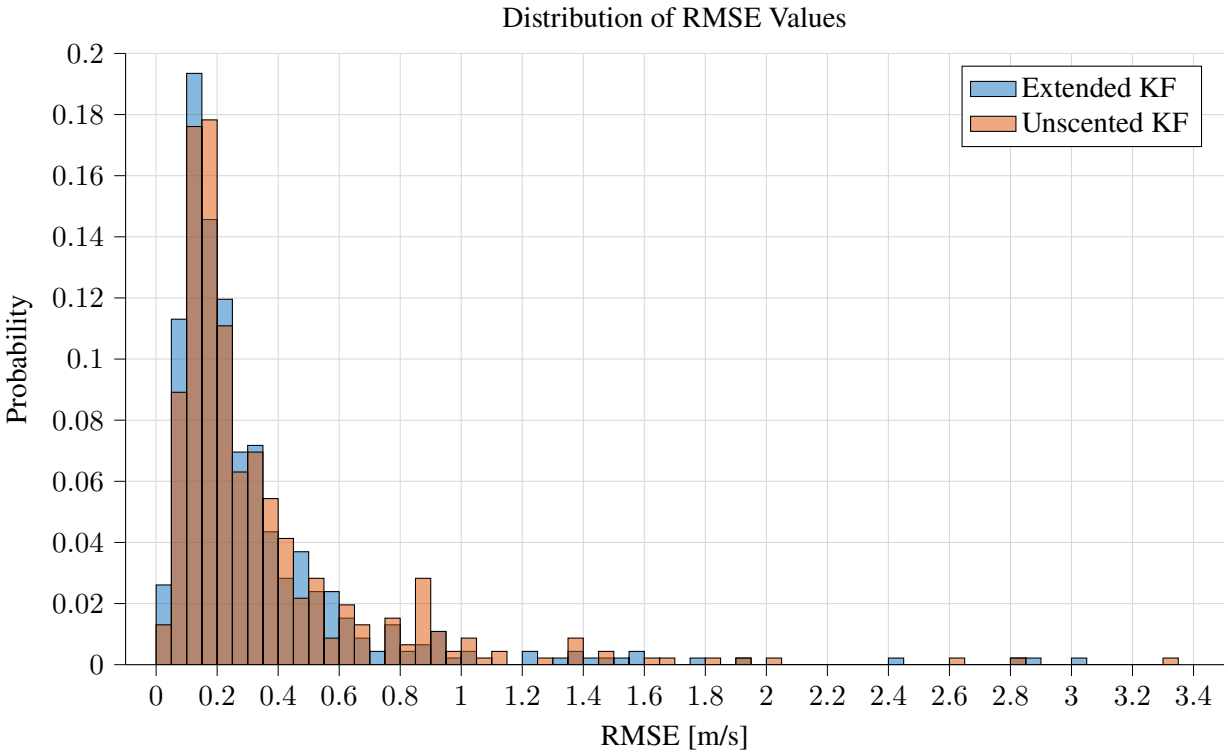


Figure 5.4.: Distribution of the per-run wind estimation RMSE for the EKF and UKF across the investigated test suite.

The results indicate that the EKF and UKF achieve comparably good overall wind estimation performance. The distributions exhibit a similar range and concentration of RMSE values, with only relatively small differences between the two filters. This suggests that the choice between the EKF and UKF has no substantial influence on the wind estimation accuracy under the investigated operating conditions.

Since no significant difference between the two filtering approaches is observed, the following analysis focuses on the factors that instead determine the general wind estimation performance. In particular, the influence of the turbulence intensity and the flight scenario is investigated.

The intensity of the turbulent wind represents an important factor influencing the estimation problem. In the Dryden model used in this work, the turbulence intensity increases with the true reference wind speed $W_{20,\text{true}}$. Consequently, higher values of $W_{20,\text{true}}$ result in stronger fluctuations in the actual wind and therefore in more demanding estimation conditions. Figure 5.5 shows the average RMSE for both filters for different values of $W_{20,\text{true}}$. For each reference wind speed the RMSE is averaged over all six flight scenarios and 27 wind conditions.

For both filtering approaches, the estimation error generally increases as the turbulence intensity is increased. This behavior is expected, as stronger turbulence results in larger and more rapid variations of the actual wind, making it more difficult for the filters to track the changing wind conditions. Nevertheless, the EKF and UKF exhibit a similar response to increasing turbulence intensity. No systematic improvement of one filtering approach over the other can be observed across the investigated range of $W_{20,\text{true}}$.

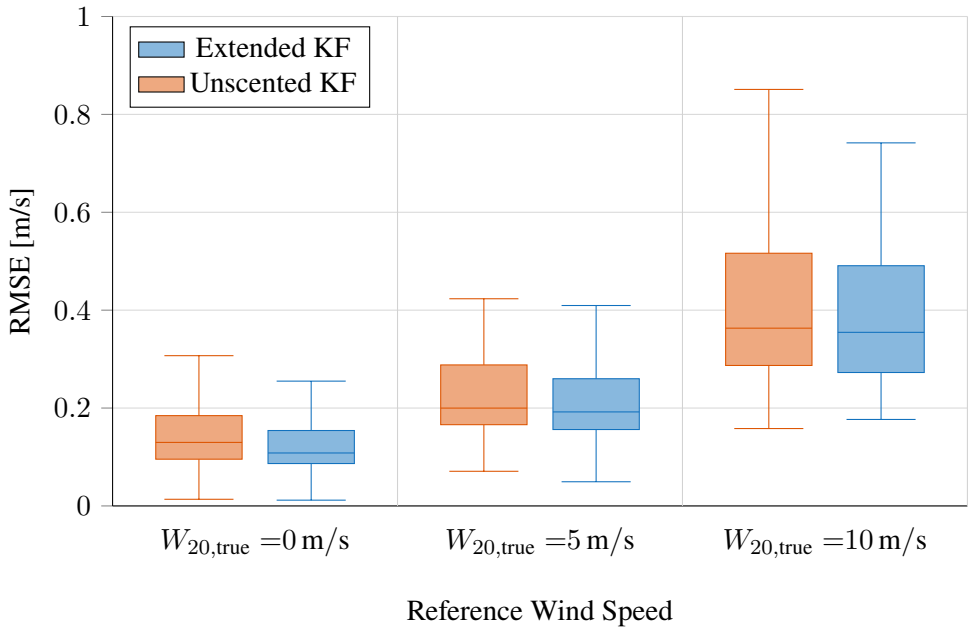


Figure 5.5.: Average wind estimation RMSE for different values of $W_{20,\text{true}}$, averaged over the six flight scenarios and 27 wind conditions. Boxes show the interquartile range (Q1–Q3) with the median line; whiskers extend to the most extreme point within 1.5×IQR. Outlier points beyond the whiskers are omitted.

The influence of the flight scenario on wind estimation performance is investigated next, keeping the filter configuration fixed and varying only the aircraft motion. The resulting RMSE values, averaged across all 27 wind conditions and three reference wind speed values, are shown in Fig. 5.6.

Whereas the relative performance of the EKF and UKF stays similar across all flight scenarios, the results show a clear dependence of the wind estimation performance on the flight scenario. In particular, the hover scenario exhibits a noticeably larger estimation error than the other investigated cases. This result is unexpected, since the pure-rotation scenario — in which the vehicle likewise remains at a fixed position and undergoes no translational motion — achieves considerably better performance. To understand this behavior, the influence of the flight dynamics and the resulting information available for wind-state estimation is examined in more detail.

Analysis of Wind Estimation Performance in Hover

To analyze the coupling between the wind velocity and the airship dynamics, the linearized dynamics considered by the EKF (Section 2.4.1) are utilized. Since the wind velocity influences the airship dynamics only through the aerodynamic generalized forces, its influence is limited to the translational and rotational dynamics described by the original 6-DOF equations derived in Section 2.2. This coupling is captured by the corresponding Jacobian block

$$\boldsymbol{F}_{v_W} = \frac{\partial \boldsymbol{f}_{\text{6DOF}}}{\partial \boldsymbol{v}_W}, \tag{5.2}$$

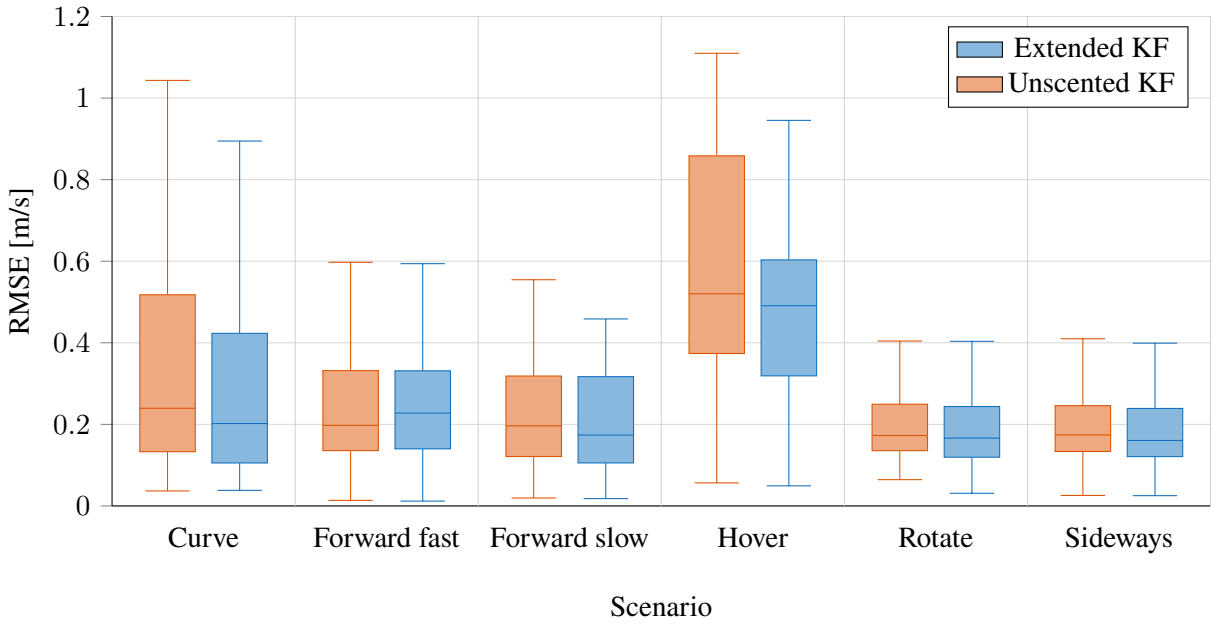


Figure 5.6.: Average wind estimation RMSE by scenario for Unscented KF and Extended KF. Boxes show the interquartile range (Q1–Q3) with the median line; whiskers extend to the most extreme point within 1.5×IQR. Outlier points beyond the whiskers are omitted.

which describes how perturbations of the wind velocity affect the six-degree-of-freedom airship dynamics. The singular values of this matrix provide a measure of the strength of this coupling. As shown in Fig. 5.7 the singular values are considerably smaller in hover than in forward flight or during rotation. Thus, in the absence of translational or rotational motion, perturbations of the wind velocities have only a weak influence on the vehicle dynamics.

This reduced coupling directly affects the ability of the filter to correct the wind estimate. An error in the estimated wind velocity first affects the predicted airship states through the system dynamics. If the coupling represented by $\boldsymbol{F}_{v_W}$ is weak, the resulting change in the predicted airship states is small. Since the predicted measurements are obtained from these predicted states, the influence of the wind-velocity error on the predicted measurements is likewise small. Consequently, the innovation is only weakly influenced by the wind-velocity error and therefore contains limited information about the wind-state error.

As the innovation determines the correction applied during the measurement update according to Eq. (2.82), the limited sensitivity of the innovation to the wind-velocity error also limits the extent to which the measurement update can correct the wind estimate. This can lead to slower convergence of the estimated wind and a larger estimation error over the finite simulation duration. In contrast, during Forward Flight or Rotation, perturbations of the wind states produce larger changes in the airship dynamics and, consequently, in the predicted measurements. The resulting innovations are therefore more sensitive to wind-state errors, allowing the filter to correct the wind estimate more effectively.

An additional effect further degrading the wind estimation performance during Hover results from the characteristics of the Dryden turbulence model. In this model, the intensity of the generated wind turbulence scales with the magnitude of the body-fixed velocity $V = \|\boldsymbol{v}_B\|_2$. During Hover, $V \approx 0$, and the corresponding process noise of the wind dynamics vanishes, $\boldsymbol{Q}_{\text{wind}} \approx \boldsymbol{0}$. As a result, the wind in the simulation reduces to a near-constant disturbance rather than a time-varying turbulent process.

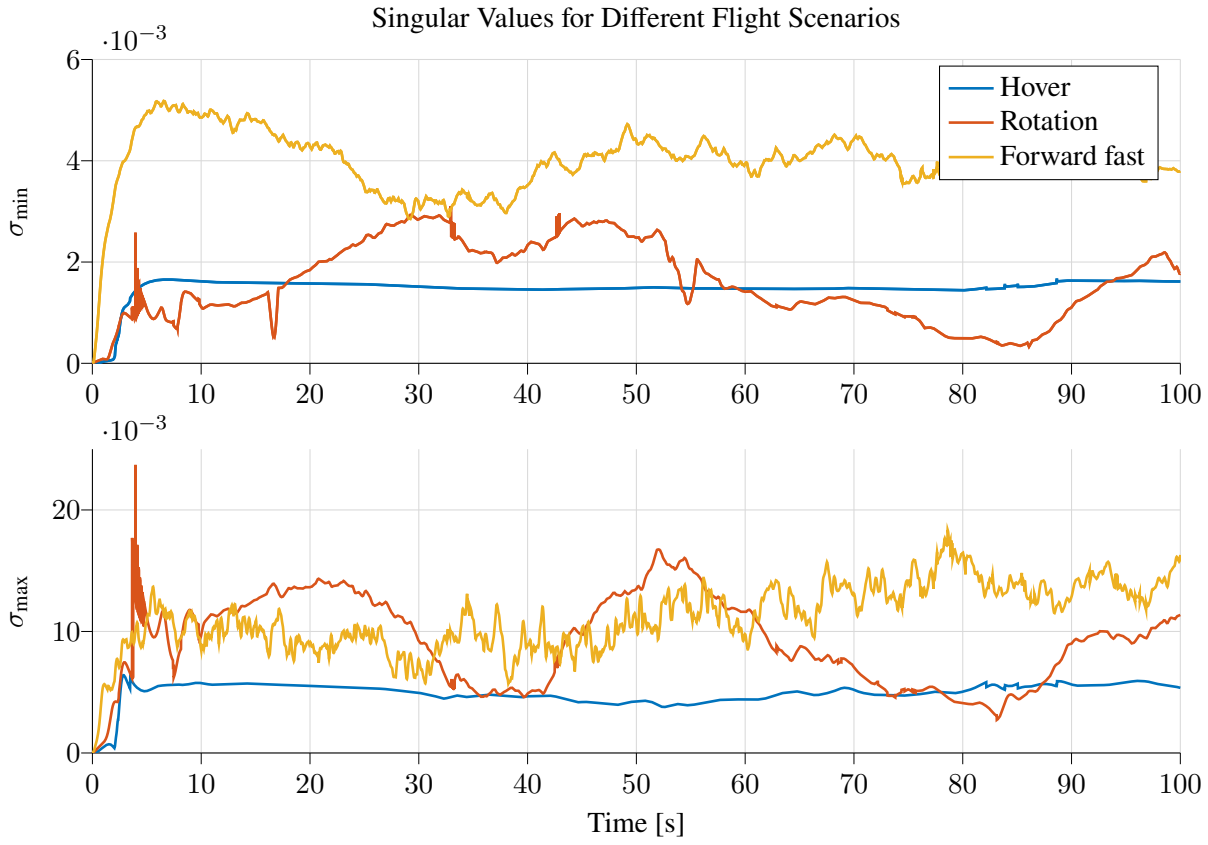


Figure 5.7.: Minimum (top) and maximum (bottom) singular values of $\boldsymbol{F}_{v_W}$ over time for Hover, Forward fast, and Rotation, with constant wind component $\boldsymbol{v}_{W,\text{const}} = [-2\ 0\ -0.5]\text{m/s}$ and reference wind speed $W_{20,\text{true}} = 5\,\text{m/s}$.

This has two consequences for the estimator. First, $\boldsymbol{Q}_{\text{wind}}$ enters the covariance prediction of the EKF in an additive way according to Eq. (2.80). A vanishing process noise therefore results in a smaller *a priori* covariance of the wind states. Since the Kalman gain increases with the *a priori* covariance, a smaller predicted covariance generally results in a smaller Kalman gain and, consequently, in slower adaption of the wind estimate [**Maybeck1979KF**].

Second, the near-constant true wind results in a lack of time-varying excitation of the airship dynamics. If turbulent fluctuations are present, they lead to corresponding variations in the vehicle motion and the measured states. These time-varying responses provide additional information that is correlated with the wind states and can therefore help the filter distinguish wind-induced changes from changes associated with other state uncertainties. In Hover, where the Dryden turbulence contribution vanishes, this additional source of excitation and information is largely absent. Figure 5.8 visualizes the effect on the wind estimation performance during hover.

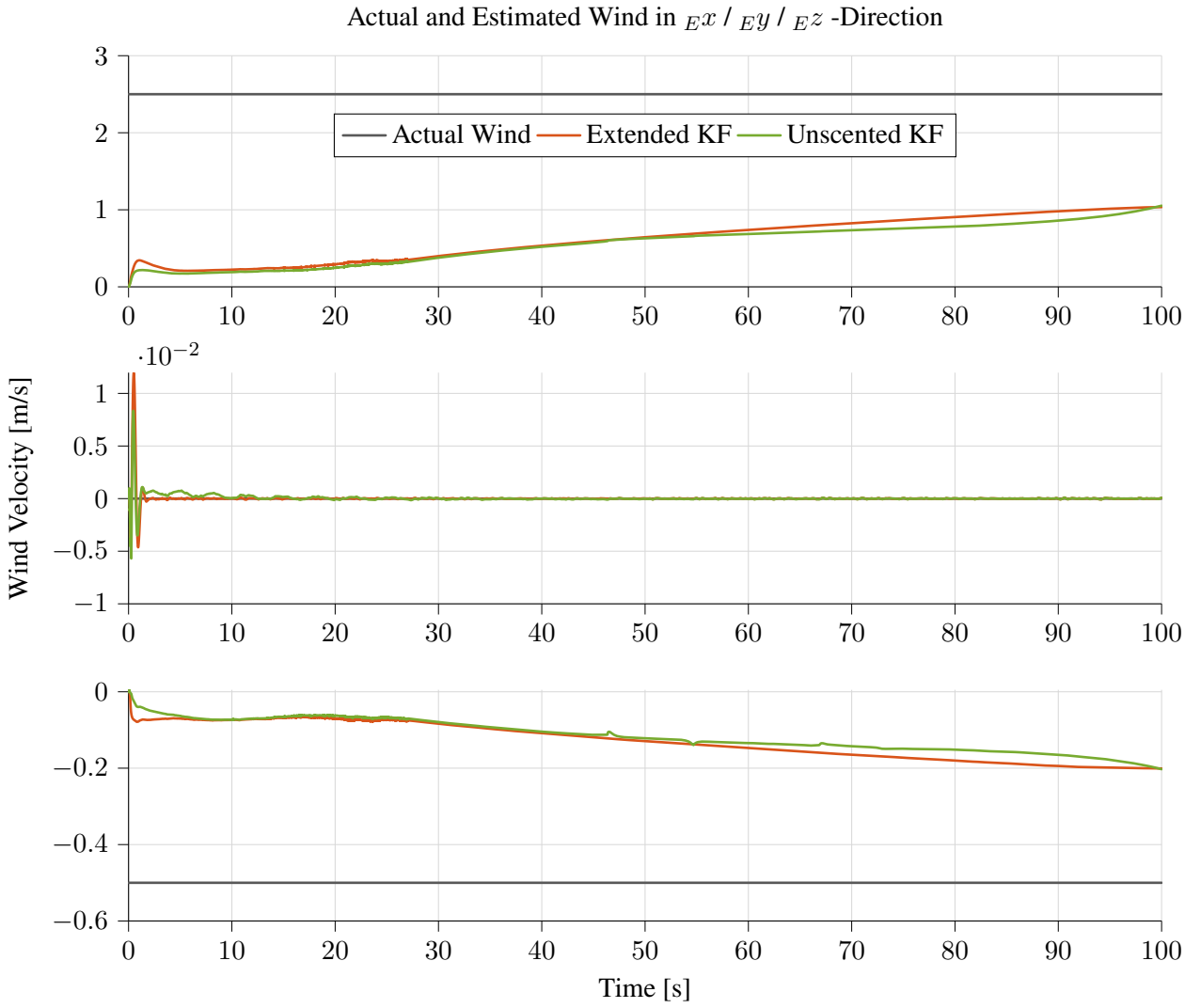


Figure 5.8.: Actual and estimated wind for hover flight scenario, constant wind component $\boldsymbol{v}_{W,\text{const}} = [2.5\ 0\ -0.5]$ m/s and $W_{20,\text{true}} = 0$ m/s. Process noise is disabled for visualization purposes to highlight the convergence behavior of the wind estimate.

Overall, the increased wind-estimation error during Hover is likely the result of a combination of all three effects discussed above. The individual contribution of each effect cannot be isolated based on the present analysis. In a real environment, however, $V \approx 0$ does not necessarily imply the absence of atmospheric turbulence. Therefore, this particular characteristic of the Dryden model is expected to be less pronounced in real-world conditions, and the wind-estimation performance during Hover relative to the other flight modes may be better.

5.1.3. Computational Cost

As the choice between the EKF and UKF has no substantial influence on the wind-estimation accuracy under the investigated operating conditions, the choice between the two approaches can instead be guided by other considerations, in particular their computational effort.

To compare the computational cost of both approaches, the EKF and UKF are evaluated separately using isolated filter implementations with identical input data and simulation conditions. An initial simulation is performed before the runtime measurements to reduce the influence of initialization effects. The subsequent 100 simulations are used to determine the mean computational runtime per simulation step, with the results given in Table 5.1.

Table 5.1.: Mean computational runtime per simulation step for the EKF and UKF. The results are obtained from 100 simulations with a simulated duration of $1000\,\mathrm{s}$ and a simulation step size of $0.01\,\mathrm{s}$. The normalized runtime is given relative to the EKF.

Filter	Mean runtime per step [ms]	Normalized runtime [-]
EKF	0.153	1.00
UKF	0.296	1.93

The UKF requires approximately $93\,\%$ more computational time per simulation step than the EKF. However, the absolute runtimes measured in this test are not representative of the actual computational time on the airship's flight-control hardware. Therefore, the results should be interpreted as a relative comparison of the computational cost of the EKF and UKF, rather than as an indication of their real-time computational requirements on the deployed system.

The difference in computational cost primarily results from the different treatment of the nonlinear system and measurement models. In the EKF, the state-transition Jacobian $\boldsymbol{F}$ is approximated numerically using forward differences. For the 17-dimensional state vector, this requires $n = 17$ additional evaluations of the nonlinear state-transition function per step. The measurement Jacobian $\boldsymbol{H}$ is obtained analytically and only needs to be evaluated once per simulation step.

The UKF, in contrast, propagates the nonlinearities through $2n + 1 = 35$ sigma points. Generating these sigma points requires a matrix square-root operation, after which all $2n + 1$ sigma points are propagated through the nonlinear state-transition function $\boldsymbol{f}$. For the measurement update, a new set of sigma points is generated from the predicted state and propagated through the nonlinear measurement function $\boldsymbol{h}$.

Consequently, the UKF requires more evaluations of the nonlinear models and additional matrix operations compared with the EKF. This additional computational effort is reflected in the measured runtime and explains the higher computational cost of the UKF in the present implementation. Since the wind estimation errors of both filters are comparable, the UKF offers no accuracy advantage that would justify its higher cost. Therefore, the EKF is chosen as the representative of the Kalman filters for the following comparison with the neural networks in Section 5.2.

5.1.4. Effect of Airspeed Sensor

The 1D airspeed measurement provides an additional source of information for the wind-state estimation. It is therefore of interest to assess to what extent its availability affects the estimation performance. For this purpose, the wind estimation is compared with and without the airspeed measurement from the pitot tube. Figure 5.9 compares the estimation performance for a representative test scenario to illustrate the effect of the airspeed sensor on the estimation behavior.

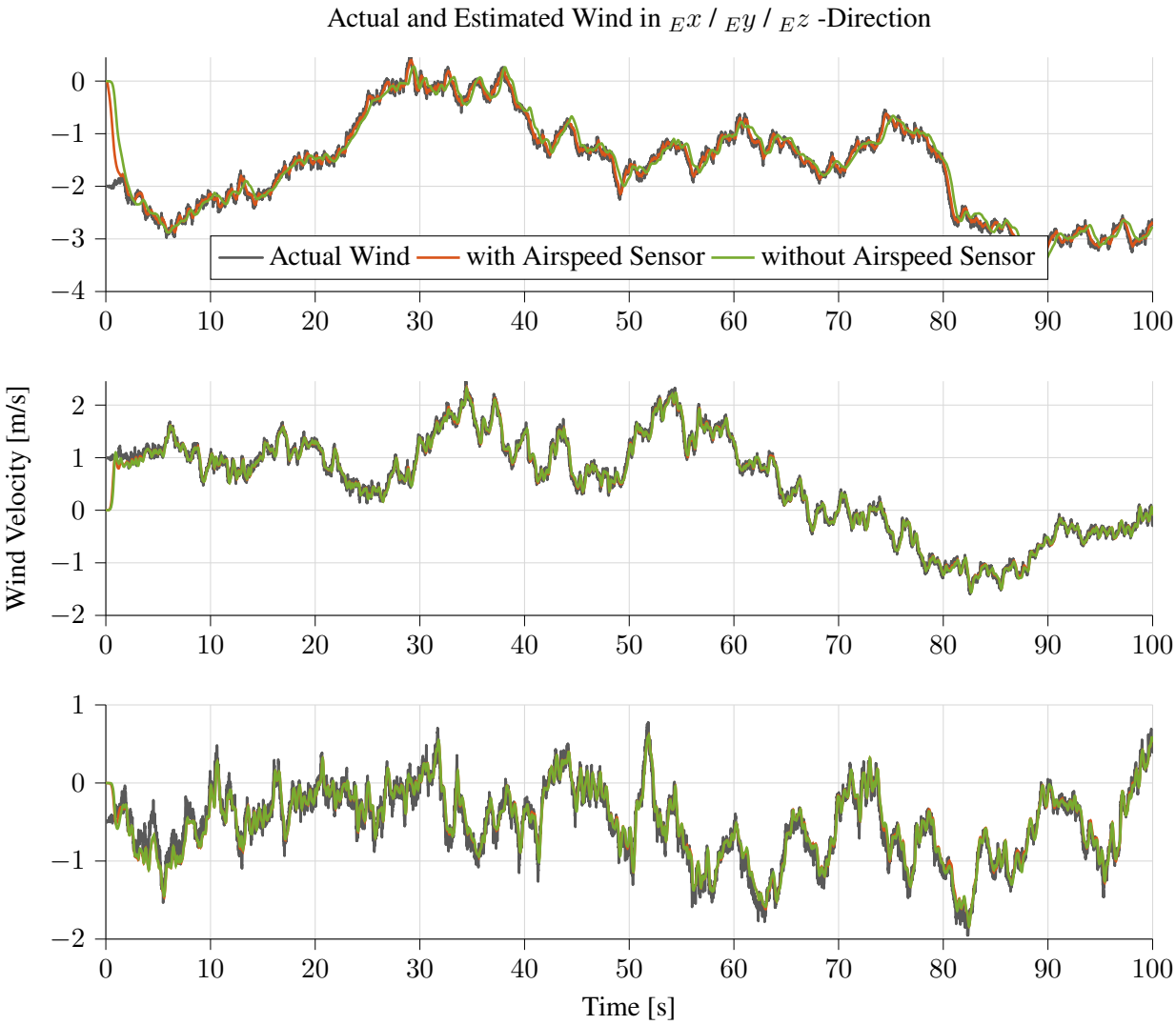


Figure 5.9.: Comparison of EKF wind estimation with and without the 1D airspeed measurement for the fast forward flight scenario with $\boldsymbol{v}_{W,\text{const}} = [-2\ 1\ -0.5]\text{m/s}$ and $W_{20,\text{true}} = 5\,\text{m/s}$.

The comparison indicates that including the 1D airspeed measurement yields only a marginal improvement in wind-estimation performance. This effect is primarily visible in the wind estimate along the x-axis, as this is the direction in which the airspeed sensor provides additional information in this case. Notably, the representative test case shown here (fast forward flight) is among the scenarios that benefit the most from the airspeed measurement, since the airspeed in the x-direction is typically above $2\,\text{m/s}$ in this flight mode, allowing the sensor to provide valid measurements throughout most of the maneuver. To investigate the effect of the airspeed sensor measurement across all test scenarios, the complete EKF and UKF test

suite described in Section 5.1.2 is repeated without the airspeed measurement, and the resulting RMSE values are compared against those obtained with the complete measurement set. The results are given in Table 5.2.

Table 5.2.: Comparison of mean wind-estimation RMSE, averaged over all scenarios and wind conditions, with and without the 1D airspeed measurement, considering only converged test cases from the complete EKF and UKF test suite.

	With Airspeed Sensor	Without Airspeed Sensor
UKF	0.3473	0.3510
EKF	0.3180	0.3573

The results show that the wind estimation performance remains good even without the airspeed measurement, indicating that the remaining measurements provide sufficient information for the estimation of the wind states. The 1D airspeed measurement therefore provides additional information, but is not essential for achieving good wind estimation performance under the investigated conditions. This limited impact can be attributed to the restricted functionality of the airspeed sensor: it only provides a measurement when the airspeed in the x-direction exceeds $2\,\mathrm{m/s}$, and even then, it captures only a single axis of the wind vector. Given these constraints, the airspeed measurement contributes comparatively little information to the overall estimation problem, which is consistent with the small performance difference observed between the two scenarios.

5.1.5. Mean Wind Computation

As discussed in Section 3.1.2, the wind is represented in the augmented state by the five Dryden filter states, without introducing a separate state for the mean wind component. While this representation is sufficient for wind estimation, an explicit estimate of the mean wind can be useful for subsequent applications in which the slowly varying wind component is of interest independently of the turbulent fluctuations. Therefore, in addition to the wind estimate provided by the Kalman filter, a separate mean-wind estimate is computed from the estimated wind velocity.

Two approaches are considered for this purpose. The first approach uses a low-pass filter to suppress the higher-frequency turbulent fluctuations in the estimated wind. The cutoff frequency of the low-pass filter is chosen to reflect the time scale on which the mean wind is expected to vary, e.g., due to gradual changes in the ambient wind field or flight altitude. A cutoff of

$$\omega_c = 0.02\,\mathrm{rad/s} \quad \Leftrightarrow \quad \tau_c = \frac{2\pi}{\omega_c} \approx 314\,\mathrm{s} \approx 5.2\,\mathrm{min} \tag{5.3}$$

is selected, corresponding to a time constant of approximately $5.2\,\mathrm{min}$. This ensures that the low-pass filter isolates only the slowly varying, quasi-constant component of the wind. The cutoff frequency is applied identically to all three wind components.

The second approach uses a moving average over a fixed time window. The estimated wind is averaged over the specified window, with the window length determining the degree of attenuation of the turbulent fluctuations. Analogous to the cutoff frequency of the low-pass filter, the window length $T = 300\,\mathrm{s}$ is chosen to reflect the time scale on which the mean wind is expected to vary, and can be adjusted individually for different flight scenarios or estimated wind conditions.

Both approaches generally result in different mean-wind estimates due to their different filtering characteristics. The moving average is particularly attractive for hardware implementation due to its simple structure, and its window length provides an intuitive and easily adjustable tuning parameter. The low-pass filter offers comparable ease of tuning through its cutoff frequency, while additionally providing a smoother, continuously updated estimate rather than one based on a finite history of past samples. In principle, either parameter, the window length or the cutoff frequency, could be adapted online based on the current wind conditions; such an adaptive implementation is not considered in this work, and fixed values are used for both methods instead.

Figure 5.10 compares the long-term estimation results for the two methods.

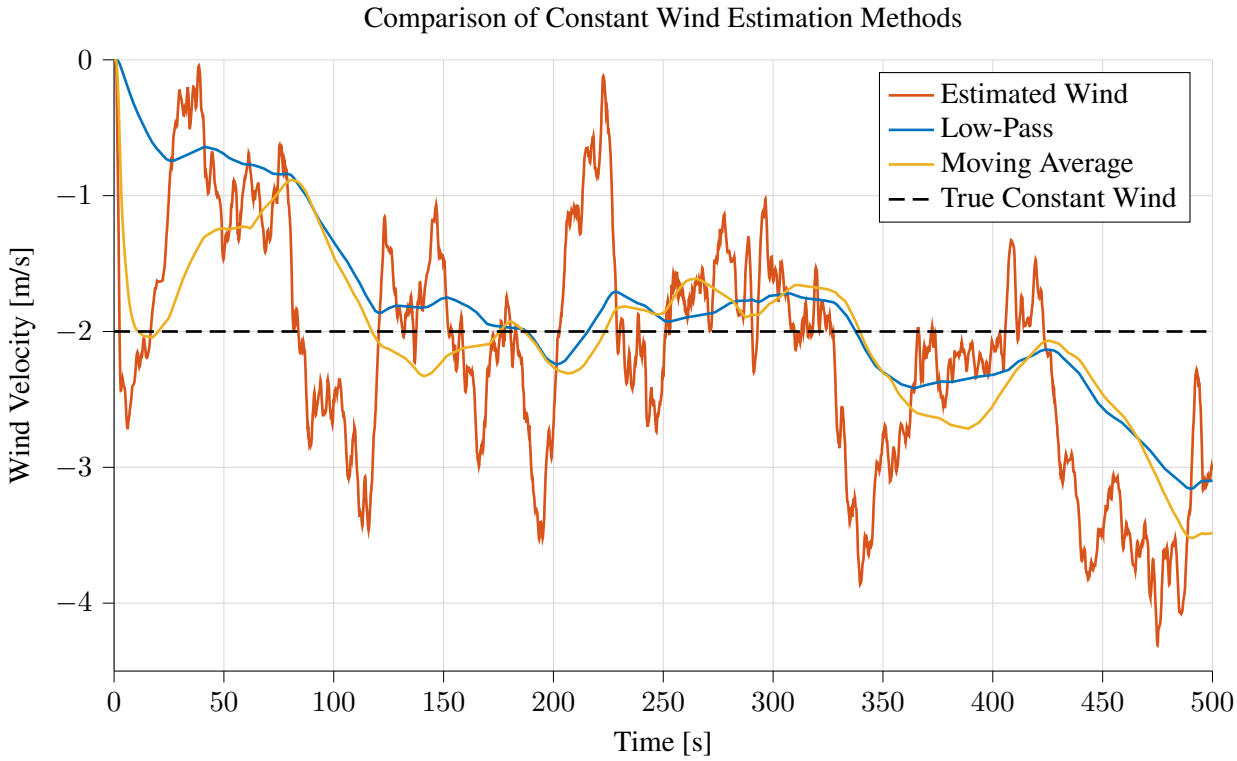


Figure 5.10.: Comparison of the true constant wind component with estimates obtained by applying low-pass filtering and a moving average to the wind estimate from the Kalman filter, over a 500 s flight.

5.2. Data Driven Wind Estimation

In this section, the wind estimation performance of the trained networks from Chapter 4 is evaluated. The evaluation is based on the test data from the simulated data set introduced in Section 4.2, which covers the same flight and wind scenarios that were used to evaluate the EKF and UKF in the preceding section. It comprises $N = 486$ simulation runs, obtained by combining six flight scenarios with 27 constant wind vectors and three turbulence levels $W_{20,\text{true}} \in \{0, 5, 10\}\text{m/s}$ of the Dryden turbulence model.

As described in Chapter 4, seven networks of each type (MLP and LSTM) were trained on different data splits to assess the generalization performance. In Experiment A, the networks are tested on 72 simulation runs that were not used during training, drawn at random from each combination of flight scenario and turbulence level. All six flight scenarios are therefore part of the training data, and the experiment assesses

how well the networks generalize to unseen combinations of flight scenarios and wind conditions. In Experiment B, one flight scenario is held out completely in each of the six splits B1–B6, and all 81 runs of this scenario form the test set. This experiment assesses how well the networks generalize to flight scenarios that were not part of the training data, and is expected to be the more demanding of the two. In the following, the performance of the respective networks is evaluated.

As the networks estimate the wind velocity in body-fixed coordinates, the estimates are compared with the true body-fixed wind of the simulation. Similar to the evaluation of the Kalman filter, the RMSE is calculated separately for each axis and then averaged over the three axes, which yields a single performance metric. The network outputs are transformed back to physical units before the evaluation, so that all errors are given in $\mathrm{m/s}$.

Figure 5.11 shows the wind estimates of the MLP and the LSTM networks trained on the Experiment A split for one representative test run, together with the true wind in body-fixed coordinates.

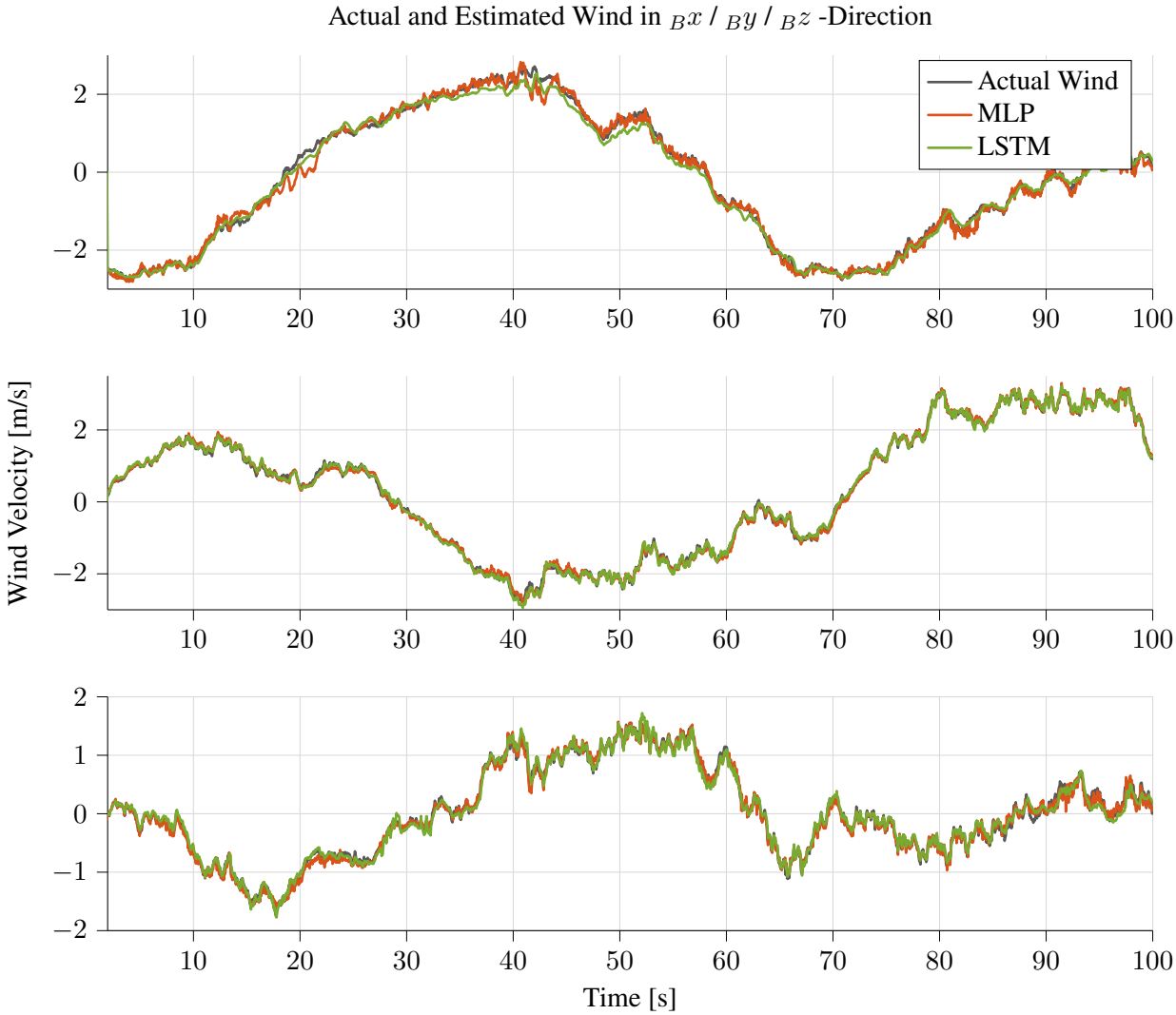


Figure 5.11.: Comparison of MLP and LSTM wind estimation for the rotation flight scenario with $\boldsymbol{v}_{W,\text{const}} = [-2.5\ 0\ 0]\mathrm{m/s}$ and $W_{20,\text{true}} = 10\,\mathrm{m/s}$.

The results indicate that both networks produce a good wind estimate for an unseen test run. For this run, the estimation performance of the MLP and the LSTM is similar, with average RMSE values of $0.1082\,\mathrm{m/s}$ for the MLP and $0.1024\,\mathrm{m/s}$ for the LSTM. The LSTM estimate is slightly less oscillatory, which is likely

due to the temporal context of its input sequence: the LSTM processes the last 2 s of data, whereas the MLP only sees the current time step. As this result is based on a single run, it is not sufficient to draw general conclusions. Therefore, the evaluation is extended to all test runs of the seven data splits, in order to assess whether the networks generalize to unseen test runs of known flight scenarios (Experiment A) and to completely unseen flight scenarios (Experiment B). Figure 5.12 shows the resulting average RMSE of the MLP and the LSTM over all test runs for each of the seven data splits.

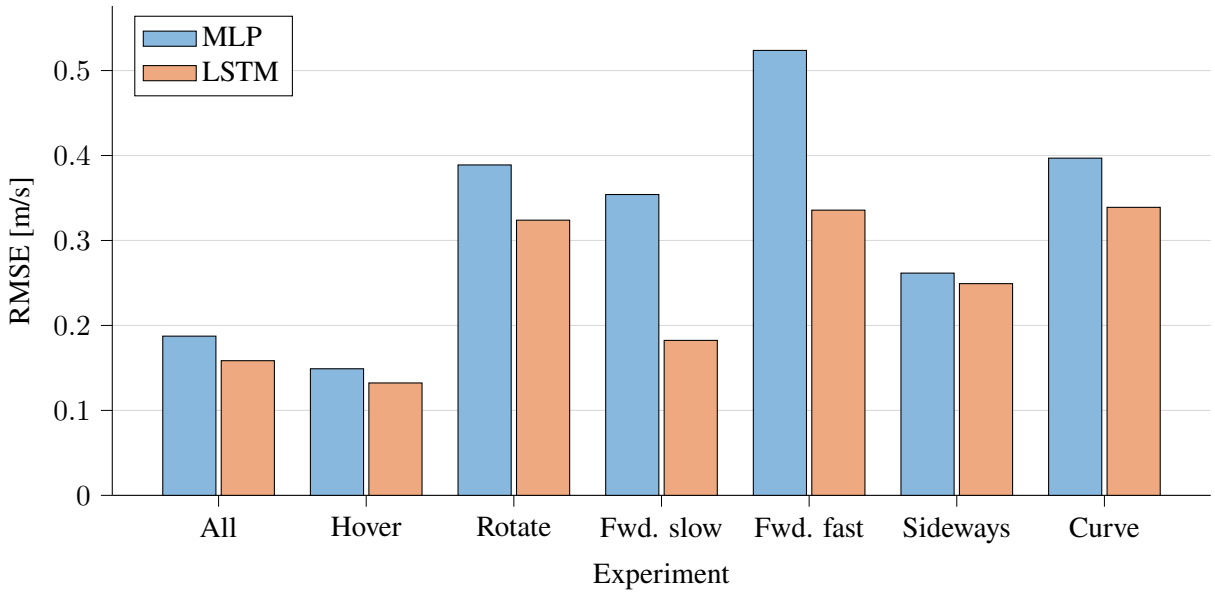


Figure 5.12.: Comparison of the RMSE of the estimated wind for all seven experiments. The x-axis indicates the scenario on which the networks are tested. All denotes Experiment A, in which all scenarios are used for training. Each of the remaining labels denotes one run of Experiment B, in which the named scenario is excluded from the training data and used for testing.

The average RMSE of the MLP stays below $0.6\,\mathrm{m/s}$ and that of the LSTM below $0.4\,\mathrm{m/s}$ in all seven data splits. Both networks therefore estimate the wind well, not only for unseen runs of known flight scenarios (Experiment A), but also for flight scenarios that were not part of the training data (Experiment B).

The LSTM achieves a lower RMSE than the MLP in every split. The difference is largest for the held-out scenarios rotate, forward slow, and forward fast, in which the temporal information of the input sequence seems to be particularly helpful.

The lowest error of both networks is obtained for the split in which the hover scenario is held out during training. This should not be interpreted as a better generalization of the networks when trained without hover, but rather reflects a property of the test data. In the Dryden turbulence model, the turbulence depends on the absolute velocity of the airship and vanishes for $V \approx 0$, so that the wind in hover is almost constant and thus easier to estimate.

Experiment A achieves the second lowest errors, with both networks far below $0.2\,\mathrm{m/s}$. In this experiment, all flight scenarios are covered by the training data, and the networks only have to generalize to unseen combinations of wind conditions and flight scenarios. In contrast, the error is considerably higher in all splits of Experiment B except hover, in which one flight scenario is missing in the training data. This highlights the importance of training data that covers the relevant flight scenarios, and confirms that the generalization to unseen flight scenarios is the more demanding task.

The ability to generalize to unseen wind conditions and flight scenarios is an important property for real flight applications, as it cannot be ensured that the training data covers every situation the airship will encounter during operation. The low errors in Experiment B indicate that the networks do not merely memorize the flight scenarios of the training data, but learn a relation between the vehicle's motion and the acting wind that transfers to new maneuvers. At the same time, the higher errors compared to Experiment A show that this transfer is not perfect, and that a training data set covering the relevant operating conditions remains beneficial. It should be noted that these results are obtained on simulated data, and that the generalization to real flight data, which is subject to additional effects such as sensor noise and unmodeled dynamics, has yet to be demonstrated.

The following section examines the robustness of the trained networks to noisy inputs and compares their performance with that of the EKF. This comparison focuses on noisy data, as the EKF is designed to estimate the system state from uncertain measurements by accounting for measurement noise and process uncertainty. Evaluating the EKF on noise-free data would therefore not reflect the conditions for which it is designed.

5.2.1. Robustness to Noisy Inputs

So far, the networks have been evaluated using noise-free simulation data. In real flight, however, sensor measurements are affected by noise, while unmodeled dynamics and other effects can introduce further discrepancies between the simulation and the real system. The robustness of the estimators to noisy inputs is therefore important for determining their potential for real-flight application. To investigate this, noise is added to the test data to perturb the simulated measurements, while the trained networks remain unchanged. This experiment examines how well the networks generalize from clean training data to noisy inputs.

Every component $x_{j,t}$ of the input vector $\boldsymbol{x}_t$ is perturbed by zero-mean Gaussian noise at different noise levels according to:

$$x_{j,t}^{\text{noisy}} = x_{j,t} + \varepsilon_{j,t}, \qquad \varepsilon_{j,t} \sim \mathcal{N}\big(0, (\nu\,\sigma_{x,j})^2\big), \qquad j = 1, \ldots, 24. \tag{5.4}$$

Here, $\sigma_{x,j}$ denotes the standard deviation of the respective input component in the training set, as defined in (4.2), and $\nu \in \{0, 0.01, 0.05, 0.10, 0.20\}$ specifies the noise level. The case $\nu = 0$ corresponds to noise-free inputs and reproduces the results from the previous section. The remaining values correspond to noise levels of 1%, 5%, 10%, and 20% of the standard deviation, respectively.

The noisy input vectors are obtained by stacking their respective components:

$$\boldsymbol{x}_t^{\text{noisy}} = [x_{1,t}^{\text{noisy}}, \ldots, x_{24,t}^{\text{noisy}}]^T. \tag{5.5}$$

For each noise level, the average RMSE across all test runs is computed as before, with the network predictions transformed back to physical units. The robustness analysis focuses on the networks from Experiment A, as they are trained on all flight scenarios and achieve the lowest errors on noise-free data, making them the most relevant candidates for practical use.

Figure 5.13 compares the resulting RMSE of the MLP, LSTM, and EKF across the different noise levels. The EKF uses the same noisy test inputs as the neural networks but additionally incorporates position measurements. The estimators also differ in several aspects: the neural networks run at 20 Hz, whereas the EKF runs at 100 Hz; the EKF error excludes the initial transient phase (the first 20 s); and the RMSE of

the MLP and LSTM is computed from the body-fixed wind estimate and true wind, while that of the EKF is computed from the Earth-fixed estimate and true wind. Nevertheless, the RMSE provides a common basis for comparing their estimation accuracy.

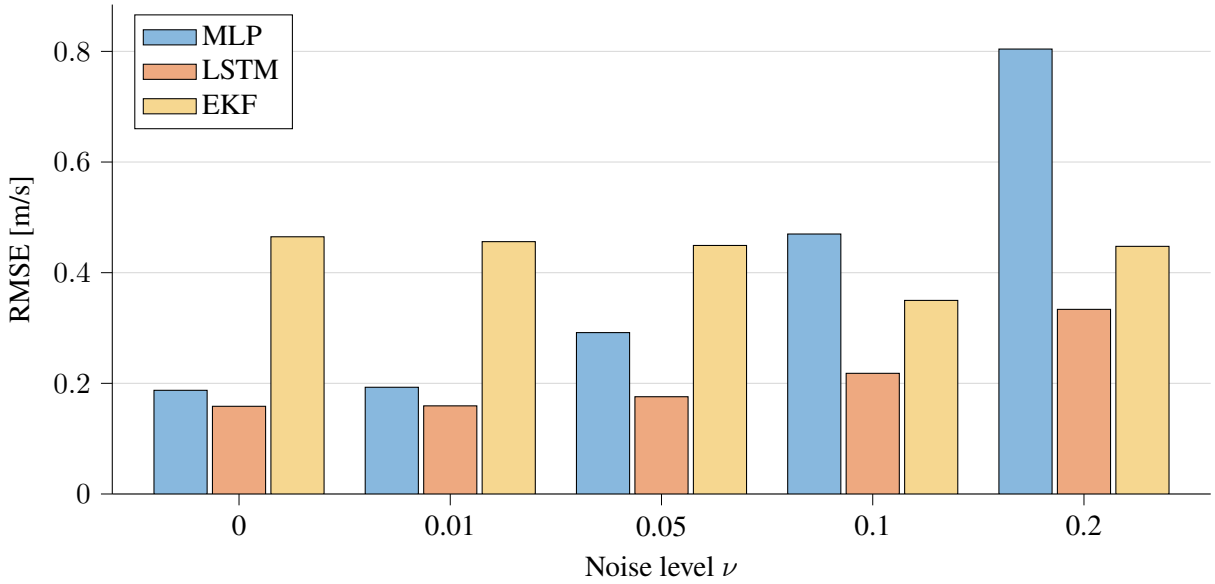


Figure 5.13.: Comparison of wind estimation RMSE for MLP, LSTM and EKF for different noise levels.

At low noise levels, both neural networks substantially outperform the EKF. This is consistent with the networks being trained on noise-free simulation data, allowing them to learn the relationship between the inputs and the wind components under these conditions. The EKF, in contrast, explicitly accounts for measurement and process uncertainty in its estimation.

As the noise level increases, the EKF's RMSE remains nearly constant, indicating that its estimation accuracy is largely unaffected by the added measurement noise over the tested range. In comparison, the performance of both neural networks degrades notably at higher noise levels. This is expected, as the networks were trained on noise-free data and were not explicitly trained to handle noisy inputs.

The LSTM, however, is considerably more robust than the MLP at noise levels of $\nu \geq 0.05$. By exploiting temporal information across consecutive input samples, the LSTM can use patterns over time rather than relying solely on individual measurements. This temporal context may help mitigate the effect of noise, explaining its better performance compared with the MLP. Moreover, the RMSE of the LSTM remains below that of the EKF at all noise levels, which makes the LSTM a promising candidate for wind estimation in the presence of measurement noise.

5.2.2. Computational Cost

Besides the estimation accuracy, the computational cost of the estimators is relevant for the application on the airship's flight-control hardware, where computation time and memory are limited. This subsection therefore compares the cost of the MLP and the LSTM with that of the EKF and, as a reference, the UKF.

The runtime is measured in the same way as in the comparison of the two filters: each estimator is evaluated separately as an isolated implementation with identical input data, an initial simulation is performed to reduce the influence of initialization effects, and the subsequent 100 simulations are used to determine the

mean runtime per step. For the networks, one step corresponds to one prediction, i.e., a single forward pass of the MLP or the evaluation of one input sequence of $T = 40$ time steps for the LSTM. The results are given in Table 5.3.

Table 5.3.: Mean computational runtime per step of the EKF, UKF, MLP, and LSTM. The results are obtained from 100 simulation runs with a simulated duration of $1000\,\mathrm{s}$. One step is performed every $0.01\,\mathrm{s}$. The normalized runtime is given relative to the EKF.

Estimator	Mean runtime per step [ms]	Normalized runtime [-]
EKF	0.153	1.00
UKF	0.296	1.93
MLP	0.022	0.14
LSTM	0.686	4.48

The MLP is the cheapest estimator and requires only about one seventh of the runtime of the EKF per step, whereas the LSTM requires about 4.5 times as much as the EKF and about 30 times as much as the MLP. As before, the absolute values are not representative of the flight-control hardware, so the results should be interpreted as a relative comparison.

The differences can be explained by the number of operations per step. A prediction of the MLP consists of a single forward pass with four matrix-vector products, which amounts to a number of multiply-accumulate operations of the order of 10^4. The LSTM, in contrast, evaluates the four gate computations of both layers for each of the $T = 40$ time steps of the input sequence, which results in about $2 \cdot 10^6$ operations per prediction, i.e., roughly 280 times as many as for the MLP. For the EKF, the cost per step is determined by $n + 1 = 18$ evaluations of the nonlinear state-transition function, $n = 17$ of which are used for the numerical Jacobian, and by matrix operations on the 17×17 covariance matrix. As the cost of the model evaluations depends on the complexity of the airship model, an operation count cannot be compared directly with that of the networks.

Regarding the memory requirements, the MLP has 7 939 and the LSTM 56 003 trainable parameters. Stored in single precision ($4\,\mathrm{B}$ per value), the weights occupy about $32\,\mathrm{kB}$ for the MLP and about $224\,\mathrm{kB}$ for the LSTM, which is about seven times as much. The LSTM additionally requires a buffer holding the last $T = 40$ input vectors and memory for its hidden and cell states, whereas the EKF only stores its 17-dimensional state vector, the 17×17 covariance matrix, and the model parameters. The flight controller considered here, a Cube Orange, provides $2\,\mathrm{MB}$ of flash memory and $1\,\mathrm{MB}$ of RAM. While the MLP is clearly small enough, it is questionable whether the LSTM, which would take up about a fifth of the available RAM if its weights are held there, is acceptable considering the memory already occupied by the other flight-control software.

Overall, both networks generalize to unseen flight scenarios and remain functional on noisy inputs, although the LSTM is clearly more robust than the MLP, which is likely due to the temporal context of its input sequence. In the investigated simulation setting, both networks are competitive with the EKF: they reach lower errors on noise-free data, and the LSTM stays below the EKF error at all tested noise levels, while the MLP falls behind at higher noise levels. The EKF, in turn, offers the advantages of interpretability and a lower memory demand, as it is based on a physical model and only stores its state and covariance. Its computational cost per step lies between that of the MLP and the LSTM. The performance of the networks, however, depends strongly on the quality and volume of the training data, and since the networks are trained and tested on data from the same simulator, they benefit from similar conditions during training, while the EKF is designed to account for uncertainties it has not seen before. The comparison with the EKF therefore has to be interpreted with care. A conclusion on the practical potential of the data-driven estimators requires a test on real flight data, which is carried out in Chapter 6.

5.3. Wind compensation

Building on the wind-estimation results from the previous sections, this section investigates the influence of wind compensation methods on the tracking performance. The estimated wind velocity is used by the two compensation approaches, disturbance feedforward (DFF) and nonlinear dynamic inversion (NDI), which are both implemented as complementary components to the PID controller according to the structure introduced in Section 3.2. The PID controller without compensation therefore serves as the baseline, while DFF and NDI are evaluated in combination with the PID controller. The objective is to identify a controller configuration that provides consistently good performance across all investigated flight scenarios.

Since the compensation approaches explicitly account for a portion of the wind-induced disturbances, the PID gains can be adjusted to take advantage of the additional disturbance rejection. To investigate the influence of the controller tuning, three different PID gain sets are considered for each compensation approach. The original PID gains are retained as the Robust gain set. These gains are tuned for the standalone PID controller with an emphasis on achieving robust tracking performance across all investigated flight scenarios. The Aggressive gain set is obtained by scaling all PID gains by a factor of two while maintaining their relative values, resulting in a more responsive controller tuning. Finally, the Compensated (Comp) gain set is specifically tuned using the no-wind model and is therefore optimized for tracking performance in the absence of wind disturbances. This results in seven distinct controller configurations: the PID baseline and three gain settings each for DFF and NDI.

To compare the control performance across the different compensation approaches and gain settings, all configurations are evaluated using the same test suite. The six flight scenarios described at the beginning of this section are considered. For each flight scenario, the same set of nine constant wind contributions is applied, while the true turbulence intensity is varied through $W_{20,\text{true}} \in \{0, 5, 10\}$ m/s. This results in 27 unique wind conditions per flight scenario. Across all seven configurations, this results in a total of $N = 1134$ simulation runs. Using identical flight scenarios and wind conditions for all configurations ensures that differences in control performance can be attributed to the compensation approach and PID gains.

The wind estimate required by the compensation is provided by the UKF in all tests, as these tests were carried out before the comparison of the EKF and UKF was completed. Since all compensation approaches use the identical estimate, the choice of the estimator does not influence their relative comparison.

For the evaluation of the tracking performance, all 12 airship states, consisting of position, velocity, attitude, and angular rate, are considered. For each state, the RMSE is computed with respect to the corresponding reference trajectory, i.e., the reference position, velocity, attitude, and angular rate. In contrast to the wind-estimation evaluation, where the wind components share the same physical unit, these state variables differ both in their units and magnitudes. Their RMSE values can therefore not be directly averaged to obtain a meaningful overall measure of control performance, as states with larger numerical magnitudes would dominate the resulting metric. To obtain a single overall performance metric for each controller/gain combination, the RMSE values are processed in four steps.

First, for each combination of flight scenario and controller/gain configuration, the RMSE of each of the 12 airship states is averaged across the 27 wind conditions considered for that scenario. This yields one representative RMSE value for each state, scenario, and controller/gain configuration, denoted $\text{RMSE}_{x,s,c}$.

Second, since these RMSE values still differ in unit and magnitude across the 12 states, they cannot yet be combined into a single metric. To make them comparable, the values are normalized separately for each scenario and state. For a given scenario and state, the RMSE values obtained across all seven controller/gain configurations are averaged to obtain a reference value. Each individual RMSE value is then divided by this reference:

$$\widetilde{\text{RMSE}}_{x,s,c} = \frac{\text{RMSE}_{x,s,c}}{\frac{1}{7}\sum_{c'=1}^{7} \text{RMSE}_{x,s,c'}}. \tag{5.6}$$

This yields a dimensionless value that expresses the performance of a given controller/gain configuration relative to the average across all configurations, for the respective scenario and state. A value below one indicates better-than-average performance, while a value above one indicates worse-than-average performance.

Third, since the normalized values are dimensionless and share a common reference, they can be meaningfully combined across the 12 state variables by averaging them,

$$\text{Scenario Performance Metric}_{s,c} = \frac{1}{12}\sum_{x=1}^{12} \widetilde{\text{RMSE}}_{x,s,c}. \tag{5.7}$$

The result is a single measure of the relative performance of each controller/gain configuration within a given flight scenario, referred to as the Scenario Performance Metric. As with the individual normalized RMSE values, values below one indicate better-than-average performance, while values above one indicate worse-than-average performance.

Finally, the Scenario Performance Metric is averaged across the six flight scenarios to obtain the Overall Performance Metric for each controller/gain configuration:

$$\text{Overall Performance Metric}_{c} = \frac{1}{6}\sum_{s=1}^{6} \text{Scenario Performance Metric}_{s,c}. \tag{5.8}$$

This metric is used to compare the PID, NDI, and DFF control architectures under the Robust, Aggressive, and Comp gain settings. Table 5.4 lists the resulting Overall Performance Metrics.

Table 5.4.: Overall Performance Metric for each controller architecture and gain setting. Values below one indicate better-than-average, and values above one worse-than-average, tracking performance.

Method	Gains	Overall Performance Metric
NDI	Robust	0.8309
DFF	Aggressive	0.9012
NDI	Aggressive	1.0278
DFF	Robust	1.0320
DFF	Comp	1.0405
PID	Robust	1.0643
NDI	Comp	1.1033

The Overall Performance Metric shows the best performance for the NDI method with the Robust gain set. These gains were originally developed for the standalone PID controller with an emphasis on achieving robust tracking performance across all investigated flight scenarios. In contrast, the Comp gain set, which is tuned under no-wind conditions, yields by far the worst performance among the considered gain sets for both DFF and NDI. This behavior can be explained by the fact that neither compensation method completely eliminates the effects of the wind disturbances. The Comp gain set is tuned in the absence of such disturbances and therefore provides insufficient robustness when these residual disturbances are present. Consequently, its tracking performance decreases considerably under the investigated wind conditions, and the Comp gain set is not considered further.

Figure 5.14 provides a visual impression of the tracking behavior of the PID baseline and the Robust and Aggressive gain sets for DFF and NDI. The figure shows the resulting trajectories a representative test run for the Curve scenario, illustrating the differences in trajectory tracking between the considered controller configurations.

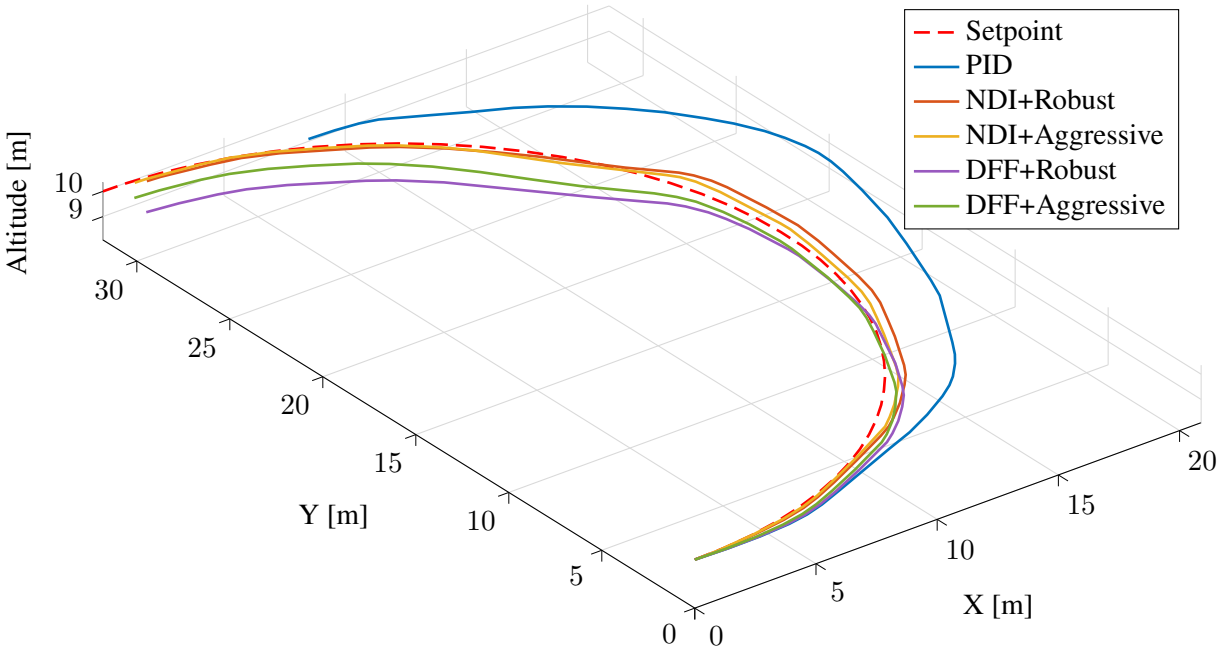


Figure 5.14.: Comparison of the trajectories for the PID baseline and the DFF and NDI compensation approaches with Robust and Aggressive gain settings for a representative Curve scenario test run.

The results show a clear improvement over the baseline PID controller for this specific test run. The Overall Performance Metric shows the NDI with the Robust gain set as the best-performing configuration across the investigated conditions. However, the overall metric alone does not reveal how consistently this performance is achieved across the individual flight scenarios. Therefore, the controller configurations are subsequently compared using the Scenario Performance Metric separately for each of the six flight scenarios. This allows the strengths and weaknesses of the different compensation approaches and gain settings to be identified. The Scenario Performance Metric (SPM) for the controller/gain configurations are shown in Fig. 5.15.

The NDI with the Robust gain set achieves the best or second-best Scenario Performance Metric in five of the six flight scenarios. This consistently strong performance demonstrates that the NDI-Robust configuration provides a robust overall solution across the different flight conditions, rather than achieving its performance primarily through optimization for individual scenarios. The exception is hover, where the NDI-Robust configuration performs considerably worse.

The comparatively poor performance of NDI-Robust in hover can be attributed to two effects. First, NDI relies more strongly on an accurate wind estimate than DFF, as the estimated wind is directly used to determine the compensation through the nonlinear dynamic inversion. Since the wind estimation performance is generally worse in hover, as discussed in section 5.1.2, the resulting wind-estimation error has a stronger impact on the NDI compensation and consequently on the tracking performance.

Second, the Aggressive gain set performs comparatively well in hover. In this flight condition, the tracking deviations are generally smaller and the airship is subject to less dynamic excitation than during translational or rotational motion. Consequently, the disadvantages typically associated with the higher gains,

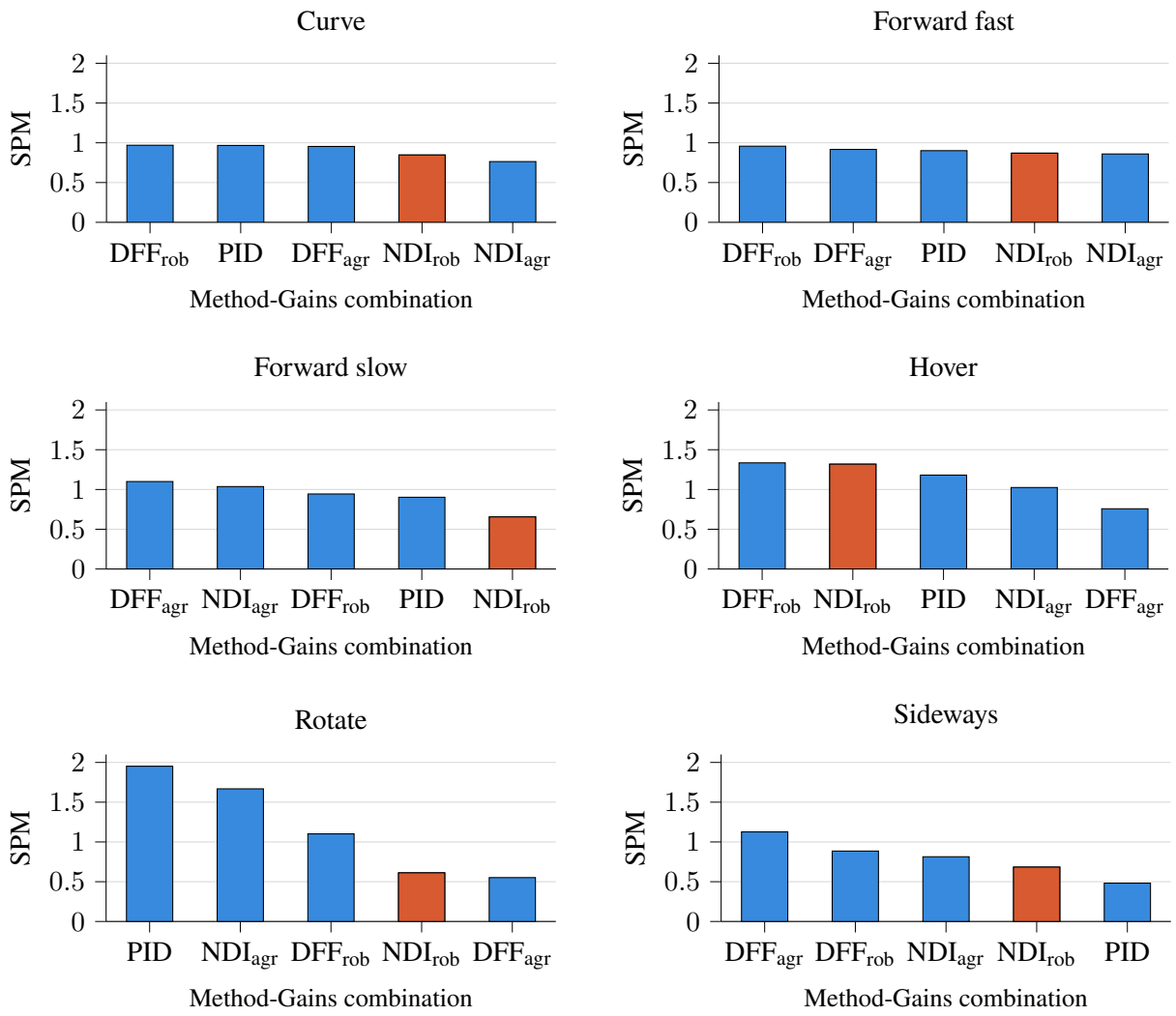


Figure 5.15.: Comparison of the Scenario Performance Metric across selected controller/gain combinations for each flight scenario, sorted from highest to lowest. The orange bar highlights the NDI controller with Robust gains.

such as increased oscillations or sensitivity to disturbances, are less pronounced in hover. The higher gains can therefore improve the tracking performance without introducing the same degree of adverse effects observed in the other flight scenarios.

The comparatively lower Scenario Performance Metric of NDI with Robust gains in hover does not necessarily indicate poor absolute tracking performance in this flight condition. Rather, it indicates that its performance is worse relative to the other controller/gain configurations considered for hover. To illustrate this distinction, Fig. 5.16 shows the RMSE values for the NDI with Robust gains across the six flight scenarios. The RMSE values are grouped according to the four categories of airship states – position, velocity, attitude, and angular rate – allowing the absolute tracking errors in the different flight scenarios to be compared directly. The figure shows that the tracking errors during hover are generally not large in absolute terms, despite the comparatively lower Scenario Performance Metric. This highlights that the metric primarily captures the relative performance of a controller configuration within each flight scenario, rather than its absolute tracking accuracy across different scenarios.

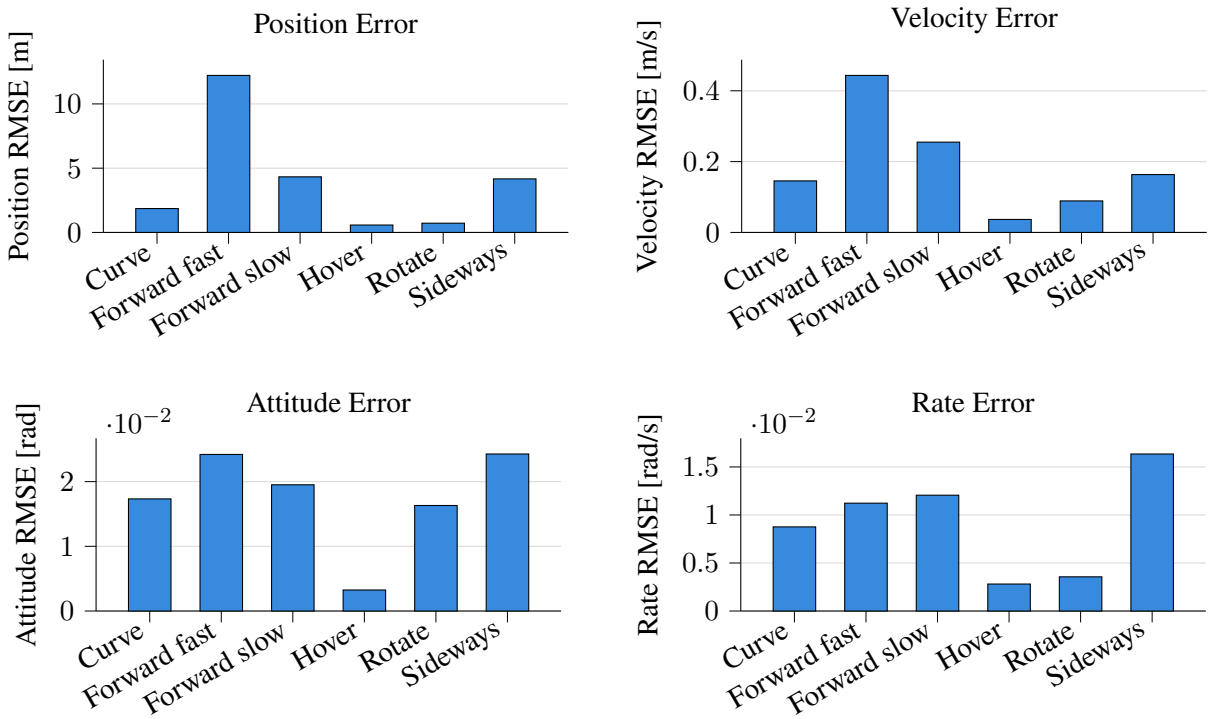


Figure 5.16.: Grouped errors (position, velocity, attitude, and angular rate) for the NDI controller with Robust gains across all evaluated flight scenarios.

Overall, the results show that the choice of compensation method and PID tuning has a significant influence on the tracking performance. Among the investigated configurations, NDI with the Robust gain set provides the most consistent performance across the different flight scenarios, achieving the best or second-best result in all scenarios except hover. The comparatively weaker performance in hover is mainly attributed to the increased influence of wind-estimation errors on the NDI compensation and the favorable behavior of higher gains in this flight condition. Overall, NDI with the Robust gain set is therefore identified as the most suitable configuration for achieving consistent tracking performance across the investigated flight conditions.

6. Evaluation with Real Flight Data

The simulation results show the potential of the data-driven estimators, but a conclusion on their practical use requires data from a real flight. This chapter therefore evaluates the EKF and the trained networks on data of a real test flight and validates their wind estimates with a dedicated wind sensor.

6.1. Test Flight Setup

6.1.1. Test Flight and Logged Data

The test flight consisted of two flights of about 15 minutes each. The weather conditions were favorable, with no to light wind, mainly from easterly directions, and light gusts. During the flights, different maneuvers were carried out, which correspond to the flight scenarios of the simulation: rotation about the airship's own axis, hovering motions, circuits combining curved and straight flight, and sideways flight .

state whether forward flight speeds are comparable to the simulated ones

The airship is equipped with the sensor suite described in Section 1.3.1. The signals of the flight controller were logged during the flights with the rates given in Table 6.1.

Table 6.1.: Logged signals of the test flights.

Signal	Frame	Rate [Hz]
Position, velocity, acceleration	Earth-fixed	125
Attitude, angular rate	Body	250
Angular acceleration	Body	100
Motor inputs	–	10
Wind sensor	Body-fixed	10

6.1.2. Wind Reference Measurement

To validate the wind estimates, the airship is equipped with an FT602-D-SM wind sensor . The sensor is mounted on a rod about $1.25\,\mathrm{m}$ below the center of gravity and measures the aerodynamic velocity in the horizontal plane at $10\,\mathrm{Hz}$, given as magnitude and direction, with the direction ranging from 0 to 2π. As the sensor is two-dimensional, only the horizontal wind components can be validated.

measurement principle, range, accuracy, cite

6.2. Offline Evaluation Procedure

The wind estimation algorithms could not yet be implemented on the flight hardware, so they were not executed during the test flights. Consequently, the compensation methods of Chapter 5, which rely on the wind estimate, could not be tested in flight either. However, as all signals required by the estimators

were logged, the estimates can be reconstructed offline by replaying the logged data and compared with the measurements of the wind sensor. This allows the estimation accuracy to be assessed on real data, but not the behavior in closed loop or the real-time capability on the flight hardware.

All logged signals are first aligned to a common time grid and then resampled to the rate of the respective estimator, 100 Hz for the EKF and 20 Hz for the networks. The accelerations and angular accelerations are low-pass filtered , as is done on the PX4 hardware, so that the estimators receive the signals as they would be available onboard. The input vector of the networks is assembled as in simulation, i.e., without position and pitot tube measurement, and standardized with the training statistics of the networks from Experiment A. For this purpose, the velocity and the accelerations are transformed from the Earth-fixed to the body-fixed frame using the logged attitude. The motor inputs are available at 10 Hz and are held constant between samples to obtain the rate of 20 Hz.

cutoff

The wind sensor measures the aerodynamic velocity at the sensor location. The measured magnitude and direction are converted to the components $\boldsymbol{v}_{a,s}^B$ in the body-fixed frame. The aerodynamic velocity at the center of gravity follows from the angular rate $\boldsymbol{\omega}^B$ and the sensor position $\boldsymbol{r}_s^B$, and the reference wind is obtained by subtracting it from the airship's velocity,

$$\boldsymbol{v}_a^B = \boldsymbol{v}_{a,s}^B - \boldsymbol{\omega}^B \times \boldsymbol{r}_s^B, \qquad \boldsymbol{w}_{\text{ref}}^B = \boldsymbol{v}^B - \boldsymbol{v}_a^B, \tag{6.1}$$

where only the horizontal components are used.

Check signs and the conversion of the sensor direction, which is given counterclockwise as seen from above.

Comparison. The networks estimate the wind in the body-fixed frame and can be compared directly with $\boldsymbol{w}_{\text{ref}}^B$. The EKF estimate, given in the Earth-fixed frame, is transformed into the body-fixed frame using the logged attitude. The estimates are evaluated at the time stamps of the wind sensor . The error is quantified by the RMSE of the two horizontal axes, averaged as in the previous chapters, and the initial convergence phase of the estimators is excluded .

interpolation, time synchronization

window

7. Conclusion and Outlook

Overall, the comparison of the wind estimation performance demonstrates that both the EKF and UKF provide comparable wind estimation performance for the investigated airship operating conditions. Although the UKF avoids the explicit model linearization required by the EKF, this does not result in a significant reduction in estimation error within the considered test range. The observed differences are comparatively small and depend on the individual scenario rather than consistently favoring one filter. Consequently, the EKF provides a suitable alternative to the UKF for the subsequent analyses, while offering the advantage of a lower computational effort associated with the propagation of a single linearized state model instead of multiple sigma points.

Outlook

The model-based approach proves to be a powerful tool for wind estimation given a good model of the system is available. The Dryden wind model does not only provide a mathematical description for the wind speeds acting at the center of gravity but also yields a description for the effect of the wind on the angular rates, which are caused by differences in wind magnitudes within the prevailing wind field.

Including the angular rates induced by the differences in wind speeds within the wind field can improve the performance as the airship is especially vulnerable to wind fields considering the airship's large side surfaces. Angular rates of the wind are described by transition functions of order 2 (for the u-direction) and order 3 (for v, w direction) resulting in a increase of one order for every direction compared to wind speeds. It remains to investigate whether the gain in performance can justify the increased complexity.

The Wind estimation proved to be particulary reliable. This information can be used for advanced control and trajectory planning algorithms like MPC to increase the performance compared to the current PID setup.

Increase performance for hovering

Split up estimation in turbulence and mean wind to correctly use the mean wind CS for the Dryden turbulence.

A. First Appendix

Write your appendix content here.

Table A.1.: Modeling Coefficients and constants

Coefficient	**Symbol**	**Value**	**Unit**
Motor Coefficient	C_{motor}	x	y
alphas	α	x	in N!!!

Table A.2.: Measurement noise standard deviations used for the Kalman filter.

Measurement	Unit	Standard deviation σ_η
Position x	m	1
Position y	m	1
Position z	m	1
Velocity u	m/s	0.15
Velocity v	m/s	0.15
Velocity w	m/s	0.25
Roll angle ϕ	°	1
Pitch angle θ	°	1
Yaw angle ψ	°	2.5
Angular velocity p	°/s	0.5
Angular velocity q	°/s	0.5
Angular velocity r	°/s	0.5

B. Second Appendix

B.1. W20

$W_{20,\text{true}}$	$W_{20,\text{ass}}$	Case	RMSE_x	RMSE_y	RMSE_z	$\overline{\text{RMSE}}$
0.0	0.0	match	1.6667	0.6667	0.3333	0.8889
	15.0	best	0.3844	0.1603	0.1233	0.2227
1.6	1.6	match	0.5006	0.2634	0.2329	0.3323
	15.0	best	0.3918	0.1885	0.1501	0.2434
5.0	5.0	match	0.4597	0.3414	0.3393	0.3801
	15.0	best	0.4599	0.3292	0.3094	0.3662
8.3	8.3	match	0.7350	0.6243	0.6192	0.6595
	15.0	best	0.6801	0.5593	0.5784	0.6059

Table B.1.: Mean RMSE of wind estimation error signals per ($W_{20,\text{true}}$, $W_{20,\text{ass}}$) pair. For each $W_{20,\text{true}}$ the matching case ($W_{20,\text{ass}} = W_{20,\text{true}}$) and the case yielding the lowest overall RMSE are shown.

B.2. Wind estimation

Table B.2 shows the difference in the performance for converged and non converged data.

Table B.2.: Overall wind-estimation RMSE across all test runs, for the full flight trajectory and for the converged time window ($t = 20$–$100\,\text{s}$).

	UKF			EKF		
	Mean	Std	Median	Mean	Std	Median
Full trajectory	0.423	0.358	0.329	0.375	0.342	0.282
Converged	0.347	0.371	0.219	0.318	0.362	0.205

Data for the wind estimation for different W20 barchart in Table B.3

Table B.4 shows the data for the boxplot

Table B.3.: Wind-estimation RMSE by turbulence intensity W_{20}, computed on the converged window ($t = 20$–100 s).

	UKF			EKF		
W_{20}	Mean	Std	Median	Mean	Std	Median
0 (none)	0.239	0.282	0.130	0.190	0.228	0.108
5 (light)	0.314	0.329	0.200	0.281	0.321	0.192
10 (moderate)	0.509	0.446	0.363	0.506	0.446	0.355

Table B.4.: Wind-estimation RMSE by flight scenario, computed on the converged window ($t = 20$–100 s).

	UKF			EKF		
Scenario	Mean	Std	Median	Mean	Std	Median
hover	0.605	0.317	0.520	0.471	0.216	0.491
forward_slow	0.276	0.304	0.196	0.258	0.302	0.174
forward_fast	0.282	0.281	0.198	0.296	0.305	0.228
sideways	0.249	0.326	0.174	0.237	0.343	0.160
curve	0.435	0.563	0.239	0.398	0.559	0.202
rotate	0.245	0.265	0.173	0.258	0.359	0.166